Topic Title: Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

Student Name: Jordan Mohindra **Student ID:** z5418455

A. **Problem statement**

High-resolution cascaded MIMO radars produce more raw data than a vehicle's network and central processor can comfortably absorb. Compressing at the sensor saves link bandwidth, but the compressor must be lossless (anything it discards is lost before target detection), keep up with the sensor, and be small enough for sensor hardware. Radar compression methods are rarely compared on the same real multi-transmitter data, and the lossless baseline was evaluated on a single-transmitter radar with no decoder.

B. **Objective**

Compare compression methods side by side on real, public, multi-transmitter radar data; select the best lossless methods; and implement their encoders and decoders on an FPGA, verified and measured on a board.

C. **My solution**

Eight methods were evaluated in MATLAB on 50 frames of the ColoRadar dataset (twelve-transmitter Texas Instruments cascade). The two lossless predictive coders, DRHE and a slow-time LPC, were adapted to time-division MIMO by predicting from the same transmitter, given Huffman dictionaries re-derived for their prediction errors, written in C++ for Vitis HLS with matching decoders, and built into a test system on an AMD KCU116 board, where their output was verified bit-exact against simulation.

D. **Contributions** (at most one per line, most important first)

Bit-exact lossless radar compression on an FPGA: 4.22× (DRHE), 4.40× (LPC)
Same-transmitter prediction lag for time-division MIMO radar
Comparison of eight compression methods on real cascade radar data
Breakdown of the compression ratio; Huffman dictionaries matched to the errors
Hardware encoders and decoders measured on the board (time, latency, resources)
Reconstruction of, and comparison with, the baseline design

E. **Suggestions for future work**

Fixed-point DRHE datapath to cut logic and DSP use
Streaming (backward-adaptive) LPC, and a parallel LPC decoder
All sixteen receive channels, a range-FFT core on chip, and power measurement

While I may have benefited from discussion with other people, I certify that this report is entirely my own work, except where appropriately documented acknowledgements are included.

Signature: _____ Date: _____ / _____ / _____

Pointers

List relevant page numbers in the column on the left. Be precise and selective.

14	Problem statement
16	Objective

Theory (up to 5 most relevant ideas)

22	Time-division MIMO radar and the data cube
24	Lossless compression and entropy coding
27	Fixed-point (FX16) quantisation
28	FPGAs and high-level synthesis
33	The baseline: Kiem's DRHE design

Method of solution (up to 5 most relevant points)

43	Eight candidate methods
44	Shared S_4/Huffman entropy coder
50	Prediction on a TDM-MIMO radar
68	Verification ladder, MATLAB to board
72	Hardware implementation

Contributions (most important first)

88	Bit-exact lossless compression on the FPGA
84	Same-transmitter prediction lag
81	Comparison of eight methods
85	Breakdown of the compression ratio; matched dictionaries
89	Encoders and decoders measured on the board

My work

58	Slow-time LPC coder
60	Re-derived Huffman dictionaries
77	Hardware decompressors
79	KCU116 board test system

Results

81	Succinct presentation of results
94	Analysis
94	Significance of results

Conclusion

101	Statement of whether the outcomes met the objectives
103	Suggestions for future research

Literature (up to 5 most important references)

33	[11] L. Kiem, master's thesis (the baseline design)
26	[12] A. Kramer *et al.*, ColoRadar dataset
24	[15] D. A. Huffman, minimum-redundancy codes
24	[14] K. Sayood, *Introduction to Data Compression*



UNSW
AUSTRALIA

SCHOOL OF ELECTRICAL ENGINEERING
AND TELECOMMUNICATIONS

Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

by

Jordan Mohindra

Student ID: z5418455

Thesis submitted as a requirement for the degree

Bachelor of Engineering (Electrical Engineering)

Submitted: November 2026
Supervisor: A/Prof. Beena Ahmed

Abstract

High-resolution cascaded MIMO radar produces more raw data than a vehicle's network and central processor can comfortably absorb. Compressing that data at the sensor buys back bandwidth, but only if the compressor is lossless, fast and small, because anything it discards is lost before target detection ever sees it.

This thesis compares data compression methods for automotive radar on real data and implements the most suitable of them in hardware. Eight methods were evaluated in MATLAB on 50 frames of the public ColoRadar dataset, recorded with a twelve-transmitter Texas Instruments cascade radar. The lossy quantisers reached compression ratios of up to 3.76 but missed up to 28.7 % of targets, and floating-point storage barely compressed at all. Two lossless predictive coders compressed by more than three: Doppler Redundancy Hybrid Encoding (DRHE), which predicts magnitude and phase with fixed filters, and a second-order linear predictive coder (LPC) whose coefficients are fitted to each frame. On a time-division MIMO radar both must predict each ramp from the previous ramp of the *same transmitter*, twelve ramps back; predicting from the adjacent ramp does worse than not predicting at all.

The compression ratio was broken down into its sources. Huffman coding of the 16-bit range-FFT data alone achieves 3.48; prediction adds a further 0.22–0.27; and a Huffman dictionary re-derived for the prediction errors adds 0.47 for DRHE and 0.71 for LPC, giving 4.22 and 4.40 overall, measured on the hardware, with every frame reconstructed exactly.

Both compressors and their decompressors were written in C++, synthesised with Vitis HLS and Vivado, and run on an AMD KCU116 (Kintex UltraScale+) board at 100 MHz. On the board all 50 frames were compressed byte-identically to the simulation, and the board's own decoders reconstructed every frame exactly. DRHE compresses a frame in 0.61 ms (5.1 Gbit/s) and streams its output with 13 µs of latency. LPC takes 3.25 ms and, because its coefficients depend on the whole frame, cannot output anything until the whole frame has arrived: its last output word follows its last input word by 3.0 ms. The decoders take 1.97 ms (DRHE) and 15.4 ms (LPC) per frame. Encoder and decoder together use 23 % (DRHE) and 25 % (LPC) of the chip's logic. LPC compresses about 4 % better; DRHE is recommended for a streaming sensor front end, because it needs no frame buffer and has a small fraction of LPC's latency.

Acknowledgements

I would like to thank A/Prof. Beena Ahmed for her guidance throughout this project, and Dr. Mostafa Shahin for his assessment and feedback. I am also indebted to Lukas Kiem, whose master's thesis provided the algorithmic baseline this work builds on; it was specified carefully enough to be reimplemented and tested exactly, which made much of this thesis possible.

Abbreviations

ADAS Advanced Driver Assistance Systems

ADC Analog-to-Digital Converter

AR Autoregressive

ASIC Application-Specific Integrated Circuit

AXI Advanced eXtensible Interface

BAQ Block Adaptive Quantization

BRAM Block Random Access Memory

CR Compression Ratio

C-sim C-Simulation (the HLS C++ run as software)

DDR Double Data Rate (memory)

DMA Direct Memory Access

DRHE Doppler Redundancy Hybrid Encoding

DSP Digital Signal Processing / DSP slice

EGE Exponential-Golomb Encoding

FDR False Discovery Rate

FF Flip-Flop

FFT Fast Fourier Transform

FMCW Frequency-Modulated Continuous Wave

FN False Negative

FP16 16-bit (half-precision) floating point

FPGA Field-Programmable Gate Array

FX16 16-bit fixed-point (integer) representation

HLS High-Level Synthesis

IF Intermediate Frequency

IIR Infinite Impulse Response

II Initiation Interval

I/Q In-phase / Quadrature

LPC Linear Predictive Coding

LSB Least Significant Bit

LUT Look-Up Table

MIMO Multiple-Input Multiple-Output

MSB Most Significant Bit

RDVLE Range-Dependent Variable-Length Encoding

RTL Register-Transfer Level

RX Receive / Receiver

SerDes Serialiser/Deserialiser

SNR Signal-to-Noise Ratio

SoC System-on-Chip

TDM Time-Division Multiplexing

TX Transmit / Transmitter

UDRE Uniform Dynamic-Range re-Encoding

WNS Worst Negative Slack

Contents

Acknowledgements 1

Abbreviations 2

Contents 4

List of Figures 9

List of Tables 11

1 Introduction 14

1.1 The problem . . . 14
1.2 Motivation . . . 14
1.3 Why the compressor must be lossless . . . 15
1.4 Aim and objectives . . . 16
1.5 Contributions . . . 17
1.6 Thesis outline . . . 18

2 Background 19

2.1 How an FMCW radar measures range . . . 19
2.2 Angle, MIMO and the virtual array . . . 21
2.3 The radar data cube, and why it is large . . . 23
2.4 Lossless compression: how it actually works . . . 24
2.5 The sensor and the dataset . . . 26
2.6 Number representation and quantisation . . . 26
2.7 FPGAs and high-level synthesis . . . 28
2.8 Related work and the baseline . . . 30

2.9 The baseline: Kiem's thesis . . . 32
2.10 Summary of what the later chapters assume . . . 32

3 The Baseline: Kiem's Thesis 33
3.1 The problem he set himself . . . 33
3.2 His simulation workflow . . . 34
3.3 His data . . . 35
3.4 His design space . . . 36
3.5 His design-space result . . . 37
3.6 How he adapted DRHE for hardware . . . 38
3.7 His final compression scheme . . . 39
3.8 His hardware . . . 39
3.9 What he concluded, and what he left open . . . 41
3.10 What the comparison takes from this chapter . . . 41

4 Compression Methods 43
4.1 The eight candidate methods . . . 43
4.2 The shared entropy coder: turning residuals into bits . . . 44
4.3 Predicting on a time-division MIMO radar . . . 50
4.4 DRHE: Doppler Redundancy Hybrid Encoding . . . 56
4.5 LPC: linear prediction with Yule–Walker coefficients . . . 58
4.6 The Huffman dictionary . . . 60
4.7 The two algorithms side by side . . . 61

5 Evaluation Method 63
5.1 Two stages . . . 63
5.2 The reference signal chain . . . 64
5.3 Metrics, defined precisely . . . 66
5.4 The verification ladder . . . 68
5.5 Edge-case testing . . . 69
5.6 An independent implementation . . . 70
5.7 Reproducing the baseline . . . 70
5.8 Tools . . . 71

6 Hardware Implementation **72**

6.1 Common interface and bit packing . . . 72
6.2 DRHE compressor . . . 73
6.3 LPC compressor . . . 74
6.4 Exactly lossless on any input . . . 76
6.5 Decompressors . . . 77
6.6 The board test system . . . 79
6.7 Building and timing closure . . . 80

7 Results **81**

7.1 Stage 1: screening eight methods . . . 81
7.2 Stage 2: the prediction axis . . . 84
7.3 Where the compression comes from . . . 85
7.4 The re-derived dictionary . . . 86
7.5 Frame-to-frame variation . . . 87
7.6 Verification . . . 88
7.7 Speed and latency on the board . . . 89
7.8 Chip resources . . . 91
7.9 The two algorithms side by side . . . 93

8 Discussion **94**

8.1 Which algorithm for a sensor front end? . . . 94
8.2 Scaling to larger frames . . . 95
8.3 What the compression ratio measures . . . 96
8.4 How far the dictionary and the lag generalise . . . 97
8.5 Hardware cost against the baseline design . . . 98
8.6 Comparison with the baseline's results . . . 98
8.7 Observations on verifying a compressor . . . 99
8.8 Threats to validity . . . 100

9 Conclusions and Future Work **101**

9.1 Summary . . . 101
9.2 Findings . . . 101
9.3 Future work . . . 103

Bibliography **105**

A Interpreting the Hardware Numbers **108**

A.1 Two ways of counting the same circuit . . . 109
A.2 Where the numbers come from, and how far to trust them . . . 113
A.3 The yardsticks . . . 114
A.4 Rebuilding the reference design . . . 116
A.5 From this thesis's datapath to Kiem's, one change at a time . . . 118
A.6 Latency: three different things with the same name . . . 131
A.7 Throughput and initiation interval . . . 137
A.8 Resources against the device . . . 140
A.9 Power . . . 141
A.10 Compression ratio: what counts as good . . . 143
A.11 Every difference from Kiem, accounted for . . . 144

B Comparison with the Baseline **147**

B.1 Three kinds of comparison . . . 148
B.2 Rebuilding Kiem's simulation . . . 148
B.3 Results at Kiem's noise level . . . 151
B.4 The noise level at which DRHE gives 3.25 . . . 152
B.5 Compression ratio across noise levels . . . 153
B.6 This thesis's chain on synthetic scenes . . . 155
B.7 Kiem's noise floor and his compression ratio do not agree . . . 156
B.8 LPC-Huffman, the method Kiem excluded . . . 157
B.9 Adding transmitters . . . 159
B.10 The DRHE algorithm, line by line . . . 160
B.11 What the replication can and cannot show . . . 164
B.12 Kiem's hardware, rebuilt . . . 164
B.13 Every result, compared . . . 164
B.14 The compression ratio: 3.25 and 3.75 measure different things . . . 164
B.15 Run-length coding . . . 167
B.16 The fixed dictionary . . . 167
B.17 Linear prediction . . . 168

B.18 Detection metrics and the noise floor . . . 169
B.19 What the comparison says . . . 170

C The Huffman Dictionaries **171**

D Bitstream Layouts **173**

D.1 DRHE . . . 173
D.2 LPC . . . 173

E Reproducing the Results **174**

E.1 Source files . . . 174
E.2 Commands . . . 175

F Verification Record **176**

List of Figures

1.1 Raw versus compressed radar transport 15

2.1 FMCW chirp and beat frequency 20
2.2 TDM-MIMO ramp interleaving 23
2.3 Radar data cube and processing chain 23
2.4 Two-stage lossless compressor 25

4.1 The sixteen samples, plotted 52
4.2 Bit budget by category 56
4.3 DRHE block diagram 58

5.1 One input frame 65
5.2 The verification ladder 68

6.1 LPC three-pass architecture 74
6.2 The KCU116 test system 79

7.1 Compression against missed targets 82
7.2 Compression ratio, step by step 85
7.3 Bits needed per value 86
7.4 Compression ratio of each frame 88
7.5 Time per frame on the FPGA 90
7.6 FPGA resources 92

8.1 Where the bits go 96

A.1 HLS estimate against implemented LUTs, per module 112
A.2 Resources at each step from DRHE-1 to the replica 119

A.3 Pipeline schedule of one sample 123
A.4 Latencies on one time axis 132
A.5 Throughput capacity against requirement 139

B.1 The rebuilt scene 150
B.2 CR against noise floor on Kiem's scene 154

List of Tables

2.1 ColoRadar cascade capture parameters used throughout this thesis. 27

3.1 Kiem's use case 34
3.2 Kiem's data 36
3.3 Kiem's design-space results 38
3.4 Kiem: DRHE without run-length coding 38
3.5 Kiem's hardware results 40

4.1 Raw values against residuals 46
4.2 S4 category ranges 47
4.3 Category 3 enumerated 48
4.4 Sixteen samples traced through both predictors 51
4.5 Why the two lags differ 53
4.6 Bit budget by category 55
4.7 DRHE and LPC compared 62

5.1 Synthetic edge cases 70

6.1 The four board builds 80

7.1 Stage 1: the eight methods 81
7.2 Quantisation study 83
7.3 The prediction lag 84
7.4 Where the compression comes from 85
7.5 What the re-derived dictionary changes 86
7.6 How close the coder gets to the entropy 87
7.7 Measured timing on the board 89

7.8 Resources per core . . . 91
7.9 DRHE and LPC compared . . . 92

8.1 Hardware against the baseline . . . 98

A.1 HLS estimates against implemented results . . . 110
A.2 Where each hardware number comes from . . . 113
A.3 Sensor and system yardsticks . . . 115
A.4 The replica against Kiem's published design . . . 118
A.5 From DRHE-1 to the replica, one change per step . . . 119
A.6 How the predictor state is stored . . . 120
A.7 Where the DRHE compressor's DSP slices go . . . 121
A.8 Pipeline depth and what sets it . . . 123
A.9 Where DRHE-1's LUTs and flip-flops go . . . 126
A.10 Testing explanations for the gap to Kiem's LUT and FF counts . . . 128
A.11 Routed clock of each design . . . 130
A.12 Throughput: measured, theoretical and required . . . 138
A.13 Resources as a fraction of two devices . . . 140
A.14 Power and energy of the implemented cores . . . 142
A.15 Every difference from Kiem's design: cause and evidence . . . 145

B.1 Replication: stages and sources . . . 149
B.2 Replication at −75 dBFS . . . 152
B.3 Replication at −87.9 dBFS . . . 153
B.4 FX16 scaling and noise floor . . . 155
B.5 What Kiem's dictionary was built for . . . 157
B.6 LPC-Huffman of Meucci and Mancuso by order . . . 158
B.7 One against twelve transmitters . . . 159
B.8 DRHE audit against Kiem's specification . . . 161
B.9 Kiem's results against this thesis's . . . 165
B.10 Kiem's dictionary against a re-derived one . . . 168
B.11 The LPC question in one table . . . 168

C.1 The original Huffman dictionary . . . 171

C.2 The re-derived dictionaries 172

F.1 Verification record 176

Chapter 1

Introduction

1.1 The problem

A modern high-resolution automotive radar generates data faster than the vehicle it is mounted on can conveniently move it. The cascaded radar used in this thesis produces about 3 MiB of raw digitised samples every frame, and it is one of several such sensors on a car. Getting that data to the central processor means either expensive shielded high-speed cabling or compressing it at the sensor first. Compression is the cheaper answer, but it has to keep up with the sensor, fit in a small amount of silicon, and — because this data feeds a system that decides whether to brake — lose nothing. This thesis compares the available compression methods on real radar data, selects the ones that meet those conditions, and builds and measures them on an FPGA.

1.2 Motivation

Radar earns its place in a vehicle's sensor suite because it works when the other sensors do not. Cameras fail in fog, glare and darkness; lidar is degraded by rain and airborne dust; radar sees through all of them [1, 2]. That robustness is why radar has been in production vehicles for two decades, initially for simple distance warning and now for adaptive cruise control, automatic emergency braking, blind-spot detection and lane-change assistance [3].

What has changed recently is resolution. Supporting SAE Level 2+ autonomy needs more than "something is 40 m ahead". It needs a map: how many objects, where, how fast, how wide, and what they probably are. Angular resolution is what separates "an object" from "a car and

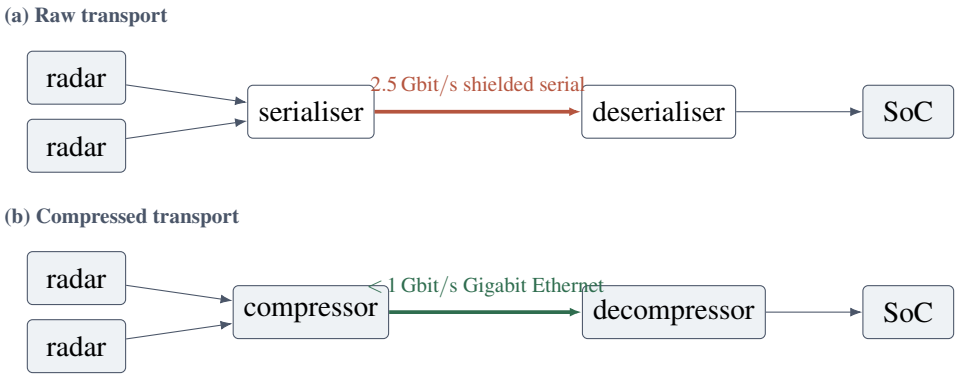


Figure 1.1: The economic argument for sensor-edge compression. Compressing by a factor of 2.5 or better at the radar module replaces an expensive shielded high-speed serial link with standard Gigabit Ethernet, without changing what the central processor receives.

a pedestrian standing beside it", and angular resolution comes from having many antennas. A radar with N_t transmitters and N_r receivers can synthesise an effective array of $N_t \times N_r$ elements, so resolution improves roughly with the product. The data volume, unfortunately, scales the same way.

At the same time, vehicle electrical architectures have been consolidating. Where a car once had a hundred small electronic control units each doing one job, the industry has moved to domain controllers, then to *zonal* architectures where sensors report to a nearby gateway, and toward fully centralised designs where one high-performance system-on-chip does everything [4, 5, 6]. Centralising the computation has obvious advantages, but it means the raw sensor data has to travel.

Figure 1.1 shows the consequence. A Serialiser/Deserialiser (SerDes) link [7, 8] carries the aggregated output of several radar modules at 2.5 Gbit/s or more. Those data rates are achievable, but they require shielded cable, they cost money and weight, and they put a matching ingest burden on the SoC's memory system [9]. If instead the data is compressed at the radar module by a factor of 2.5 or better, the same information fits inside Gigabit Ethernet [10] — a cheap, standard, unshielded interconnect. That single substitution is the economic argument for this entire body of work.

1.3 Why the compressor must be lossless

It is tempting to relax the requirement. Lossy compression reaches much higher ratios, and radar data is noisy anyway, so surely a little extra noise is harmless?

It is not, and the reason is specific rather than philosophical. The compressor sits *upstream* of detection. Whatever it discards is discarded before the detector ever sees it, and the detector's job is to find weak returns close to the noise floor — a pedestrian at range, a motorcycle between two trucks. Lossy compression raises the noise floor precisely where the margin is thinnest. Chapter 7 quantifies this on real data: Block Adaptive Quantization reaches the same compression ratio as the lossless coders studied here but misses nearly three times as many real detections and reports nearly nine times as many false ones.

Losslessness also makes verification unambiguous. The pass criterion is that every reconstructed sample equals its original exactly. There is no tolerance to argue about and no distortion metric to trade off: a single differing bit anywhere in the 157 million input bits of the test set is a failure, and the test reports it as one. Every design in this thesis is checked against that criterion, from MATLAB through C-simulation and RTL co-simulation to the encoders and decoders running on the FPGA board.

1.4 Aim and objectives

The starting point is the master's thesis of Lukas Kiem [11], which evaluated a range of radar compression schemes on recordings from a single-transmitter Infineon radar, selected Doppler Redundancy Hybrid Encoding (DRHE) as the best of them, and implemented its compressor on a Zynq UltraScale+ device (compression ratio 3.17 in hardware). Chapter 3 describes that work. Four questions that his work leaves open define the objectives of this thesis.

Aim. Compare compression methods side by side on real, public, multi-transmitter radar data; select the best *lossless* methods; and implement both their encoders and their decoders on an FPGA, measured on a real board.

1. **Validate on public MIMO data.** Kiem's results come from private recordings of a single-transmitter radar. This thesis uses 50 consecutive frames of the public ColoRadar dataset [12], recorded with a Texas Instruments twelve-transmitter, sixteen-receiver cascade, for every result from simulation to hardware.
2. **Examine the quantisation step.** Kiem identified the conversion to 16-bit fixed point (FX16) as the source of his detection errors. This thesis measures what that step costs in detection, and whether floating-point storage is a usable alternative.
3. **Broaden the comparison.** Eight methods are compared on the same data with the same

metrics: two reference formats, three lossy quantisers, DRHE on integer and on floating-point data, and a linear predictive coder (LPC) designed for this thesis as a controlled comparison with DRHE.

4. **Implement and measure the complete pipeline.** Kiem implemented compression only and measured it on his own system. This thesis implements compression *and* decompression for both selected algorithms, verifies them bit-exactly from MATLAB to the board, and measures throughput, latency and resources on an AMD KCU116 board.

1.5 Contributions

1. **A comparison of eight compression methods on real cascade radar data** (Section 7.1). Only the two predictive lossless coders, DRHE (3.75) and LPC (3.70), compress by more than three without losing detections; the lossy quantisers reach up to 3.76 but miss 10 % to 29 % of targets, and floating-point storage compresses by only 1.04.
2. **The correct prediction axis for time-division MIMO radar** (Section 4.3). On a TDM cascade consecutive ramps come from different transmitters, so slow-time prediction must use the previous ramp of the same transmitter, n_{Tx} ramps back. Predicting from the adjacent ramp, as single-transmitter descriptions of these algorithms do, compresses worse than not predicting at all.
3. **A breakdown of where the compression comes from** (Sections 7.3 and 8.3). Huffman coding alone achieves 3.48 on this data — mostly from unused fixed-point headroom and the sparsity of a range-FFT frame — prediction adds 6 % to 8 %, and the Huffman dictionary decides the rest.
4. **Huffman dictionaries matched to the prediction errors** (Section 7.4). Re-derived from five frames, they raise DRHE to 4.22 and LPC to 4.40, hold on the 45 frames they were not trained on, and reverse the ranking of the two algorithms.
5. **Complete, verified hardware for both algorithms** (Chapters 6 and 7). Encoders and decoders were implemented in Vitis HLS and Vivado and run in a board test system on an AMD KCU116. Compressed output is byte-identical to the simulation on all 50 frames with both dictionaries, the hardware decoders reconstruct every frame exactly, and throughput, latency and resources are measured for both halves.

6. **A comparison with the baseline** (Appendices A and B), including a reconstruction of Kiem's simulation from his text, which reaches his simulated DRHE ratio at one plausible noise level and explains how his results relate to the ones here.

1.6 Thesis outline

Chapter 2 gives the necessary background: how an FMCW radar measures range and velocity, how MIMO and time-division multiplexing produce a virtual array, why the data is large, how lossless compression works, the sensor and dataset, number representation, FPGAs and high-level synthesis, and related work. Chapter 3 describes the baseline, Kiem's thesis, in enough detail that every later comparison can be followed without it. Chapter 4 describes the eight compression methods, the entropy coder shared by the two predictive ones, the choice of prediction axis, the two predictors in detail, and the Huffman dictionary. Chapter 5 sets out how they are evaluated: the two-stage evaluation, the signal chain, the metrics and the verification ladder from MATLAB to the board. Chapter 6 describes the hardware architectures of the encoders and decoders and the board test system. Chapter 7 reports the results. Chapter 8 discusses what the results mean for a radar sensor and how they compare with the baseline, and Chapter 9 concludes and proposes future work.

Two appendices carry the detailed comparison with the baseline. Appendix A interprets the hardware numbers — what each one means, how far to trust it, and why each differs from Kiem's design — and Appendix B compares every one of Kiem's published results with this thesis's, using a reconstruction of his simulation. The remaining appendices give the Huffman dictionaries, the bitstream layouts, how to reproduce the results, and the verification record.

Chapter 2

Background

This chapter assumes no prior knowledge of radar. It builds up, in order, what the radar measures, how it measures it, what the resulting data looks like, why there is so much of it, how lossless compression works in general, and what an FPGA can do about it. Each section ends with the specific fact that the later chapters depend on.

2.1 How an FMCW radar measures range

2.1.1 The chirp

Automotive radar is almost universally *frequency-modulated continuous wave* (FMCW). Instead of emitting a short pulse and timing the echo — which would demand impractically fast electronics for the distances involved — an FMCW radar transmits continuously, sweeping its frequency linearly upward over a short interval. One such sweep is called a *chirp*. The sensor used here sweeps 1.798 GHz of bandwidth around a 77 GHz centre frequency while its echo is sampled, and starts a new chirp every 78.5 µs.

When the chirp reflects off an object at range R, the echo arrives delayed by the round-trip time

$$\tau = \frac{2R}{c}, \tag{2.1}$$

where c is the speed of light. Because the transmitter has kept sweeping in the meantime, the echo's frequency at any instant is lower than what is currently being transmitted, by an amount proportional to the delay. If the chirp sweeps B hertz in T_c seconds, the sweep slope is $S = B/T_c$

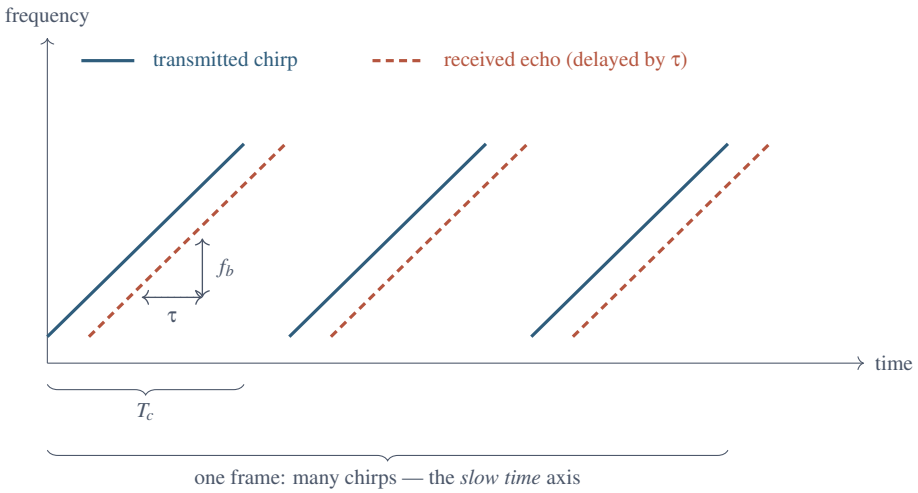


Figure 2.1: FMCW operation. Each chirp sweeps linearly in frequency. The echo is a delayed copy, so at any instant the transmitted and received frequencies differ by $f_b = S\tau$, proportional to target range. A frame consists of many such chirps in succession; the axis running across chirps is called *slow time* and is where Doppler information lives.

and the frequency difference is

$$f_b = S\tau = \frac{2SR}{c}. \tag{2.2}$$

A mixer multiplies the transmitted and received signals, producing a low-frequency *beat* or intermediate-frequency signal at exactly f_b. That signal is low-pass filtered and digitised by an analog-to-digital converter (ADC) at a modest rate — 8 MHz here, rather than the tens of gigahertz the raw RF would need. This is the central trick of FMCW: it converts a timing measurement that would need picosecond resolution into a frequency measurement that an ordinary ADC can make.

2.1.2 From beat frequency to a range profile

Measuring f_b is a spectral estimation problem, so the natural tool is the Fourier transform. Taking an N_s-point FFT across the N_s ADC samples of a single chirp produces a *range profile*: bin n of the FFT corresponds, through Equation (2.2), to range

$$R_n = n \cdot \frac{c}{2B}, \tag{2.3}$$

so the range resolution is

$$\Delta R = \frac{c}{2B} \approx 8.3\,\text{cm} \tag{2.4}$$

for the 1.798 GHz bandwidth used here. A peak in the range profile is an object. This first transform is called the **Range FFT**, and its output is the data this thesis compresses.

2.1.3 Slow time and Doppler

A single chirp gives range but not velocity. To get velocity, the radar transmits many chirps in rapid succession — a *frame* — and looks at how the *phase* of a given range bin evolves from chirp to chirp.

The reasoning is worth spelling out, because the whole of DRHE rests on it. A target at range R produces a range-FFT value whose phase is $\theta = -4\pi R/\lambda$, where λ is the wavelength. Over the chirp-to-chirp interval T, a target moving at radial velocity v changes range by vT, so its phase changes by

$$\Delta\theta = -\frac{4\pi v T}{\lambda}. \tag{2.5}$$

At 77 GHz, $\lambda \approx 3.9\,\text{mm}$, so even a very slow target produces a measurable phase step — which is exactly why radar is good at velocity. Critically for compression, $\Delta\theta$ is *constant* for a target at constant velocity. The phase of a given range bin therefore advances linearly across chirps, and its magnitude barely changes at all. That is a highly predictable sequence, and predictable sequences compress.

The axis running across chirps within a frame is called **slow time**, as opposed to the *fast time* axis of samples within a chirp. Taking a second FFT along slow time resolves $\Delta\theta$ into velocity: this is the **Doppler FFT**.

Carry forward: slow time is where the redundancy is. A target at constant velocity produces a range-bin sequence whose magnitude is nearly constant and whose phase advances by a constant increment. Both DRHE and LPC exist to exploit exactly this.

2.2 Angle, MIMO and the virtual array

2.2.1 Why more antennas

Range and velocity are not enough to drive a car. The radar also has to know *where* a target is in angle, and how well it can separate two targets at the same range and velocity but different bearings.

Angle is measured by comparing the phase of the same echo at physically separated receive antennas. A wavefront arriving at angle ϕ reaches adjacent antennas, spaced d apart, with a path difference $d\sin\phi$, hence a phase difference

$$\Delta\psi = \frac{2\pi d \sin\phi}{\lambda}. \tag{2.6}$$

An FFT across the receive-antenna axis — the **Angle FFT** — turns those phase differences into a bearing estimate. The angular resolution improves with the physical size of the array, which in practice means with the number of elements.

2.2.2 MIMO: buying antennas cheaply

Building a large physical receive array is expensive. Multiple-Input Multiple-Output (MIMO) radar gets the same effect more cheaply. If N_t transmitters and N_r receivers are arranged suitably, and the signal from each transmitter can be separated at each receiver, then each of the $N_t \times N_r$ transmit–receive pairs behaves like an element of a *virtual array*. The sensor used here has $N_t = 12$ and $N_r = 16$, giving 192 virtual elements from 28 physical antennas.

2.2.3 Time-division multiplexing, and why it matters here

The separability requirement has to be met somehow. The simplest method, and the one this sensor uses, is *time-division multiplexing* (TDM): the transmitters fire one at a time, in a fixed rotation, so whatever a receiver hears during transmitter t's slot must have come from transmitter t.

TDM is simple and robust, and it has one consequence that matters throughout this thesis. Inside every repetition of the frame — every "chirp loop" — all N_t transmitters fire in sequence before the loop repeats. When the resulting data is laid out as a two-dimensional array of (ramp, range bin), which is how a streaming compressor sees it, the ramp index is

$$m = qN_t + t, \qquad q \in [0, N_c), \quad t \in [0, N_t), \tag{2.7}$$

where q indexes the chirp loop and t the transmitter. The transmitter index varies *fastest*.

This is the correct physical time order. It is also, as Figure 2.2 makes visual, a trap. Ramp m and ramp $m-1$ are different physical transmitters $N_t - 1$ times out of N_t. They are separated in *space* by the array baseline, not in *time* by a pulse interval, and the phase relationship between them is governed by Equation (2.6) — the target's bearing — not by Equation (2.5) — the target's velocity. The genuine slow-time neighbour of ramp m, the one whose phase differs by a Doppler step, is ramp $m - N_t$. From Chapter 4 on, this distance is written n_{Tx}; on this sensor $n_{\mathrm{Tx}} = N_t = 12$.

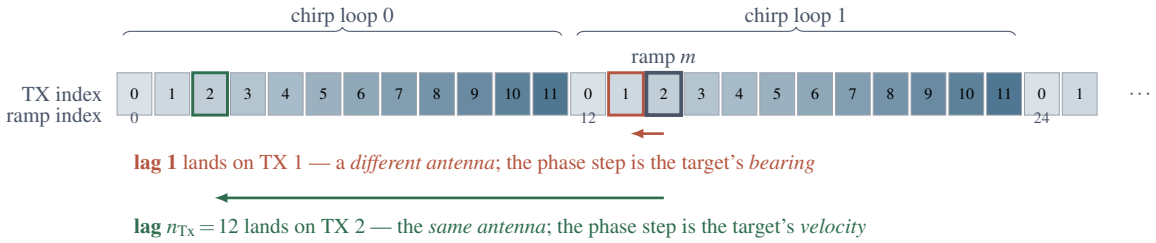


Figure 2.2: Why "the previous ramp" is the wrong neighbour on a TDM-MIMO cascade. Each cell is one ramp, labelled with the transmitter that produced it; the twelve transmitters rotate inside every chirp loop. Predicting ramp m from ramp $m-1$ (red) crosses a transmitter boundary eleven times out of twelve, so the two ramps differ by the target's array steering phase. Predicting from ramp $m - n_{Tx}$ (green) stays on the same physical antenna and therefore on the genuine Doppler axis.

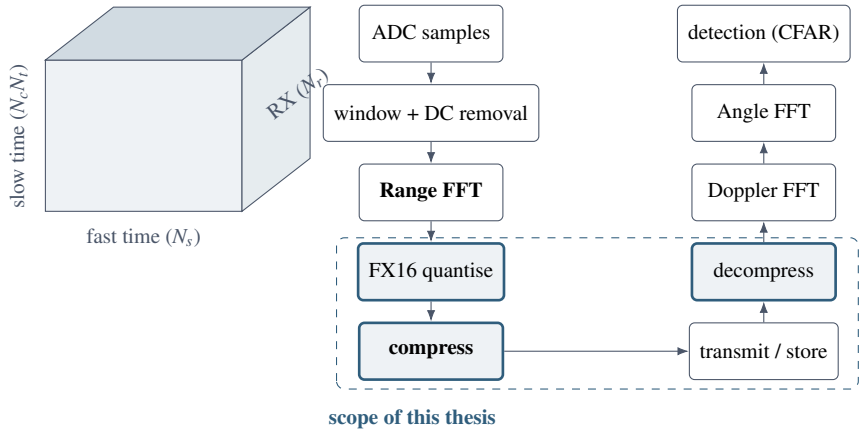


Figure 2.3: The radar data cube and the processing chain. The compressor sits immediately after range-FFT and fixed-point quantisation, before the data leaves the sensor. Everything downstream — Doppler FFT, angle FFT, detection — sees the reconstructed data, which for a lossless compressor is bit-identical to the original.

Carry forward: on a TDM-MIMO radar the ramp axis is transmitter-interleaved. A slow-time predictor that assumes consecutive ramps are consecutive *in time on the same antenna* — as DRHE does in its single-transmitter form, and as a plain lag-1/lag-2 linear predictor does — is predicting across the wrong axis. Section 4.3 sets the prediction lag to n_{Tx} for this reason, and Chapter 7 measures the difference it makes.

2.3 The radar data cube, and why it is large

Putting the three axes together gives the *radar data cube*: fast time (samples within a chirp) $\times$ slow time (ramps) $\times$ receive channel. Three FFTs are applied along the three axes in turn, yielding range, velocity and angle. Figure 2.3 shows the arrangement and where in the chain this thesis intervenes.

The volume follows directly. For the sensor used here, one frame is

$$N_s \times N_c \times N_r \times N_t = 256 \times 16 \times 16 \times 12 = 786{,}432 \tag{2.8}$$

complex samples. Each is two 16-bit integers, so a frame is 3,145,728 B — exactly 3 MiB. At 20 frames per second, a common automotive rate, that is about half a gigabit per second from one sensor, and a vehicle carries several.

2.4 Lossless compression: how it actually works

2.4.1 The two categories

Compression divides into *lossless*, where the decompressed data is bit-identical to the input, and *lossy*, where some information is permanently discarded in exchange for a smaller payload [14]. Lossy compression can reach far higher ratios; lossless compression is bounded by the information content of the data itself. Section 1.3 explained why this application needs the lossless kind.

2.4.2 The bound: entropy

There is a hard limit on lossless compression, and knowing it is what makes the results in Chapter 7 interpretable. If a source emits symbols x with probabilities $p(x)$, its *order-0 entropy* is

$$H = -\sum_{x} p(x) \log_2 p(x) \quad \text{bits per symbol,} \tag{2.9}$$

and no code that treats symbols independently can average fewer bits than H [14]. This gives a concrete yardstick: measure H for the data, compare it against what the coder actually achieves, and the gap is the coder's inefficiency. Section 7.3 uses exactly this to separate what the predictor contributes from what the Huffman dictionary contributes.

2.4.3 The standard two-stage structure

Almost every practical lossless compressor has the same shape (Figure 2.4): a *decorrelation* stage followed by an *entropy coding* stage.

The **decorrelation** stage exploits structure. If consecutive samples are similar, predict each one from its neighbours and transmit only the *residual*, the difference between prediction and

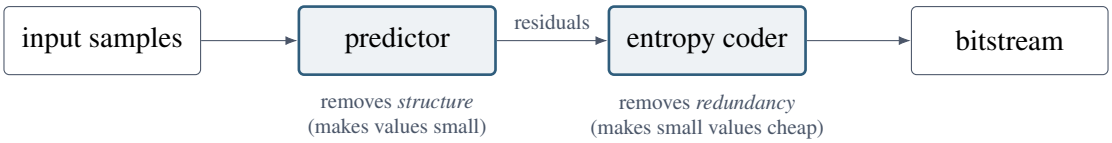


Figure 2.4: The standard structure of a lossless compressor. The two stages do different jobs, and each can succeed or fail independently of the other.

truth. When the prediction is good, residuals cluster tightly around zero even though the original samples spanned the full dynamic range. Nothing is lost: given the residual and the same prediction rule, the decoder reconstructs the original exactly.

The **entropy coding** stage exploits the resulting skew in the distribution. Values that occur often get short codewords; values that occur rarely get long ones. Huffman coding [15] is the classic construction and produces the optimal prefix-free code for a known symbol distribution.

Crucially, the two stages are independent. A predictor that fails — that produces residuals no more skewed than its input — does not break the coder; the coder simply carries on compressing whatever it is handed, and the compression ratio can still look respectable. Detecting this requires deliberately measuring what the coder achieves *without* the predictor. This thesis reports that no-prediction baseline alongside every predictive result.

2.4.4 Open-loop versus closed-loop prediction

One subtlety in predictive coding matters enough to state now, because the hardware of Chapter 6 is built around it (Section 6.4).

The encoder predicts from past samples. So does the decoder — but the decoder does not have the past *original* samples, only the past *reconstructed* ones. In *open-loop* prediction the encoder uses the originals; in *closed-loop* prediction (classically, differential pulse-code modulation, DPCM) the encoder reconstructs each sample exactly as the decoder will, and predicts from that.

When compression is perfectly lossless the two are identical, because reconstruction equals the original. The moment anything — a saturation, a rounding, a clamp — makes reconstruction differ from the original by even one bit, an open-loop encoder and its decoder begin predicting from different histories and *diverge*. With an IIR predictor, as in DRHE, that divergence is then carried forward through every subsequent ramp. Closing the loop confines the error to the affected sample.

Carry forward: a compression ratio is the product of two independent effects, and a healthy-looking ratio does not imply a working predictor. Always measure the entropy coder alone as a baseline.

2.5 The sensor and the dataset

2.5.1 AWR2243 and the MMWCAS-RF-EVM

The radar front end assumed throughout is the Texas Instruments AWR2243, a 76–81 GHz FMCW transceiver with three transmit and four receive channels [13]. Importantly for this work, the AWR2243 performs no signal processing of its own: it transmits, receives, mixes and digitises, then streams raw ADC samples out over a MIPI CSI-2 interface. It therefore exposes exactly the payload that creates the transport bottleneck.

Four AWR2243 devices are cascaded on the MMWCAS-RF-EVM evaluation module, giving 12 transmit and 16 receive channels on one synchronised board — the 192-element virtual array of Section 2.2.

2.5.2 ColoRadar

Rather than synthetic stimulus, this thesis uses real captures from the public ColoRadar dataset [12], which includes cascade recordings from exactly this module across indoor, outdoor and subterranean environments. Table 2.1 lists the relevant parameters. Fifty consecutive frames were used throughout; that is 39,321,600 complex ADC samples, or 150 MiB of raw data, which is enough to make a claim of losslessness meaningful.

Each complex sample is stored as two contiguous `int16` values. The index of the sample at (fast-time s, chirp loop q, receiver r, transmitter t) in the file is

$$i = 2\big(s + N_s(q + N_c(r + tN_r))\big), \tag{2.10}$$

so the layout from slowest to fastest varying is $[t, r, q, s]$. Reshaping this into the (fast time, ramp, receiver) cube the processing chain expects is what produces the transmitter-interleaved ramp axis of Equation (2.7).

2.6 Number representation and quantisation

Table 2.1: ColoRadar cascade capture parameters used throughout this thesis.

Parameter	Symbol	Value
ADC samples per chirp	N_s	256
Chirp loops per frame	N_c	16
Receive channels	N_r	16
Transmit channels (TDM)	N_t	12
Ramps per frame	$N_c N_t$	192
Fast-time sample rate	f_s	8 MHz
Chirp bandwidth	B	1.798 GHz
Ramp repetition interval	T_r	78.5 µs
Centre frequency	f_c	77 GHz
Range resolution	ΔR	8.34 cm
Frame size (raw)	—	3,145,728 B

2.6.1 Fixed point versus floating point

Hardware has to represent numbers somehow, and the choice is consequential.

Fixed-point representation allocates a constant number of bits to the integer and fractional parts, commonly written *Qm.n*. Because the binary point never moves, addition, subtraction and multiplication are ordinary integer operations: cheap, single-cycle, and small in silicon [16]. The price is a fixed dynamic range. Values too large wrap around; values too small round to zero.

Floating-point representation stores a sign, exponent and mantissa, so the binary point moves with the magnitude [17]. This buys enormous dynamic range but costs a great deal: every operation requires exponent alignment, normalisation and special-case handling, which needs a dedicated floating-point unit and takes multiple clock cycles. Section 6.2 shows what this costs in the DRHE design: the floating-point operators account for about two thirds of its LUTs and all of its DSP slices.

2.6.2 Quantisation error, and the FX16 step

The range-FFT output is naturally a complex floating-point quantity. Before it can be compressed by a fixed-point pipeline it has to be quantised to 16-bit integers — the **FX16** step. This is a scaling followed by rounding:

$$\sigma = \frac{32767}{A_{\max}\, 2^{\text{shift}} \sum_k w_k \,/\, (2N_s)}, \qquad x_{\text{FX16}} = \text{round}(\sigma \cdot x), \tag{2.11}$$

where $A_{\max}$ is the full-scale ADC amplitude and w is the window applied along fast time.

Two things about Equation (2.11) matter later. First, the rounding introduces an error bounded by half a least-significant bit, which acts as a wideband noise floor [18]; this is the dominant source of detection degradation in the whole chain, and Chapter 7 separates it cleanly from the cost of compression. Second, and more consequentially for this thesis, the scale factor is *fixed* — it is calibrated from a theoretical full-scale ADC sinusoid, not from the data. If the actual capture sits well below full scale, the top bits of every resulting `int16` are only copies of the sign bit. On ColoRadar the largest FX16 code is 437, so fewer than ten of the sixteen bits ever carry information; Section 8.3 shows how much of the compression ratio this accounts for.

Carry forward: the FX16 scale factor is fixed and calibrated to full scale. Whether the capture actually fills the container is a property of the sensor's gain staging, and it silently inflates any compression ratio measured downstream.

2.7 FPGAs and high-level synthesis

2.7.1 What an FPGA is

A field-programmable gate array is a chip full of small configurable logic blocks and configurable wiring between them. Rather than executing instructions, an FPGA *becomes* the circuit you describe. The practical consequences for this application are three:

- **Parallelism.** A processor handles four receive channels one after another. An FPGA can instantiate four copies of the datapath and do all four in the same clock cycle. Every design in this thesis does exactly that.
- **Deterministic latency.** There is no cache, no operating system, no interrupt jitter. The DRHE compressor of this thesis takes 61,472 or 61,473 cycles on every one of the 50 test frames, which is what a safety-critical pipeline needs.
- **Reconfigurability.** The design can be changed after fabrication, which is what makes an FPGA the right platform for evaluating designs that are still being compared.

In production, a design proven on an FPGA is normally migrated into an ASIC or into a fixed-function accelerator block inside a larger SoC, for cost and power reasons [19, 21]. The FPGA here is a proof of concept, not a proposed product.

2.7.2 The four currencies: LUT, FF, BRAM, DSP

FPGA resource usage is reported in four units, and it is worth knowing what each is because the two algorithms in this thesis trade them against each other in an interesting way.

LUT

Look-up table. A small memory that implements an arbitrary Boolean function of a few inputs. LUTs are the raw material of combinational logic. Complicated arithmetic — square roots, arctangents — costs a lot of LUTs.

FF *Flip-flop.* A one-bit register. Flip-flops hold values between clock cycles; deep pipelines need many of them.

BRAM

Block RAM. Dedicated on-chip memory, here counted in 36 kbit blocks. Arrays that are too large to live in flip-flops go here. BRAMs typically have two access ports, which becomes important below.

DSP *DSP slice.* A hardened multiply–accumulate unit. Using a DSP for a multiply is far cheaper than building one from LUTs.

The target device throughout is an AMD Kintex UltraScale+ `xcku5p-ffvb676-2-e`, which provides 216,960 LUTs, 433,920 flip-flops, 480 BRAMs of 36 kbit and 1,824 DSP slices.

2.7.3 Pipelining, initiation interval and $F_{\max}$

Three timing concepts recur throughout the results.

Pipelining splits a long computation into stages separated by registers, so several inputs can be in flight at once. The *depth* of a pipeline is how many cycles a single input takes to traverse it.

Initiation interval (II) is how many cycles must elapse between accepting one input and the next. $\mathrm{II} = 1$ means a new sample every cycle, which is the goal. $\mathrm{II} = 2$ halves throughput. II is limited by the tightest resource constraint in the loop body — very often, as Chapter 6 shows, by the number of access ports on a block RAM.

$F_{\max}$ is the highest clock frequency at which the longest combinational path still settles within one period. HLS estimates it before the circuit exists; the figure that counts is Vivado's, after routing. The two can disagree badly: both compressors in this thesis were estimated by HLS below 100 MHz, and both close above it once placed and routed (Section 6.7).

Latency, throughput and $F_{\max}$ interact: a frame of K beats at initiation interval II and clock f takes roughly $K \cdot \mathrm{II}/f$ seconds, *plus* the pipeline depth once per invocation. That last term is often small, but not always: the LPC compressor of Section 6.3 invokes two short loops 512 times per frame, and their pipeline fill costs about 65,000 cycles that a trip-count model misses entirely.

2.7.4 High-level synthesis

Traditionally FPGA designs are written in a hardware description language such as VHDL or Verilog, at the level of registers and wires. *High-level synthesis* (HLS) instead takes annotated C++ and generates the RTL automatically. Loops become state machines; arrays become memories; pragmas tell the tool to pipeline a loop, partition an array, or expose a port as an AXI4-Stream interface.

HLS is used throughout this thesis for a specific reason beyond convenience: because the design source is C++, the *same source* can be compiled and run as ordinary software against the MATLAB reference, before any hardware is generated. That makes the verification ladder of Section 5.4 possible: all 50 frames can be checked bit for bit in minutes rather than in hours of RTL simulation.

2.7.5 AXI4-Stream

The interface used by both compressors is AXI4-Stream: a simple valid/ready handshake carrying a fixed-width data word per beat. The designs here accept 128 bits per beat — four receive channels, each a 16-bit real and a 16-bit imaginary part — and emit 256 bits per beat of compressed bitstream. A useful consequence for benchmarking is that "128 bits per cycle" is the theoretical maximum input rate, and therefore what a perfectly efficient design at $\mathrm{II} = 1$ would achieve. Appendix A uses this to interpret a published throughput figure.

2.8 Related work and the baseline

2.8.1 Deterministic industrial schemes

Because automotive safety systems demand bounded latency, commercial radar processors use lightweight deterministic compressors. Texas Instruments uses Exponential-Golomb Encoding

[20]; NXP's S32R family uses block floating-point formats such as CP4D, in which several mantissas share one exponent [21]. These are described as "near lossless" — CP4D introduces under 0.25 dB of signal loss — and reach ratios of roughly 2 to 4.

2.8.2 Block Adaptive Quantization

BAQ originates in synthetic aperture radar [22]. It divides the data into blocks, estimates each block's statistics, and adapts the quantiser step size accordingly, so weak blocks are quantised finely and strong blocks coarsely. It is lossy. It appears in this thesis as a comparison point, and Chapter 7 shows what its losses cost in detection terms.

2.8.3 Range-dependent variable-length encoding

Received power falls with range, so far range bins hold smaller values than near ones. Range-dependent variable-length encoding (RDVLE) [26] exploits this by spending fewer bits on far bins; a second stage, uniform dynamic-range re-encoding (UDRE), re-quantises blocks of bins to a fixed width. Both are lossy, and both are among the eight methods compared here.

2.8.4 Predictive coding: DRHE and linear prediction

Doppler Redundancy Hybrid Encoding [26, 25] predicts each range-FFT sample along slow time from earlier ramps of the same range bin, using two infinite-impulse-response filters in polar form, and entropy-codes the prediction error. Linear predictive coding is the general form of the same idea: each sample is predicted as a weighted sum of earlier ones, with weights fitted to the data. Meucci and Mancuso [27] apply it to raw ADC samples along fast time with a Huffman code derived from the data. Chapter 4 specifies both predictors as used here.

2.8.5 Neural and spectral approaches

Autoencoder-based compressors reach high ratios but are expensive to deploy at the sensor [23]. AdaRadar [24] avoids this by keeping the edge side simple — a discrete cosine transform and spectral pruning — and placing the neural decoder at the SoC, with a feedback loop that adapts the rate to downstream perception performance. These are outside the scope of this thesis, which concerns itself with lossless schemes, but they define the upper end of the ratio spectrum.

2.9 The baseline: Kiem's thesis

The direct predecessor of this work is the master's thesis of Lukas Kiem [11], carried out at Infineon. It defined the transport problem for a single-transmitter Infineon radar (2.4 Gbit/s of raw data, so a ratio of 2.4, or 2.7 with protocol headers, to fit Gigabit Ethernet), compared deterministic industrial schemes with DRHE on private recordings, selected DRHE (3.25 in simulation), simplified it for hardware with a fixed Huffman dictionary, and implemented the compressor on a Zynq UltraScale+ device (3.17 in hardware). Because most comparisons in this thesis are made against it, Chapter 3 describes that work in full: his use case and data, his design space and results, how he adapted DRHE, his hardware, and what he left open.

2.10 Summary of what the later chapters assume

1. The data to be compressed is the *range-FFT output*, quantised to 16-bit integers, arranged as (ramp $\times$ range bin $\times$ receiver).
2. The redundancy worth exploiting lives along *slow time*: a target at constant velocity gives nearly constant magnitude and a constant phase increment.
3. On this sensor the ramp axis is *transmitter-interleaved*, so the slow-time neighbour of ramp m is ramp $m - N_t$, not ramp $m - 1$.
4. Lossless compression is required, so a method is judged first on whether it loses detections and only then on its compression ratio.
5. A compression ratio is the product of a decorrelation gain and an entropy-coding gain, and the two must be measured separately.
6. The FX16 scale factor is fixed to full scale, so container occupancy is a confounding variable in any measured ratio.
7. FPGA cost is measured in LUTs, flip-flops, BRAMs and DSPs, and timing in initiation interval, pipeline depth and $F_{\max}$.

Chapter 3

The Baseline: Kiem's Thesis

Almost every number in this thesis is eventually set beside a number from Lukas Kiem's master's thesis [11], carried out at Infineon. A comparison is only as good as one's understanding of what the other number measures, so this chapter sets out what Kiem did, on what data, with what tools, and what he concluded — in enough detail that the comparison of Appendix B can be followed without his thesis open. Section and table numbers quoted in this chapter ("his Table 3.5") refer to his thesis.

3.1 The problem he set himself

Kiem worked on the transport problem of Section 1.2: a single high-resolution radar MMIC produces more raw data than an inexpensive in-vehicle network can carry. He defined it quantitatively with a use case (his Section 3.3, Table 3.1). One Infineon CTRX8191F MMIC, four receive channels, 12-bit IF samples at an output sample rate of 50 MS/s, gives

$$50\,\mathrm{MS/s} \times 12\,\mathrm{bit} \times 4 = 2.4\,\mathrm{Gbit/s}, \tag{3.1}$$

so carrying it over one 1000BASE-T1 Gigabit Ethernet link needs a compression ratio of 2.4, or 2.7 once protocol headers are counted. The sensor cycles at 20 Hz (50 ms); a frame of 1024 ramps of 1024 samples takes about 21 ms to acquire, and he allowed compression 15 ms of the remaining 29 ms.

Two features of this setting matter for everything that follows, and both differ from the sensor used in this thesis.

Table 3.1: Kiem's use case (his Table 3.3) and the requirements he derived from it (his Section 3.3).

Parameter	Value
MMICs / transmitters used	1 / 1
Samples per ramp	1024
Ramps per frame	1024
Receive channels	4
IF data width	12 bits
Output sample rate	50 MS/s
Raw data rate	2.4 Gbit/s
Cycle time	50 ms (20 Hz)
Required compression ratio	2.4 (2.7 with headers)
Compression time budget	15 ms

One transmitter. Kiem's radar is a single MMIC operated with one transmitter (his Table 4.4 fixes "Number of TX: 1"). Consecutive ramps are therefore consecutive observations by the same antenna pair, one pulse interval apart. There is no transmitter interleaving on his ramp axis, so the question on which the prediction design of this thesis turns (Section 2.2.3 and Section 4.3) never arises for him: on his radar the previous ramp *is* the previous observation of the same channel.

Real-valued samples. The CTRX delivers real IF samples. The FFT of a real sequence is conjugate-symmetric, so Kiem keeps the first half of the spectrum because the second half is redundant. The ColoRadar cascade delivers complex I/Q samples; its spectrum is not symmetric, and the halving applied in this thesis is a deliberate choice to keep positive ranges (Section 5.2), not a consequence of symmetry.

3.2 His simulation workflow

Kiem evaluated compression schemes in a MATLAB environment (his Figure 3.1, Section 3.2.1) whose stages this thesis adopts almost unchanged:

1. **Input data**: a cube of raw ADC samples, real-world or synthetic.
2. **Pre-processing**: shift the ADC samples left by $16 - bitWidth + 16$ bits into a 32-bit full-scale word, then apply a Hanning window of the ramp length.
3. **Range FFT**, divided by the number of samples (to prevent overflow in his FFT IP core),

keeping the first half.

4. **Compress** and **decompress**, including any quantisation a scheme needs.
5. **Second-stage processing** in double precision: a Hanning-windowed Doppler FFT divided by the number of ramps, non-coherent integration (NCI) of the power over all receive channels, and peak detection combining local maxima with a noise-based global threshold.
6. **Comparator**: detections and metrics of each scheme against the reference path.

His metrics (his Section 3.4.2) were the compression ratio; false negatives and false positives against the reference; the noise floor estimate, defined as "the mean of the noisy part of the signal in dB" in dBFS; and the SNR, defined as the RMS signal of the detected peaks in dB minus the noise floor. The reference format was 16-bit floating point (FP16). His "false positive" percentage is normalised by the number of test detections, so it is a false discovery rate (Section 5.3.3). He does not state how 0 dBFS is defined, what exactly the "noisy part" of the signal is, or the detector's threshold; Section B.2.2 records how each of these gaps is filled when his chain is rebuilt.

3.3 His data

Kiem used two kinds of input (his Section 3.2.2), and it matters which produced which result.

Real-world recordings. Measurements taken with the CTRX in several scenarios, from a single stationary target to several moving ones, all with the configuration of Table 3.2: one MMIC, 512 samples, 512 ramps, four receive channels, 14-bit IF data. His design-space table (his Table 3.5), his removal of run-length coding (Table 3.6), the derivation of his fixed Huffman dictionary ("the real radar data with id = 1", Section 3.5.2) and his hardware evaluation (the SD-card input of Section 5.2.1) all use these recordings. They are not publicly available.

Synthetic scenes. Generated from a list of targets (range bin, Doppler bin, amplitude) with additive white Gaussian noise, for configurations where no recording existed. He shows one such scene (Table 3.2, his Table 3.2 and Figures 3.5–3.7) but reports no compression results on it.

Every compression result Kiem publishes, in simulation and in hardware, was therefore

Table 3.2: Kiem's two kinds of input data (his Tables 3.1 and 3.2).

	Real-world recordings (his Table 3.1)	Synthetic scene (his Table 3.2)
MMICs	1	1
Samples × ramps	512×512	1024×512
Receive channels	4	4
IF bit width	14	14
Targets	real scenes	5: range bins [100, 150, 150, 200, 400], Doppler bins [50, 50, 100, 200, 250], amplitudes [0.8, 0.3, 0.4, 0.5, 0.7]
Noise	real	AWGN, -75 dBFS after NCI
Used for	every published result	illustration only

measured on his real CTRX recordings; the synthetic scene is an illustration of his processing chain. Those recordings are private, come from a single-transmitter sensor with real-valued samples, and are much smaller than a cascade frame. That is the case for validating the same algorithms again on public MIMO data, which is the first objective of this thesis (Section 1.4). It also means that his published numbers can be reached from this thesis only through a reconstruction of his chain, which Appendix B provides.

3.4 His design space

Kiem surveyed the automotive compression schemes available to him (his Section 2.7) and took all but one into a design-space exploration (his Table 3.4).

FP16

Half-precision floating point: the reference format.

CMPRA, CMPRB

Infineon AURIX common-exponent formats that pack 16 or 32 complex bins into one 256-bit word with a shared exponent: a fixed CR of 2 or 4 against 16-bit data.

EGE10 . . . EGE4

Texas Instruments' compression engine: order-k exponential-Golomb coding of blocks of range bins, with least significant bits dropped to meet a fixed CR of $16/k$.

DRHE

Doppler Redundancy Hybrid Encoding, from Liang Li's master's thesis [26] and Zivkovic and Li [25]: prediction along the Doppler (slow-time) dimension of the

range-FFT output with two IIR filters in polar form, then run-length and Huffman coding of the differences. The original papers report "ideal" compression ratios of 9.7–9.8 (his Table 2.1).

The exception was **LPC-Huffman**, the method of his reference [25] (Meucci and Mancuso [27]). His treatment of it is quoted here in full, because Section 4.5.6 and Appendix B depend on exactly what it says.

> "In the work [25], the authors present a solution to compress FMCW radar data using a combination of linear predictive coding (LPC) and Huffman coding. The input data is said to be int16 data. Therefore, we assume that this work is performed on raw ADC data." (Section 2.7.4)
>
> "The test results in the work achieve a CR of 2.056." (Section 2.7.4)
>
> "We take all these solutions except the LPC-Huffman compression algorithm into the design space. LPC-Huffman compression is not taken because it has the same approach as DRHE, but instead of compressing range FFT data, it operates on raw ADC data; the results for DRHE are more promising." (Section 3.4.1)

Three things follow directly from these sentences. Kiem *assumed* the domain of [27] rather than knowing it. He did not implement or run LPC-Huffman. And "more promising" compares two published numbers — 2.056 on the data of [27] against 9.7–9.8 on the data of the DRHE papers — not two algorithms on the same data. His own Table 3.4 describes DRHE as "LPC on range FFT data": in his words the two methods differ in *where* they predict, not in *whether*. Section B.8 runs the method he excluded on his own scene, and Section B.17 sets it beside the slow-time LPC of this thesis.

3.5 His design-space result

Table 3.3 reproduces his Table 3.5. On his real recordings DRHE reached CR 3.25 with exactly FX16's detection metrics — it is lossless with respect to the 16-bit codes, so the only degradation against FP16 comes from quantising to 16-bit fixed point. The lossy schemes either stayed at or below the CR requirement or degraded detection markedly: CMPRB and EGE4 reach a ratio of 4 but miss 7.7 % and 40 % of the reference detections.

A false-negative rate of 0.59 % is one missed detection in about 170, which is a useful check on the scale of his scene: the recording behind this table contained on the order of 170 reference

Table 3.3: Kiem's design-space exploration on his real recordings (his Table 3.5; his Table 5.1 repeats it). FP% is normalised by test detections, so it is a false discovery rate (Section 5.3.3).

Algorithm	CR	FN%	FP%	NF (dBFS)	SNR (dB)
FP16	1	0	0	−70.714	22.407
FX16	1	0.59	1.18	−70.691	22.330
CMPRA	2	0.59	0.59	−70.687	22.418
CMPRB	4	7.69	4.73	−70.253	22.111
EGE10	1.6	1.18	1.18	−70.689	22.403
EGE8	2	2.37	2.37	−70.640	22.289
EGE6	2.67	7.69	7.10	−70.367	22.197
EGE4	4	40.24	26.63	−74.689	28.245
DRHE	**3.25**	0.59	1.18	−70.691	22.330

detections, at an SNR of about 22 dB. Both figures matter in Appendix B, because a five-target synthetic scene at 60 dB SNR behaves very differently (Section B.3).

3.6 How he adapted DRHE for hardware

Two simplifications turned the published DRHE into his final scheme.

Remove the run-length code (his Section 3.5.1). The original coder emits an 8-bit symbol RRRRSSSS per non-zero residual — a 4-bit run of preceding zeros and a 4-bit size category — Huffman-codes the symbol and appends the value. On his data the R4S4 histogram was far from the one the dictionary had been designed for, and dropping the run-length half cost almost nothing: CR 3.25 with R4S4 against 3.24 with S4 alone (Table 3.4), while shrinking the dictionary from 256 symbols to 16 and bounding the longest codeword. A coder with no run-length state codes every value independently of its neighbours, which is what lets a hardware implementation handle one value per channel per clock with no serial dependency.

Table 3.4: Kiem's Table 3.6: removing the run-length part of the coder.

Algorithm	CR	FN%	FP%	NF (dBFS)	SNR (dB)
DRHE (R4S4 + RLE)	3.25	0.59	1.18	−70.691	22.33
DRHE (S4 only)	3.24	0.59	1.18	−70.691	22.33

Fix the Huffman dictionary (his Section 3.5.2). A dictionary built per frame would need a histogram pass, a tree construction and transmission of the table, none of which suit a streaming

IP core. He therefore built one dictionary once, from "the real part of the S4 values for the real radar data with id = 1", and fixed it for simulation and hardware (his Appendix A). It is the *original dictionary* of this thesis (Section 4.6 and Appendix C). Its shortest codewords — two bits — go to categories 2, 3 and 4, i.e. residuals of magnitude 2 to 15, so the dictionary is a record of what his residuals looked like. Section B.7.1 reads that record.

3.7 His final compression scheme

His Section 3.5.3 specifies the scheme implemented in hardware, and Section B.10 checks the implementation of this thesis against it element by element. In summary, for each range bin n of ramp m and each receive channel:

$$\hat{a}[m,n] = \alpha\hat{a}[m-1,n] + (1-\alpha)\,a[m-1,n], \tag{3.2}$$

$$\hat{\theta}[m,n] = \beta\hat{\theta}[m-1,n] + (2-\beta)\,\theta[m-1,n] - \theta[m-2,n], \tag{3.3}$$

with $\alpha = 0.6$, $\beta = 0.4$, $\hat{\theta}$ wrapped once into $[-\pi,\pi]$, the prediction converted back to Cartesian with rounding, and the difference $X - \hat{X}$ taken in 16-bit two's complement. Each difference is coded as its size category S_4 (his Table 3.7, reproduced as Table 4.2), Huffman-coded with the fixed dictionary, followed by S_4 APPEND bits holding v for positive and $v + 2^{S_4} - 1$ for negative values. The model state is initialised to zero (his Section 3.5.4) and updated from the input samples.

These are Equations (4.9) and (4.10) of Chapter 4 with a lag of one ramp. On Kiem's radar, where $n_{\mathrm{Tx}} = 1$, that is the same thing as predicting from the previous observation of the same transmitter; the DRHE of this thesis predicts at lag n_{Tx}, and with $n_{\mathrm{Tx}} = 1$ its prediction algorithm reduces to his. Its implementation still differs in two respects that do not change the algorithm: it computes in floating point where his hardware uses fixed point, and it updates its state from the reconstructed sample rather than the input (a closed loop, which gives the same output whenever no residual is saturated). Section B.10 lists every difference.

3.8 His hardware

He implemented the scheme with Vitis HLS and Vivado 2022.2 on an AMD Zynq UltraScale+ `XCZU3CG-SFVC784-1-e` on a Trenz TE0820 module, inside a complete system: a bare-metal

program loads pre-windowed 16-bit ADC data from an SD card, an AXI DMA streams it through a Xilinx FFT IP core into the compression IP, and the compressed stream is written back to memory (his Figure 4.2). The compression IP (his Section 4.3.4):

- reads one 128-bit AXI4-Stream word per cycle — one complex 16-bit bin for each of four channels — and processes the channels in parallel;
- uses fixed point throughout: magnitudes `ap_ufixed<16,16>`, phases `ap_fixed<16,3>`, intermediates `ap_fixed<32,4>`, with `hls::sqrt`, `hls::atan2`, `hls::sin` and `hls::cos`;
- holds the predictor state in five block-RAM arrays of 4×1024 words (four channels by up to 1024 range bins), each partitioned by channel;
- packs each channel's two codes into a 58-bit word and the four words into a 256-bit output buffer;
- runs at $\mathrm{II} = 1$ at 100 MHz.

Table 3.5 collects the numbers he reports.

Table 3.5: Kiem's published hardware figures (Section 5.2 and Tables 5.2, 5.3 and 5.5 of [11]). BRAM in 36 Kb blocks.

Quantity	Value
Compression IP: LUT / FF / BRAM (36 Kb) / DSP	45,892 / 9,243 / 10 / 44
Whole system: LUT / FF / BRAM (36 Kb) / DSP	59,353 / 32,552 / 38 / 80
Clock	100 MHz
IP latency (HLS estimate)	23 cycles, 230 ns
IP latency (logic analyser, to first output word)	31 cycles
System latency (last DMA read to last DMA write)	79,410 ns
Input throughput	"11.92 Gbit/s"
CR, MATLAB (double-precision FFT)	3.25
CR, bit-accurate simulation	3.24
CR, hardware	3.17
FN / FP after hardware FFT	1.18 % / 2.37 %

Two of these figures need interpreting before they can be compared with anything. The throughput is a theoretical figure in binary units: $128 \times 100\,\mathrm{MHz} = 12.8\,\mathrm{Gbit/s} = 11.92\,\mathrm{Gibit/s}$, the full input width at $\mathrm{II} = 1$ (Section A.7). The block RAM is counted in 36 Kb blocks, as Section A.4 confirms by rebuilding his design. His hardware CR is lower than his simulation CR because the hardware FFT is fixed point: the same ADC data produces slightly different 16-bit codes, which he confirmed by simulating the FFT core's bit-accurate model.

3.9 What he concluded, and what he left open

He concluded that DRHE met every requirement of his use case: CR 3.17 against 2.4 (2.7 with headers), 0.08 ms against a 15 ms budget, and "99.4 % detection matches with the reference". His limitations and future work (his Chapter 6) were:

- the precision lost by 16-bit fixed point before and in the compressor, with the suggestion that, since the IP core far exceeds its real-time requirement, *floating-point* arithmetic for the FFT and the model prediction could be investigated;
- a bit-accurate decompression model;
- tests with a real radar sensor in the loop and all configuration options;
- safe error handling and a guaranteed compression ratio;
- "transpose compression" to suit the Doppler FFT's memory access.

This thesis takes up the first two directly. Its DRHE uses floating-point prediction (Section 6.2), and Appendix A (Section A.4 onwards) measures what that costs in hardware and what it buys in compression. It implements a decompressor for every design (Section 6.5). On the FPGA board the compressors' output is byte-identical to simulation, and the on-chip decompressors reconstruct every frame exactly. It takes up the third in part: both compressors run on an FPGA board on recorded radar frames, though not with a live sensor attached. Its saturation of the one value the code cannot represent (Section 6.4) is a form of the safe error handling he asked for. A guaranteed compression ratio and transpose compression are not attempted.

3.10 What the comparison takes from this chapter

1. Kiem's published compression ratios come from private recordings of a single-transmitter, real-sampled radar. No number in this thesis was measured on the same data, so a CR here and a CR there are not like-for-like until something connects them. The rebuilt simulation of Sections B.2–B.11 is that connection.
2. Kiem did not measure LPC. His statement that DRHE is "more promising" compares published figures from different data, for a method (fast-time prediction of raw samples) that is not the LPC of this thesis.
3. His final DRHE scheme is specified in enough detail to implement exactly, and Section B.10 checks that this thesis does.

4. His fixed dictionary is a fingerprint of the residual statistics of his own data, which Section B.7.1 reads.
5. His hardware description is detailed enough to rebuild, and Section A.4 uses a rebuilt copy to separate the cost of design decisions from the cost of tools and devices.

Chapter 4

Compression Methods

This chapter describes the methods compared in this thesis. It starts with the eight candidates and what each one does, then specifies in detail the two that were taken to hardware: the entropy coder they share, the axis along which they predict, the two predictors themselves, and the Huffman dictionary that turns their prediction errors into bits.

4.1 The eight candidate methods

Every method operates on the same input: the range-FFT output of each frame, quantised to 16-bit integers by the FX16 step of Section 2.6.2, except where a method replaces that step. All eight were implemented in the same MATLAB chain and scored with the same metrics (Chapter 5).

FP16, FX16

The two reference formats, as in Kiem's thesis: 16-bit floating point, and the 16-bit integer codes of the FX16 step. Neither compresses; FX16 is the input to every integer method below, so its detection losses are the floor that any of them can reach.

BAQ

Block adaptive quantisation, implemented in block-floating-point form: each block of 16 range bins shares an 8-bit exponent chosen from the block maximum, and each real and imaginary part keeps a 4-bit mantissa, so the ratio is fixed at $32/(8+8/16) = 3.76$. It stands in for the block-floating-point family (NXP's CP4/CP8 and Infineon's CMPRA/CMPRB are of the same kind) and for BAQ proper [22]. Lossy.

RDVLE

Range-dependent variable-length encoding [26]: fewer bits for far range bins, following the fall of received power with range, here capped at three bits removed. Lossy.

RDVLE+UDRE

RDVLE followed by uniform dynamic-range re-encoding of blocks of 16 bins at six bits [26]. Lossy.

DRHE

Doppler Redundancy Hybrid Encoding, as specified by Kiem (Section 4.4): slow-time prediction in polar form, followed by the entropy coder of Section 4.2. Lossless with respect to the FX16 codes.

DRHE fp16

The same predictor applied to FP16 values instead of FX16 codes, to test whether floating-point storage — the remedy Kiem suggested for quantisation error — can be compressed.

LPC

A second-order linear predictive coder designed for this thesis (Section 4.5). It predicts along the same axis as DRHE and uses the same entropy coder, so that only the predictor differs. Lossless with respect to the FX16 codes.

NXP's formats were not implemented separately, since their exact bit layouts are proprietary and the block-floating-point quantiser represents the family. Neural and spectral methods such as AdaRadar [24] were left out of scope: they are lossy, and their decoders run on the central processor.

The rest of this chapter concerns DRHE and LPC, which Section 7.1 shows are the only two methods that compress by more than three without losing detections.

4.2 The shared entropy coder: turning residuals into bits

This section is deliberately detailed. The entropy coder is where the compression ratio is physically realised — it is the only part of either design that actually emits bits — so understanding exactly what it charges for is what makes the results of Chapter 7 interpretable rather than merely quotable. Both predictive compressors in this thesis use exactly this coder.

4.2.1 What a residual is

Let $x[m]$ be the sample to be transmitted and let P be some rule — the *predictor* — that produces an estimate

$$\hat{x}[m] = P\big(x[m-L], x[m-2L], \ldots\big) \tag{4.1}$$

from samples that have *already been transmitted*. The encoder does not send $x[m]$. It sends the *residual*

$$d[m] = x[m] - \hat{x}[m]. \tag{4.2}$$

The decoder, having already reconstructed $x[m-1], x[m-2], \ldots$, runs the identical rule P on them, obtains the identical $\hat{x}[m]$, and recovers

$$x[m] = d[m] + \hat{x}[m]. \tag{4.3}$$

The lag L is the distance back along the ramp axis to the samples used for prediction; Section 4.3 explains why it is $L = n_{\mathrm{Tx}} = 12$ on this radar rather than 1.

Two things follow, and both matter.

Nothing is lost. Equation (4.2) is a *re-expression*, not an approximation. Given $\hat{x}[m]$, the map $x \mapsto d$ is a bijection. The sequence $\{d[m]\}$ carries exactly the information the sequence $\{x[m]\}$ carried — no more, no less. This is why a predictive coder can be lossless at all.

The predictor must be causal. P may only use samples the decoder already has, or side information sent ahead of them. A predictor that peeked at $x[m]$ itself would be unreproducible at the far end. It is also why Section 2.4.4 makes such a point of *which* version of the past the encoder uses — the originals or the reconstructions — since those differ the moment anything is not exactly lossless.

4.2.2 Why a re-expression can be worth anything

If no information is lost and none is gained, why is the residual sequence any cheaper to write down?

Because the *distribution* has changed, and a well-chosen code charges by distribution. Table 4.1 shows this on real data: one range bin of one receive channel, sixteen consecutive ramps, taken from frame 0 of the ColoRadar capture, predicted twelve ramps back by DRHE.

Table 4.1: The same sixteen real samples, written two ways. Range bin 75, receive channel 1, ramps 168–183. Both rows carry identical information.

m	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183
x (raw)	-69	-52	59	-6	-29	-15	59	-20	-69	-73	-45	84	-73	-50	59	-5
d (residual)	3	-3	0	-1	1	3	-4	3	-1	-2	-1	0	-2	4	-1	1

The raw values swing over $[-73, +84]$; writing any of them needs eight bits. The residuals never leave $[-4, +4]$; four bits would do. Since the two rows are informationally equivalent, that factor of two is free — provided the coder can be told about it.

And there is the catch. The bound $[-4, +4]$ holds for *these* sixteen samples. Somewhere else in the frame a target appears, or the scene changes, and a residual of 400 arrives. A fixed four-bit field cannot carry it. What is needed is a code that spends few bits on the common case, many on the rare one, and lets the decoder tell which is which.

4.2.3 The parsing problem

The naive approach fails immediately. Suppose the rule were "small values get four bits, large values get twelve". When the decoder reads four bits, it has no way to know whether it has just read a complete small value or the first third of a large one. Variable-length coding is not merely a matter of writing fewer bits; it is a matter of writing them so that the boundaries can be recovered.

There are two standard answers:

1. **A prefix-free code over every possible value.** Assign every one of the 65,536 possible 16-bit residuals its own codeword, no codeword being a prefix of another. This is optimal, and it is what a Huffman code over the raw alphabet would give — but it needs a 65,536-entry table, and the table has to be derived from the data and transmitted alongside it.
2. **Split the value into a size and a payload.** Send a small *description of how big the value is*, coded with a prefix-free code over a handful of symbols, followed by the value itself in exactly that many raw bits. The decoder reads the size first, so it always knows how many bits to read next.

Both compressors here take the second route, which is the same construction JPEG uses for DC coefficients. It is cheap in every respect that matters for hardware: the prefix-free table has sixteen entries rather than 65,536, it can be fixed at design time rather than derived per frame,

and decoding a symbol is a sixteen-way comparison rather than a tree walk.

4.2.4 The category S_4

The "size" half of the split is the category

$$S_4(v) = \begin{cases} 0 & v = 0, \\ \min\bigl(15,\ \lfloor \log_2 |v| \rfloor + 1\bigr) & v \neq 0, \end{cases} \tag{4.4}$$

which is simply the number of bits needed to write $|v|$. Its ranges are given in Table 4.2.

Table 4.2: The first few categories. Category $k \geq 1$ covers $|v| \in [2^{k-1}, 2^k - 1]$, which counting both signs is exactly 2^k distinct values — precisely what k raw bits can enumerate.

S_4	values covered	count	
0	$v = 0$	1	the only category with no payload
1	$v \in \{-1, +1\}$	2	
2	$\lvert v\rvert \in \{2, 3\}$	4	
3	$\lvert v\rvert \in [4, 7]$	8	
4	$\lvert v\rvert \in [8, 15]$	16	
⋮			
15	$\lvert v\rvert \in [16384, 32767]$	32,768	exactly fills 15 bits

The design is tight by construction: category k contains exactly 2^k values, so k payload bits identify one of them with nothing wasted and nothing left over. Section 6.4 deals with the one value for which "nothing left over" becomes a problem.

4.2.5 The APPEND bits

The payload half is

$$A(v) = \begin{cases} v & v \geq 0, \\ v + 2^{S_4(v)} - 1 & v < 0, \end{cases} \tag{4.5}$$

written in $S_4(v)$ raw bits.

The offset for negative values looks arbitrary but is doing something specific. Table 4.3 enumerates category 3 in full.

Because positive values in category k span $[2^{k-1}, 2^k - 1]$ — exactly the k-bit patterns with the top bit set — and the offset maps the negatives onto $[0, 2^{k-1} - 1]$ — exactly those with it

Table 4.3: Every codepoint of category 3. The negative values map to patterns with a clear most significant bit, the positive values to patterns with it set — so the sign is recovered from that one bit without transmitting a sign bit.

v	$A(v)$	3-bit pattern	MSB
-7	0	`000`	0
-6	1	`001`	0
-5	2	`010`	0
-4	3	`011`	0
$+4$	4	`100`	1
$+5$	5	`101`	1
$+6$	6	`110`	1
$+7$	7	`111`	1

clear — the sign falls out of the payload for free. The decoder's rule is therefore

$$v = \begin{cases} A & \text{if the MSB of the } S_4 \text{ payload bits is set,} \\ A - \left(2^{S_4} - 1\right) & \text{otherwise,} \end{cases} \tag{4.6}$$

which in hardware is one bit test and one subtract.

4.2.6 Cost, and the shape of the dictionary

The total cost of coding one residual is

$$L(v) = \underbrace{\ell\big(S_4(v)\big)}_{\text{Huffman codeword}} + \underbrace{S_4(v)}_{\text{payload}}, \tag{4.7}$$

where $\ell(\cdot)$ is the codeword length for that category. The dictionary is *fixed* — not computed per frame — which saves transmitting it and removes a serial dependency from the hardware. The *original* dictionary, taken from Kiem's thesis, has lengths

$$\ell = [4,3,2,2,2,5,6,7,8,9,10,11,14,14,13,12] \text{ bits.} \tag{4.8}$$

The full dictionary, codewords included, is in Appendix C. Every one of the sixteen entries was verified against the published table [11, Table A.1]; all sixteen agree once the published most-significant-bit-first codes are reversed for the least-significant-bit-first packer used in this implementation.

Substituting into Equation (4.7) gives the cost curve of the original dictionary:

Three features of this curve deserve attention.

S_4	$\lvert v\rvert$ range	ℓ	payload	**total**
0	0	4	0	**4**
1	1	3	1	**4**
2	2–3	2	2	**4**
3	4–7	2	3	**5**
4	8–15	2	4	**6**
5	16–31	5	5	**10**
6	32–63	6	6	**12**
7	64–127	7	7	**14**
8	128–255	8	8	**16**

Everything below 16 is nearly free. A residual anywhere in $[-15,+15]$ costs between four and six bits. Against sixteen raw bits that is already a compression ratio between 2.7 and 4.0, achieved without the predictor doing anything clever — it only has to keep residuals small.

There is a cliff at category 5. Cost jumps from six bits to ten. A residual of 15 costs six bits; a residual of 16 costs ten. The coder is, by design, forgiving of small errors and severe about larger ones.

Above category 8 the code loses. A residual of 128 costs sixteen bits — exactly what the raw value would have cost. Beyond that the coder actively expands. This is not a flaw: it is the inevitable consequence of assigning short codewords to some symbols, and it is why the `random-int16` edge case of Section 5.5 expands the data by 67 %.

The dictionary's shape encodes an assumption about the data — that categories 2, 3 and 4 dominate, since those get the two-bit codewords — and it can never code a value in fewer than four bits. Section 4.6 shows that the prediction errors of this thesis are smaller than that, and derives dictionaries that match them.

4.2.7 Worked example 1: one value, in full

Take $v = -5$.

1. $\lvert v\rvert = 5$, and $\lfloor \log_2 5 \rfloor + 1 = 3$, so $S_4 = 3$.
2. $v < 0$, so $A(v) = -5 + 2^3 - 1 = 2$, written in three bits as `010`.
3. The original dictionary gives category 3 the codeword `00`, two bits.
4. The emitted bits are `00 010` — five bits, against sixteen for the raw value.

Decoding runs the same steps backwards. The decoder matches `00` against the dictionary, recovers $S_4 = 3$, reads the next three bits `010`, sees the most significant is `0` so the value was negative, and computes $2 - (2^3 - 1) = -5$.

4.3 Predicting on a time-division MIMO radar

Both predictors work along *slow time*: they predict a range bin's value in one ramp from its values in earlier ramps. Section 2.2.3 showed that on a TDM-MIMO cascade the ramp axis interleaves the transmitters, so ramp m and ramp $m-1$ come from different antennas eleven times in twelve. The two predictors are therefore applied at a lag of $L = n_{\mathrm{Tx}} = 12$ ramps: each ramp is predicted from the previous ramp of the *same* transmitter. This section shows on real data why that choice matters.

Table 4.4 traces sixteen consecutive ramps of one range bin — the same bin as Table 4.1 — through the DRHE predictor twice. The left half predicts from ramp $m-1$, which is how slow-time prediction is described for a single-transmitter radar. The right half predicts from ramp $m - n_{\mathrm{Tx}}$. Everything else — the filters, the coefficients, the entropy coder, the dictionary — is identical between the two halves.

The left half is a predictor working across the wrong axis. Its residuals (-136, $+23$, -22, -70, …) are frequently *larger in magnitude than the samples they replace*, they land in categories 5 through 8, and they cost 11.00 bits per value — a compression ratio of 1.455, which is to say the entropy coder is salvaging a little from a predictor that has actively made things worse.

The right half is the same predictor twelve ramps back. Its residuals never exceed 4 in magnitude, they land in categories 0 through 3, and they cost 4.12 bits per value.

Why the difference is so large

Table 4.5 answers this directly, and it is worth studying because it is the mechanism behind the prediction gains reported in Chapter 7. DRHE (Section 4.4) predicts magnitude with a smoother and phase with a double integrator, both of which work when the quantity being predicted changes slowly and smoothly.

Look at the $\Delta\theta$ columns. Against ramp $m-1$ the phase increment is -2.244, then $+0.592$, then $+1.230$, then -0.678 — it swings over most of $[-\pi, \pi]$ with no pattern a two-tap filter

Table 4.4: Sixteen real samples traced through DRHE at two prediction lags. Range bin 75, receive channel 1, frame 0, ramps 168–183; real part shown, the imaginary part behaves identically. The window is late enough in the frame that both predictors' filters are fully converged. The two halves differ only in which earlier ramp the prediction comes from.

			predicted from ramp $m-1$				**predicted from ramp $m-n_{\mathrm{Tx}}$**			
m	TX	x	$\hat{x}$	d	S_4	bits	$\hat{x}$	d	S_4	bits
168	0	−69	67	−136	8	16	−72	3	2	4
169	1	−52	−75	23	5	10	−49	−3	2	4
170	2	59	81	−22	5	10	59	0	0	4
171	3	−6	64	−70	7	14	−5	−1	1	4
172	4	−29	24	−53	6	12	−30	1	1	4
173	5	−15	−29	14	4	6	−18	3	2	4
174	6	59	−6	65	7	14	63	−4	3	5
175	7	−20	−83	63	6	12	−23	3	2	4
176	8	−69	−93	24	5	10	−68	−1	1	4
177	9	−73	−65	−8	4	6	−71	−2	2	4
178	10	−45	−44	−1	1	4	−44	−1	1	4
179	11	84	28	56	6	12	84	0	0	4
180	0	−73	70	−143	8	16	−71	−2	2	4
181	1	−50	−75	25	5	10	−54	4	3	5
182	2	59	81	−22	5	10	60	−1	1	4
183	3	−5	67	−72	7	14	−6	1	1	4
				total bits		**176**				**66**
				bits per value		11.00				4.12
				compression ratio		1.455				**3.879**

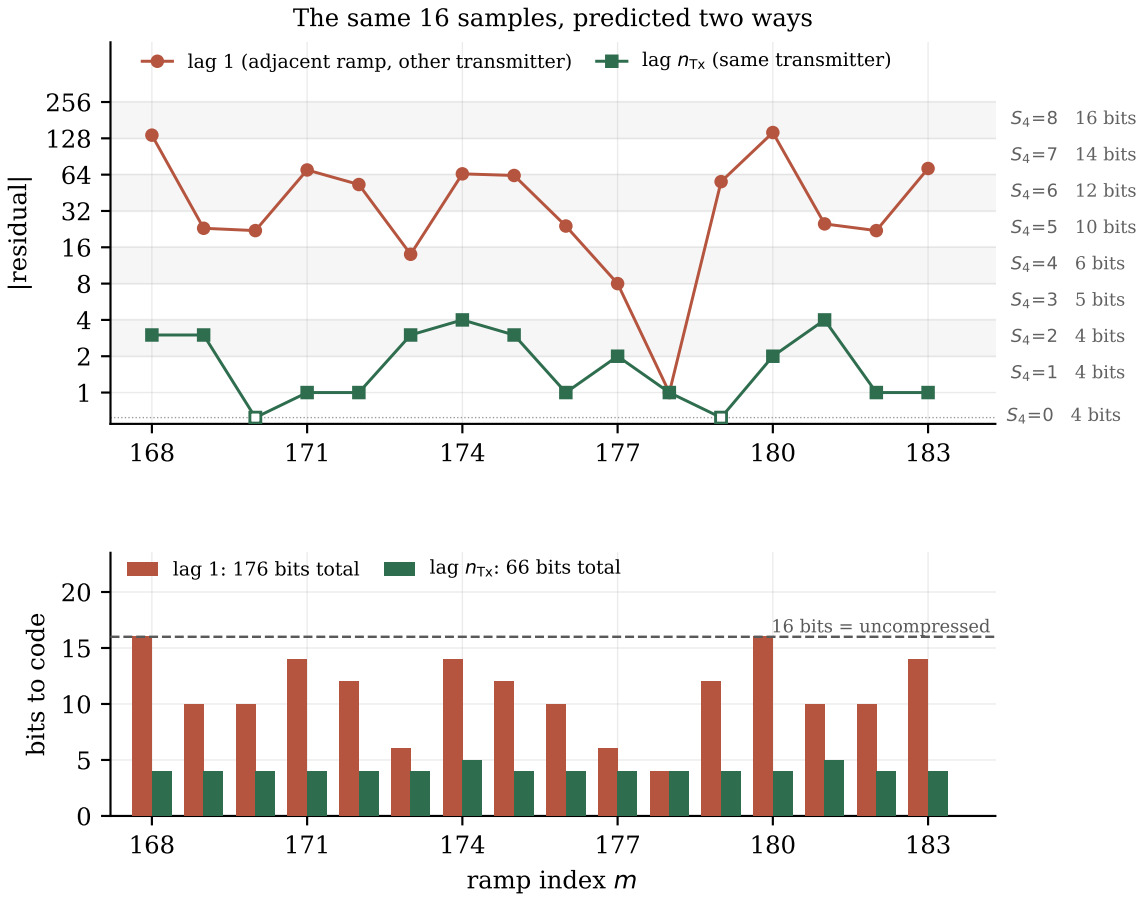


Figure 4.1: Table 4.4 drawn. Above: residual magnitude on a logarithmic scale, with the S_4 category bands and their bit costs marked on the right; hollow markers are residuals of exactly zero. Below: the bits each sample costs, against the 16 bits it would have taken uncompressed. The lag-1 predictor spends 176 bits on this window; the same predictor on the correct axis spends 66.

could follow. Against ramp $m - n_{\mathrm{Tx}}$ it is 0.002, −0.040, 0.017, −0.008: essentially zero, frame after frame, because this is a stationary target and twelve ramps later the same antenna sees the same phase.

The magnitude column tells the same story: ±22 against the immediate predecessor, ±4 against the same-transmitter predecessor.

The reason is the one given in Section 2.2.3. Ramps 168 and 167 were transmitted by *different antennas*, so the phase between them is set by where the target is in *angle*, via Equation (2.6) — and the twelve transmitters are at twelve different positions, so that phase jumps around on a period-12 pattern. Ramps 168 and 156 were transmitted by the *same* antenna, so the phase between them is set by how fast the target is *moving*, via Equation (2.5) — and it is not moving.

Table 4.5: The same window in polar coordinates. Against ramp $m-1$ the magnitude moves by up to 22 and the phase jumps unpredictably over the full $\pm\pi$ range. Against ramp $m-n_{\mathrm{Tx}}$ the magnitude moves by at most 4 and the phase increment is essentially zero — a stationary target, exactly what the double integrator is built for.

		sample		change vs. $m-1$		change vs. $m-n_{\mathrm{Tx}}$	
m	TX	$\lvert x\rvert$	$\angle x$	$\Delta\lvert x\rvert$	$\Delta\theta$	$\Delta\lvert x\rvert$	$\Delta\theta$
168	0	74.1	−2.769	−21.8	−2.244	−3.2	0.002
169	1	91.3	−2.177	17.2	0.592	−0.0	−0.040
170	2	101.0	−0.947	9.8	1.230	−2.5	0.017
171	3	110.2	−1.625	9.1	−0.678	3.0	−0.008
172	4	107.0	−1.845	−3.2	−0.220	−2.2	0.004
173	5	112.0	−1.705	5.0	0.140	−0.3	0.018
174	6	97.0	0.917	−15.0	2.622	−0.4	0.023
175	7	81.5	1.819	−15.5	0.902	0.4	−0.027
176	8	73.4	2.794	−8.1	0.975	−0.1	0.043
177	9	75.0	−2.913	1.6	0.576	2.4	0.021
178	10	95.3	−2.063	20.3	0.850	0.9	0.005
179	11	97.8	−0.537	2.5	1.526	1.9	−0.013
180	0	77.5	−2.799	−20.3	−2.263	3.4	−0.031
181	1	92.6	−2.141	15.2	0.659	1.4	0.036
182	2	102.6	−0.958	10.0	1.182	1.6	−0.011
183	3	106.1	−1.618	3.5	−0.659	−4.0	0.007

The actual bits

It is worth seeing the bitstream itself. Coding the first six residuals of the lag-n_{Tx} column, using Equations (4.4) and (4.5) and the dictionary of Appendix C:

d	S_4	$A(d)$	codeword	payload	emitted	bits
+3	2	3	`10`	`11`	`10 11`	4
−3	2	0	`10`	`00`	`10 00`	4
0	0	—	`0101`	—	`0101`	4
−1	1	0	`001`	`0`	`001 0`	4
+1	1	1	`001`	`1`	`001 1`	4
+3	2	3	`10`	`11`	`10 11`	4

which concatenates to the 24-bit stream

```
1011 1000 0101 0010 0011 1011
```

carrying six samples that would have occupied 96 bits raw. (The codewords are shown in their stored, bit-reversed form; Appendix C lists both forms.)

The same six samples at lag 1 — residuals -136, $+23$, -22, -70, -53, $+14$ — fall into categories 8, 5, 5, 7, 6 and 4 and occupy $16+10+10+14+12+6=68$ bits. Nearly three times as many, for identical information.

4.3.1 Decoding the same stream

Decoding is the mirror image, and seeing it done confirms that no side information beyond the fixed dictionary is needed. The decoder holds a bit buffer and consumes from the low end.

1. Buffer is `1011 1000`…. Match the low bits against the dictionary. The two-bit prefix `10` is category 2's codeword; no shorter or longer entry can also match, because the reversed code set is prefix-free. Consume two bits.
2. Category 2 means two payload bits. Read `11`, so $A=3$. The most significant of those two bits is set, so the value is positive: $d=+3$. Consume two bits.
3. Add the prediction: the decoder has already reconstructed ramp $168-n_{\mathrm{Tx}}=156$ for this bin, runs the identical filters on it, obtains $\hat{x}=-72$, and outputs $x=3+(-72)=-69$ — which is the value in Table 4.4.
4. Buffer is now `1000 0101`…. Match `10` again: category 2. Read payload `00`, so $A=0$, MSB clear, so $d=0-(2^2-1)=-3$. Prediction -49, output -52. Correct.
5. Buffer is `0101 0010`…. Now `10` does *not* match, because the low two bits are `01`, which is no entry's codeword. The four-bit prefix `0101` is category 0's codeword. Category 0 has no payload and means $d=0$ exactly. Prediction 59, output 59. Correct.

Step 5 is the one that shows why prefix-freeness is doing real work: the decoder never had to be told how long the next codeword would be.

4.3.2 Where the bits go over a whole frame

The sixteen-sample window is illustrative but small. Table 4.6 gives the same accounting over an entire frame — 196,608 values, all four receive channels — which is what the compression ratio actually integrates.

Several things are visible here that the headline ratios hide.

The expensive categories are rare but not negligible. Under lag-1 prediction, categories 5 through 8 account for only 5.53 % of values but 13.84 % of bits. One value in eighteen is

Table 4.6: Where the bits go, one frame, four receive channels, 196,608 values, original dictionary. "% values" is how often a category occurs; "% bits" is how much of the output it accounts for. The two columns are very different, and the difference is the whole point of entropy coding.

		no prediction		**DRHE, lag 1**		**DRHE, lag n_{Tx}**	
S_4	cost	% values	% bits	% values	% bits	% values	% bits
0	4	16.71	14.61	11.78	9.81	**18.85**	17.87
1	4	28.57	24.97	22.01	18.33	**32.94**	31.22
2	4	31.45	27.49	31.95	26.60	**34.40**	32.60
3	5	14.66	16.02	22.21	23.12	11.31	13.40
4	6	4.31	5.65	6.46	8.07	1.49	2.12
5	10	1.75	3.82	2.36	4.92	0.50	1.18
6	12	1.15	3.01	1.43	3.58	0.29	0.82
7	14	1.14	3.47	1.09	3.17	0.18	0.59
8	16	0.19	0.66	0.65	2.17	0.03	0.13
9	18	0.07	0.29	0.06	0.23	0.01	0.05
bits/value		4.576		4.803		**4.220**	
compression ratio		3.497		3.331		**3.792**	

consuming one bit in seven. That asymmetry is exactly why the tail of the residual distribution matters so much, and why Section 7.2 reports the 99th percentile of the error magnitude rather than the mean.

Same-transmitter prediction works by emptying the tail. Comparing the last two column pairs: the lag-n_{Tx} predictor moves 86.19 % of values into categories 0–2, all of which cost four bits, and cuts categories 5–8 from 5.53 % to 1.00 % of values. It is not that individual residuals got a little smaller; it is that the expensive ones almost stopped occurring.

The lag-1 predictor loses to doing nothing in a specific way. Against the no-prediction column it *gains* slightly in categories 0–2 but loses heavily in 3, 4 and 8. It is redistributing values upward, not downward. A predictor that helps should hollow out the right-hand side of this table; this one fills it in.

4.3.3 Why not one model per transmitter?

An alternative way to keep prediction on one antenna is to split each frame into twelve independent 16-ramp blocks, one per transmitter, and predict within each. For DRHE that is equivalent to the lag-n_{Tx} indexing above. For LPC it is not: fitting separate coefficients to each block

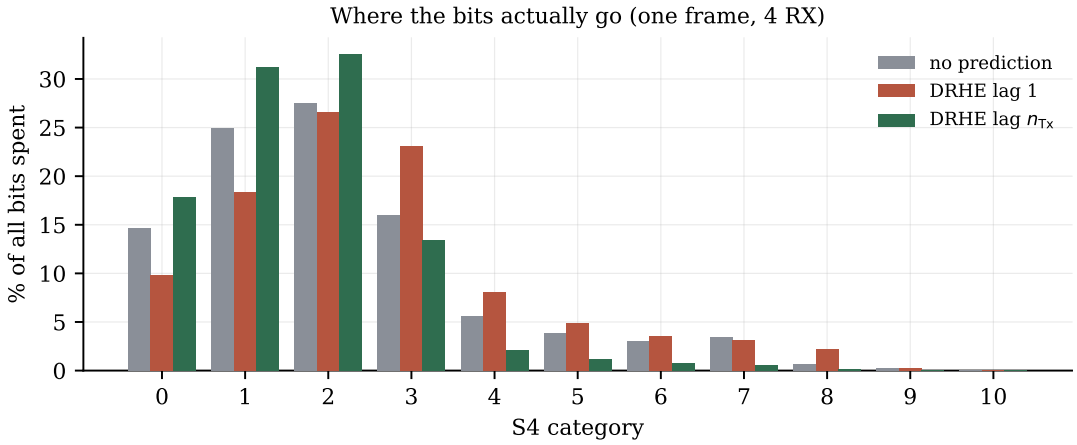


Figure 4.2: The "% bits" columns of Table 4.6 drawn. The lag-n_{Tx} predictor concentrates spending into the three four-bit categories; the lag-1 predictor pushes it out into categories 3 through 8, where each value costs two to four times as much.

multiplies the side information by twelve, from 1.04 % to 12.5 % of the uncompressed frame, and shortens each autocorrelation estimate from 192 ramps to 16, which makes the coefficients much noisier. Measured over five frames it compresses to only 2.63, worse than not predicting at all. Keeping one coefficient pair per (bin, channel, part) and moving the lags to n_{Tx} and $2n_{\mathrm{Tx}}$ costs no extra side information, and is the design used here. On a TDM-MIMO radar the right change is to the *lag*, not to the *partitioning*.

4.4 DRHE: Doppler Redundancy Hybrid Encoding

DRHE [26, 25] exploits the redundancy of radar data along the Doppler (slow-time) axis, and is a *hybrid* of prediction and entropy coding: a predictor removes the slow-time structure, and an entropy coder — originally run-length plus Huffman, here the S_4 coder of Section 4.2 — codes what is left.

4.4.1 Why predict in polar coordinates

Section 2.1 established what a range-bin sequence looks like across slow time for a target at constant velocity: the *magnitude* is nearly constant, and the *phase* advances by a constant increment. In Cartesian (I, Q) coordinates that is a point rotating on a circle — both components oscillate, and a simple difference between consecutive samples is large. In polar coordinates the same sequence is two nearly-trivial signals: a constant and a ramp.

DRHE therefore converts to polar, predicts magnitude and phase separately with filters

matched to those two shapes, converts the prediction back to Cartesian, and subtracts.

4.4.2 The two predictors

For ramp m and range bin n, with $a[m,n] = |x[m,n]|$ the magnitude and $\theta[m,n] = \angle x[m,n]$ the phase, and $L = n_{\mathrm{Tx}}$ the prediction lag of Section 4.3, the predictions are

$$\hat{a}[m,n] = \alpha\,\hat{a}[m-L,n] + (1-\alpha)\,a[m-L,n], \tag{4.9}$$

$$\hat{\theta}[m,n] = \beta\,\hat{\theta}[m-L,n] + (2-\beta)\,\theta[m-L,n] - \theta[m-2L,n], \tag{4.10}$$

with $\alpha = 0.6$ and $\beta = 0.4$ [11, Table 4.4], and $\hat{\theta}$ wrapped into $[-\pi,\pi]$.

Equation (4.9) is a first-order infinite-impulse-response smoother: it blends the previous prediction with the previous observation. For a constant magnitude it converges to that constant, and the α term gives it some immunity to noise on any single ramp.

Equation (4.10) is more interesting. Setting $\beta = 0$ reduces it to $\hat{\theta}[m] = 2\theta[m-L] - \theta[m-2L]$, which is a *double integrator*: it extrapolates the last two phases along a straight line. For a phase advancing by a constant $\Delta\theta$ per ramp — exactly Equation (2.5) — that prediction is perfect. The β term mixes in the previous prediction to damp the response to noise.

The prediction is then converted back and subtracted, with int16 wraparound $\mathcal{W}(\cdot)$:

$$d_{\Re} = \mathcal{W}\big(x_{\Re} - \mathrm{round}(\hat{a}\cos\hat{\theta})\big), \tag{4.11}$$

$$d_{\Im} = \mathcal{W}\big(x_{\Im} - \mathrm{round}(\hat{a}\sin\hat{\theta})\big), \tag{4.12}$$

and $d_{\Re}$, $d_{\Im}$ are handed to the entropy coder of Section 4.2.

4.4.3 The streaming property

DRHE has a property that matters more than its compression ratio: it is *causal and streaming*. Each output depends only on the current sample and on state carried from earlier ramps of the same transmitter. Concretely, the implementation holds five floating-point values per (transmitter, receive channel, range bin) — the magnitude prediction, the previous magnitude, the phase prediction, and the previous two phases — and nothing else. That is 120 KiB of state for this radar, against 384 KiB for the frame itself.

It therefore needs no frame buffer, and it emits its first compressed bits as soon as the first sample arrives. For a compressor sitting between an FFT and a memory bus, that is exactly the right shape. Section 8.1 returns to this when weighing DRHE against LPC.

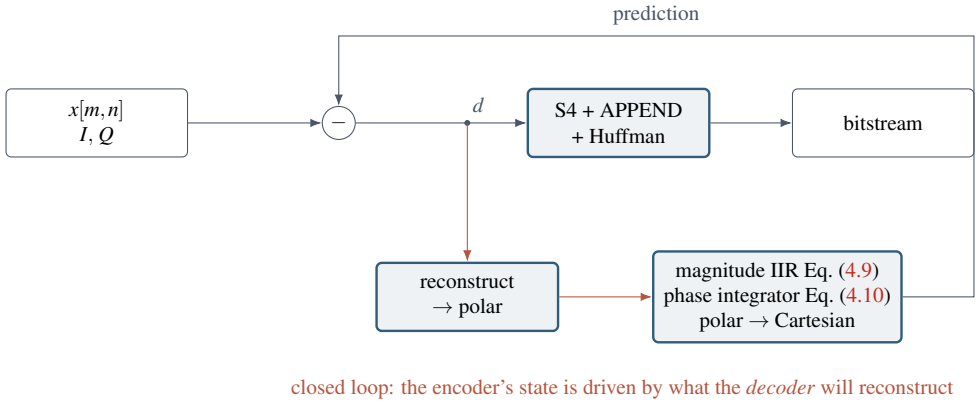


Figure 4.3: DRHE as implemented. Prediction happens in the polar domain with two filters matched to the expected behaviour of a constant-velocity target; the prediction is converted back to Cartesian before subtraction. The red path is the closed-loop correction of Section 6.4: the encoder drives its state from what the decoder will reconstruct, not from the original sample.

4.5 LPC: linear prediction with Yule–Walker coefficients

4.5.1 Autoregressive modelling

The second algorithm takes a more general approach. Rather than assuming a particular signal shape and hand-designing filters for it, it *fits* a model to the data.

A second-order autoregressive (AR) model says each sample is approximately a linear combination of the previous two:

$$\hat{x}[m] = a_1 x[m-L] + a_2 x[m-2L], \tag{4.13}$$

Given a sequence, the coefficients that minimise the mean-square prediction error satisfy the Yule–Walker equations, which for order two are

$$\begin{bmatrix} R_0 & R_1 \\ R_1 & R_0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix}, \tag{4.14}$$

where $R_k = \sum_m x[m]\,x[m-kL]$ is the autocorrelation at lag kL, summed over every ramp of the frame. Solving directly,

$$a_1 = \frac{R_1(R_0 - R_2)}{R_0^2 - R_1^2}, \qquad a_2 = \frac{R_0 R_2 - R_1^2}{R_0^2 - R_1^2}. \tag{4.15}$$

4.5.2 Stability

An AR model whose poles lie outside the unit circle is unstable: prediction errors grow rather than decay. For order two the stability triangle gives the conditions

$$|a_2| < 1 \qquad \text{and} \qquad |a_1| < 1 - a_2. \tag{4.16}$$

If either fails, or if the denominator $R_0^2 - R_1^2$ vanishes, the implementation sets $a_1 = a_2 = 0$, which degrades gracefully to "predict zero" — i.e. code the raw value. On the real data used here the test never fires: 100.00 % of coefficient sets pass.

4.5.3 Granularity and side information

The model is fitted per *(range bin, receive channel, real/imaginary part)*, across all ramps of the frame. That granularity is a deliberate compromise: fitting one model to the whole frame would ignore that different range bins contain targets at different velocities, while fitting per (bin, ramp) would have no data to fit to.

The coefficients are side information: the decoder cannot reproduce them, so they must be transmitted. Two 16-bit values per (bin, channel, part) is

$$128 \times 4 \times 2 \times 2 \times 16 = 32{,}768 \text{ bits} \tag{4.17}$$

per frame, which is 1.04 % of the uncompressed frame. That is a small but real tax, and Section 4.3 shows why fitting more coefficients — one set per transmitter, for instance — makes it the dominant cost.

4.5.4 The cost: LPC is not causal

R_0, R_1 and R_2 are sums over *all* ramps of the frame. The coefficients therefore cannot be known until the frame is complete, and the compressor cannot emit a single bit before then. Where DRHE holds five values per (channel, bin), LPC must buffer the entire frame: $4 \times 192 \times 128 \times 32 = 3{,}145{,}728$ bits, 3.0 Mibit.

This is the fundamental trade between the two algorithms, and Section 8.1 argues it matters more than the difference in compression ratio.

4.5.5 Coefficient quantisation

The Yule–Walker solution of Equation (4.15) is computed in floating point and then quantised before use, because the decoder must use exactly the coefficients the encoder used and must be able to read them from the bitstream. a_1 is stored as `ap_fixed<16,2>` (range $[-2,2)$, step 2^{-14}) and a_2 as `ap_fixed<16,1>` (range $[-1,1)$, step 2^{-15}), both with truncation towards $-\infty$, which is the HLS default. The prediction is then formed in fixed point and rounded to an integer before the residual is taken, so encoder and decoder agree to the bit. The MATLAB model reproduces this quantisation; the independent Python implementation of Appendix F does too, and matches the HLS bit counts exactly on all 50 frames.

4.5.6 What this LPC is not

Kiem's thesis mentions an "LPC-Huffman" method [27] that he judged less promising than DRHE and did not implement. That method predicts each raw ADC sample from earlier samples of the *same chirp* — along fast time, before the range FFT — and codes the residuals with a Huffman code derived from the data. The LPC here predicts each range-FFT code from the same range bin in earlier *ramps* — along slow time, after the range FFT, exactly where DRHE predicts — and uses DRHE's own entropy coder. It was designed that way on purpose: by keeping DRHE's input, axis and coder and changing only the predictor, any difference in compression ratio can be attributed to the predictor alone. Section B.17 returns to how the two methods called LPC compare.

4.6 The Huffman dictionary

The entropy coder needs one design-time constant: the sixteen codeword lengths $\ell_0,\ldots,\ell_{15}$, one per S_4 category. Two dictionaries are used in this thesis.

The original dictionary. Kiem derived his dictionary once, from the S_4 statistics of one of his own recordings, and fixed it for simulation and hardware. Its two-bit codewords go to categories 2, 3 and 4, so it is matched to residuals of magnitude 2 to 15 — the size his single-transmitter data produced. No value can be coded in fewer than four bits, which caps the compression ratio of any frame at $32/(2\times 4) = 4.0$.

The re-derived dictionaries. The prediction errors of DRHE and LPC at lag n_{Tx} are smaller than that: most fall in categories 0, 1 and 2 (Section 7.4). A new set of codeword lengths was therefore derived for each algorithm, by building a Huffman code [15] on the S_4 category counts of the algorithm's own residuals over frames 0–4 only. One is added to every count before the code is built, so that categories never seen in training still receive a codeword and the code remains lossless on any input. The lengths are assigned as canonical codes and, like the original, bit-reversed for the least-significant-bit-first packer:

$$\ell^{\mathrm{orig}} = [4,3,2,2,2,5,6,7,8,9,10,11,14,14,13,12],$$
$$\ell^{\mathrm{DRHE}} = [3,2,1,4,5,6,7,8,9,10,11,14,14,13,13,13],$$
$$\ell^{\mathrm{LPC}} = [3,1,2,4,5,6,7,8,9,10,13,13,13,13,12,12].$$

DRHE's one-bit codeword goes to category 2 and LPC's to category 1, so their commonest residuals cost three bits and two bits respectively, rather than four. Because only the contents of a sixteen-entry table change, a re-derived dictionary costs nothing in hardware; the FPGA cores of Chapter 6 use them. Both dictionaries are listed in full in Appendix C, and Section 7.4 reports the difference they make, including on the 45 frames they were not trained on.

4.7 The two algorithms side by side

Table 4.7 summarises the two designs. The trade they present is clear before anything is measured: LPC fits its model to the data and so should predict at least as well as DRHE's fixed filters, but it pays for that with side information and, more importantly, by having to see the whole frame before it can emit anything. DRHE assumes a signal shape and pays for it in floating-point arithmetic, but it streams. Chapter 7 measures how large each of these effects is.

Table 4.7: The two lossless algorithms compared on structure, before any measurement.

	DRHE	**LPC**
Prediction domain	polar (magnitude, phase)	Cartesian, I and Q independently
Model	fixed IIR filters, α, β chosen a priori	second-order AR, coefficients fitted per frame
Prediction lag	n_{Tx}	n_{Tx} and $2n_{\mathrm{Tx}}$
Causal?	yes — streaming, first output immediate	**no** — needs the whole frame before any output
Compressor state	5 values per (TX, channel, bin)	full frame, 3.0 Mibit
Side information	none	32,768 bits/frame (1.04 %)
Per-sample arithmetic	4 floating-point transcendentals (`sqrt`, `atan2`, `sin`, `cos`)	2 fixed-point multiply–accumulates
Per-frame arithmetic	—	one division per (channel, bin, part)
Entropy back end	identical: S_4 + APPEND + Huffman dictionary	

Chapter 5

Evaluation Method

This chapter sets out how the methods of Chapter 4 were evaluated: the two stages of the evaluation, the signal chain that produces their input, the metrics, and the verification ladder that carries every result from MATLAB down to the FPGA board.

5.1 Two stages

The evaluation runs in two stages, each answering one question.

Stage 1 — which methods are worth building? All eight methods of Section 4.1 are run in MATLAB over the same 50 consecutive ColoRadar frames with all 16 receive channels. Each is scored on compression ratio and on what it does to target detection. A method goes forward only if it is lossless — it loses no detections beyond those of the FX16 step every method shares — and compresses by more than three, which leaves a margin over the 2.4–2.7 that a Gigabit Ethernet link needs (Section 2.9 and Chapter 3).

Stage 2 — where does the compression come from, and what does it cost in hardware? The selected methods, DRHE and LPC, are evaluated on the four-receiver frames that the hardware processes. Their compression ratio is broken down into what the entropy coder achieves alone, what prediction adds, and what a re-derived dictionary adds. They are then implemented, with decoders, in hardware (Chapter 6), verified bit for bit at every level of the ladder of Section 5.4, and measured on the board for time per frame, latency and chip resources.

5.2 The reference signal chain

The MATLAB chain is described here in the order it executes, because several later observations depend on details of individual steps.

5.2.1 Loading

A frame is read from the ColoRadar binary and reshaped into a complex cube of (fast time $\times$ ramp $\times$ receiver), with the TDM flattening of Equation (2.7). The per-ramp mean is subtracted from each channel to remove the DC / transmitter-leakage term.

5.2.2 Windowing

A Hann window is applied along fast time to suppress spectral leakage, so that a strong nearby target does not bury a weak distant one in sidelobes.

5.2.3 Range FFT

An N_s-point FFT is taken along fast time and normalised by N_s. Only the positive half-spectrum is retained, giving $N = 128$ range bins. This halving is a deliberate choice, not a consequence of conjugate symmetry — the input here is complex I/Q, so the spectrum is *not* symmetric; the negative half corresponds to negative beat frequencies, which are not physically meaningful ranges.

5.2.4 FX16 quantisation

The complex FFT output is scaled by Equation (2.11) and rounded to `int16`. This is the last step before compression, and its output — 128 range bins $\times$ 192 ramps $\times$ 4 receive channels of complex `int16` — is exactly what both compressors consume.

Four receive channels rather than sixteen are exported to the hardware path, because four complex `int16` samples is 128 bits, the AXI4-Stream width. One such frame is $128 \times 192 \times 4$ complex samples, or 393,216 B; the 50-frame test set is 19.7 MB. Figure 5.1 shows one receiver of one such frame.

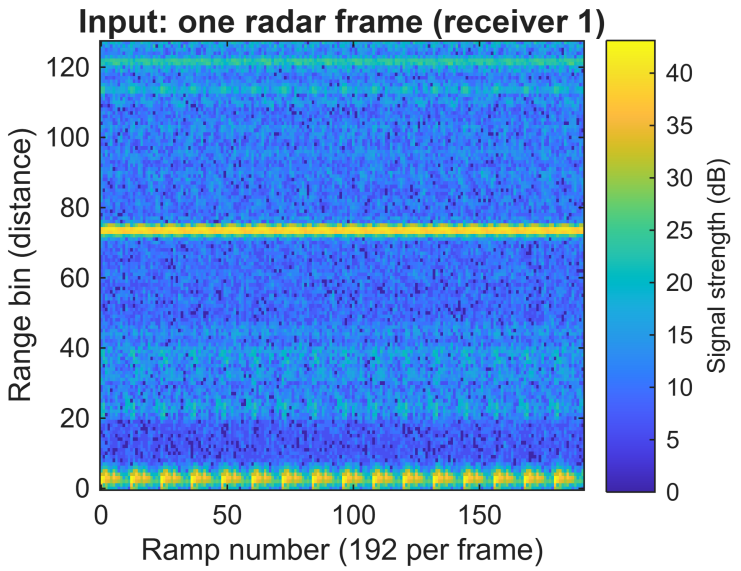


Figure 5.1: One input frame as the compressor receives it: receiver 1 of ColoRadar frame 0 after the range FFT and FX16 quantisation, 128 range bins by 192 ramps. Strong returns appear as horizontal bands at fixed range; most of the frame is low-level noise, which is why it compresses.

5.2.5 Detection and metrics

The reconstructed data is put through a Doppler FFT and a threshold detector, and the resulting detections are compared against those from the double-precision reference. Three details of this chain matter for how its results should be read:

- the Doppler FFT is a 192-point FFT across *all* ramps of the frame, in the order they were recorded — that is, across the transmitter-interleaved ramp axis of Section 2.2.3, not across the 16 ramps of any one transmitter;
- there is no angle FFT: the 16 receive channels are combined by non-coherent integration (summing magnitudes), so "detection" here means a peak in a range–Doppler map, not a localised point in space;
- the threshold is the mean of the lowest 75 % of sorted magnitudes plus a fixed 15 dB offset, with 8-connected local-maximum suppression.

Neither of the first two affects any lossless result, since a lossless compressor returns the input exactly and every downstream stage then agrees by construction. They matter only for the lossy comparison points, whose detection losses are measured by a chain that is simpler than a production one (Section 8.8).

5.3 Metrics, defined precisely

Several metric names are used loosely in this field. They are pinned down here.

5.3.1 Compression ratio

$$\mathrm{CR} = \frac{B_{\mathrm{in}}}{B_{\mathrm{out}}}, \qquad B_{\mathrm{in}} = N_{\mathrm{ramps}} \times N \times N_{\mathrm{RX}} \times 32, \tag{5.1}$$

with B_{out} the compressed length *including* AXI packet padding. Including the padding matters: a design that flushed its bit buffer per ramp rather than per frame would waste up to 255 bits per ramp, costing about 2.9 % of the ratio. Reporting the ratio without padding would hide that design choice. Ratios are computed per frame and reported as the mean over the 50 frames; the ratio of the totals differs by about 0.2 % (for example 4.208 against a mean of 4.217 for DRHE).

5.3.2 Maximum absolute difference

$$\mathrm{MaxDiff} = \max_{\text{all samples}} \left| x - \tilde{x} \right| \tag{5.2}$$

over every real and imaginary component. For a lossless coder the pass criterion is MaxDiff $=$ 0. There is no tolerance.

5.3.3 Detection metrics

Let D_{ref} be the detection set from the double-precision reference and D_{test} that from the reconstructed data. The two rates reported are

$$\mathrm{FN}\% = \frac{|D_{\mathrm{ref}} \setminus D_{\mathrm{test}}|}{|D_{\mathrm{ref}}|}, \qquad \mathrm{FP}\% = \frac{|D_{\mathrm{test}} \setminus D_{\mathrm{ref}}|}{|D_{\mathrm{test}}|}. \tag{5.3}$$

The first is a false negative rate in the ordinary sense: the share of real targets that are missed, reported as *missed targets*. The second is labelled a false positive rate in Kiem's thesis, but it normalises by the number of *test detections* rather than by the number of true negatives, which makes it a **false discovery rate** (FDR): the share of reported detections that are spurious. It is reported under that name here.

Two further properties bound how much weight the detection metrics can carry, and are stated rather than buried:

- **The comparison is cell-wise and exact.** A target detected one range–Doppler cell away from its reference position counts as both a false negative *and* a false discovery. There is no positional tolerance, which inflates both rates relative to a matched-detection criterion.
- **The** 15 dB **threshold is a free parameter.** It is not swept, and the detection rates are sensitive to it.

5.3.4 What the detection metrics actually measure here

This point is important enough to state on its own. FX16, DRHE and LPC are all exactly lossless *with respect to the FX16 codes*. Their reconstructed data is bit-identical, so their detection metrics are identical *by construction* — not by coincidence, and not as evidence that the compressors are equally good.

What the detection metrics measure for the lossless methods is therefore the cost of the **FX16 quantisation step**, which is the only lossy operation in the lossless chain. For the lossy methods they measure the cost of quantisation and compression together.

5.3.5 No-prediction baseline and order-0 entropy

To separate what the predictor contributes from what the entropy coder contributes, every predictive result is reported against a *no-prediction baseline*: the FX16 codes fed straight into the same entropy coder. A predictor that does not beat it is not helping, however good the overall ratio looks. In the same spirit, Equation (2.9) is evaluated empirically on the residual distribution: the entropy is the figure of merit for the *predictor*, and the gap between it and the achieved bits per value is the inefficiency of the *dictionary*.

5.3.6 Timing

Three timing quantities are reported for the hardware, and they answer different questions.

- **Time per frame**: clock cycles from the start of a frame's DMA transfer to the last output word being written, divided by the clock. It sets the frame rate the design can sustain.
- **Throughput**: input bits per frame divided by the time per frame.
- **End-to-end latency**: the time from the last input word being read from memory to the last output word being written back, measured by the board firmware with a hardware

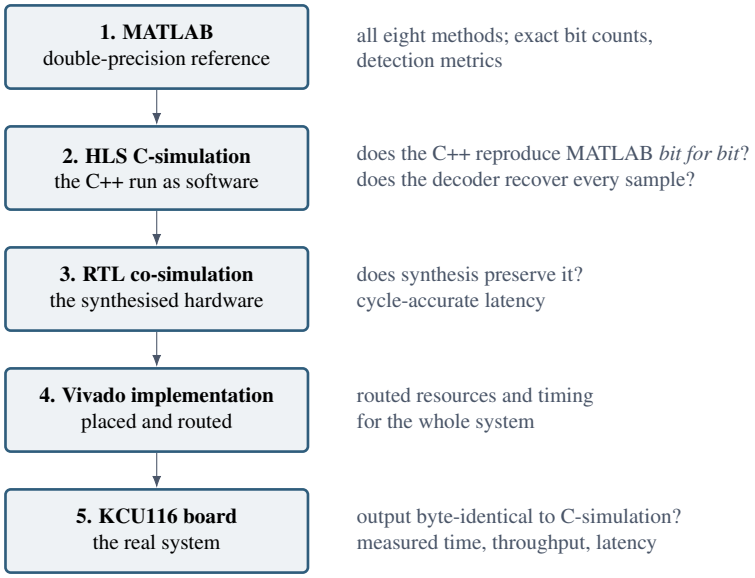


Figure 5.2: The verification ladder. Each level is checked against the one above it, so a mismatch localises to a single transition rather than to "somewhere in the design".

timer. This is Kiem's definition, and it measures how long the compressed data lags the raw data — the quantity that matters for a sensor feeding a real-time system.

For a streaming design latency is a small fraction of the time per frame; for a design that must see the whole frame first, it is most of it.

5.4 The verification ladder

A compressor that is *nearly* lossless is useless, so the method has to prove exactness rather than estimate it. Every design is therefore taken down a ladder of five levels, each checked against the one above it, so that a discrepancy always localises to a single transition (Figure 5.2).

Level 1 — MATLAB. The whole radar chain is modelled in double precision and every method is run against it. This establishes the golden reference: the exact FX16 values, the exact number of bits each algorithm should produce, and the detection metrics.

Level 2 — HLS C-simulation. Because the hardware is written in C++, it can be compiled and run as ordinary software. This level asks whether the C++ produces *exactly* the bits MATLAB says it should, and whether the decompressor recovers *exactly* the original samples, over all 50 frames. It runs in minutes. The stimulus is exported from MATLAB as `int16` rather than recomputed in C++, so both environments operate on byte-identical input and any difference in

output is unambiguously a difference in the compressor.

Level 3 — RTL co-simulation. The C++ is synthesised to Verilog and the same testbench is re-run against the generated RTL. This checks that synthesis preserved behaviour — pipelining, array partitioning and fixed-point inference can all change results if a design is subtly wrong — and gives a cycle-accurate latency figure. Co-simulation is run on one frame, since the per-frame behaviour is deterministic and 50 frames of RTL simulation with floating-point IP take many hours.

Level 4 — Vivado implementation. Each core is packaged as IP and built into the complete board system (Section 6.6), then synthesised, placed and routed by Vivado. This replaces the HLS resource and clock *estimates* with the figures of a real netlist: post-route utilisation per core and for the whole system, and the worst timing slack at 100 MHz.

Level 5 — the board. The bitstream runs on an AMD KCU116 evaluation board. All 50 frames are streamed through the compressor by DMA, the compressed output is compared byte for byte with the C-simulation output, the decompressed output is compared with the original frames, and the firmware times every frame with a hardware timer.

5.5 Edge-case testing

Real data exercises the typical case well and the extreme case not at all. The ColoRadar residuals span roughly $[-514, +416]$, which is entirely inside category 10 of a 16-category code. Everything above that is never exercised by real data.

An eight-case synthetic testbench was therefore built (Table 5.1), generating its stimulus internally so that it runs identically in C-simulation and co-simulation.

The suite exercises the parts of the design that real data never reaches, including the two situations handled by the robustness measures of Section 6.4, neither of which can arise on ColoRadar data.

Table 5.1: The synthetic edge-case suite. Each case has its own allowed reconstruction error — zero for all but the three that can produce the value −32768 and therefore exercise the saturation described in Section 6.4. Uniformly random `int16` input produces a residual of −32768 by chance, so it is one of them.

Case	What it exercises	Allowed MaxDiff
all-zeros	the $S_4 = 0$ path and the best case	0
dc-constant	a perfectly predictable sequence	0
random-int16	the worst case: incompressible input	1
tone-192ramp	a periodic sequence the AR model should fit	0
tone-1ramp	a sequence too short to fit	0
max-amplitude	full-scale values, and 128-bit correlations	0
max-alternating	±32767 alternating: the largest residuals	1
int16-min	−32768 specifically	1

5.6 An independent implementation

As a final check that the compression figures are right — not merely consistent between MATLAB and C++, which share a specification and could share a misunderstanding — the whole compression chain was written a third time, independently, in Python and NumPy, from the descriptions in Chapters 4 and 5 without reusing any code. It reads raw ColoRadar frames, applies the TDM flattening, DC removal, window, range FFT and FX16 scaling, runs DRHE and LPC with the same coefficient quantisation as the hardware, and counts the bits of the S_4, APPEND and Huffman coder with and without 256-bit padding. With the original dictionary it reproduces the HLS bit counts of both algorithms exactly on every one of the 50 frames, and with the re-derived dictionaries it reproduces the hardware's residuals exactly on the frames checked. It is also the source of the no-prediction baseline, the entropy figures and the LPC coefficient statistics reported in Chapter 7. Appendix F lists what was checked.

5.7 Reproducing the baseline

Kiem's recordings and source code are not public, so his numbers cannot be re-run directly. Two reconstructions connect them to this thesis, and Appendices A and B report what they show.

His simulation. His MATLAB chain was rebuilt from the text of his thesis — pre-processing, range FFT, FX16, DRHE with and without run-length coding, Doppler FFT and detection —

and run on the synthetic five-target scene his thesis describes. The noise level is the parameter his text leaves most open and the one that moves the ratio most, so it was swept, and the chain was also used to run the fast-time LPC-Huffman method he excluded.

His hardware. His compressor was rebuilt from its description — the same fixed-point types, functions, state arrays and packing — in the same tool flow and on the same device as this thesis's DRHE, and connected to it by a ladder of variants that each change one design decision (offset handling, packer, state reset, arithmetic, open- or closed-loop prediction). Every variant was implemented through placement and routing and checked lossless, so each step's change in resources has one identified cause.

5.8 Tools

All MATLAB work used R2026a with the Signal Processing Toolbox. All hardware work used AMD Vitis HLS 2026.1 and Vivado 2026.1, targeting the `xcku5p-ffvb676-2-e` device of the KCU116 at a 100 MHz clock, with XSIM for co-simulation and Vitis for the MicroBlaze firmware. The board is controlled from a host PC over JTAG with `xsdb` scripts, and results are checked with Python. Appendix E lists the source files and build commands.

Chapter 6

Hardware Implementation

Both algorithms were implemented as a compressor core and a matching decompressor core in C++ for Vitis HLS, and built into a test system on an AMD KCU116 evaluation board. This chapter describes the cores, the measures that keep them exactly lossless on any input, and the board system that runs and measures them.

6.1 Common interface and bit packing

All four cores share one interface. A compressor accepts AXI4-Stream input at 128 bits per beat — four receive channels, each a 16-bit real and a 16-bit imaginary part — and emits a compressed AXI4-Stream at 256 bits per beat, with TLAST marking the end of the frame. A decompressor does the reverse. Frame dimensions (range bins, ramps, receive channels and n_{Tx}) arrive on an AXI4-Lite control interface, so there is no header in the bitstream.

Both compressors share the bit packer. Codewords accumulate in a 512-bit register, least-significant-bit first, and a 256-bit word is written out whenever at least 256 bits are waiting, with one final partial word at the end of the frame. Flushing per frame rather than per ramp avoids wasting up to 255 bits per ramp, about 2.9 % of the ratio. The Huffman codewords are stored bit-reversed relative to their usual most-significant-bit-first form, because the packer is least-significant-bit first; the reversed code set is still prefix-free, so a decoder can match candidates from the low end of its buffer with no ambiguity. Compressor and decompressor read the same sixteen-entry table. Each core was built with both dictionaries of Section 4.6; the two builds differ only in the contents of this table, and have the same schedule and timing. Both were run on the board (Chapter 7); the compressors' cycle counts agree to within 13 cycles per

frame.

6.2 DRHE compressor

6.2.1 State and parallelism

DRHE holds five `float` state arrays — the magnitude prediction, the previous magnitude, the phase prediction and the previous two phases — each indexed by (transmitter, receive channel, range bin). The transmitter index is the ramp number modulo n_{Tx}, which is all it takes to put the predictor on the per-transmitter axis of Section 4.3: indexing by transmitter changes *which* address is read, not *when*, so the schedule is the same as for a lag-1 design. Each transmitter's history starts on its own first ramp, so the state is reset while $m < n_{\mathrm{Tx}}$. Frames are independent: nothing carries over from one frame to the next, so a frame lost in transit cannot corrupt its successors.

The channel dimension is completely partitioned, so the four receive channels are four parallel copies of the datapath inside one pipelined iteration of the sample loop. The state occupies 40 block RAMs (36 Kb).

6.2.2 Where the logic goes

Per sample and per channel, DRHE evaluates `sqrt` and `atan2` to convert the reconstructed sample to polar form, and `sin` and `cos` to convert the prediction back — four floating-point transcendental functions, for each of four channels, on every one of the 24,576 beats of a frame. These operators dominate the design: they account for about two thirds of its LUTs and all 132 of its DSP slices, and the two arctangent units alone are about a third of the logic. Kiem's design uses fixed point for the same functions; Appendix A measures what that choice costs.

6.2.3 Initiation interval

DRHE runs at $\mathrm{II} = 2$, and the reason has nothing to do with arithmetic. Each state array is partitioned by channel, so each channel's history is one dual-port block RAM. Every iteration reads a value and writes the updated one — two accesses, which two ports can serve. On the first ramps of a frame the iteration also writes a reset value: three accesses against two ports,

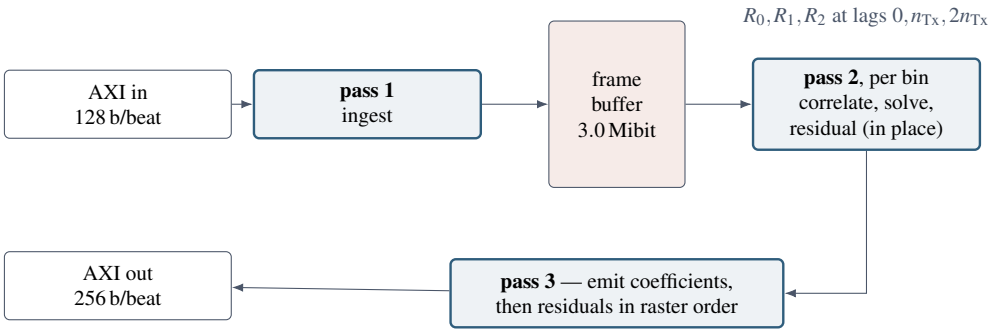


Figure 6.1: The LPC compressor. Because the Yule–Walker coefficients depend on the whole frame, the compressor must buffer it; each (channel, bin) is then correlated, solved and turned into residuals, which overwrite the buffer in place so that only one memory is needed. Emitting residuals in the original raster order is what lets the *decompressor* avoid a frame buffer entirely.

which forces the tool to serialise, and the loop settles at $\mathrm{II} = 2$ for the whole frame. Reading the state as zero on the first ramps instead of writing zeros to it reaches $\mathrm{II} = 1$. It was not adopted, because it doubles the floating-point arithmetic (units can no longer be shared between two channels) and the sensor does not need the throughput: the $\mathrm{II} = 2$ design already runs many times faster than the radar produces frames (Section 7.7).

6.3 LPC compressor

6.3.1 Three passes

LPC cannot stream, so its structure is explicitly multi-pass (Figure 6.1):

1. **Ingest.** Read the whole frame into a buffer.
2. **Correlate, solve and residual**, once per (channel, bin): compute R_0, R_1 and R_2 at lags 0, n_{Tx} and $2n_{\mathrm{Tx}}$ from the buffer, solve Equation (4.15), then walk the ramps again computing residuals and writing them back into the buffer *in place*.
3. **Emit.** Write the coefficient table, then the coded residuals.

Rewriting the buffer in place means one 3.0 Mibit memory serves both purposes rather than two.

6.3.2 Reaching back n_{Tx} and $2n_{\mathrm{Tx}}$ ramps

Both the correlation and the residual loops need $x[m - n_{\mathrm{Tx}}]$ and $x[m - 2n_{\mathrm{Tx}}]$. A shift register of depth $2n_{\mathrm{Tx}} = 24$ would move 24 entries every iteration. Instead a *ring* of exactly $2n_{\mathrm{Tx}}$ entries

gives both taps for free. At ramp m, let $k = m \bmod 2n_{\mathrm{Tx}}$. Slot k still holds the value written at ramp $m - 2n_{\mathrm{Tx}}$, because it is about to be overwritten and has not been touched since; and slot $(k + n_{\mathrm{Tx}}) \bmod 2n_{\mathrm{Tx}}$ holds the value from ramp $m - n_{\mathrm{Tx}}$. So:

read $x[m - 2n_{\mathrm{Tx}}]$	from slot k
read $x[m - n_{\mathrm{Tx}}]$	from slot $(k + n_{\mathrm{Tx}}) \bmod 2n_{\mathrm{Tx}}$
write $x[m]$	to slot k

Nothing is shifted; the cost is $2n_{\mathrm{Tx}}$ registers and two multiplexers. Every loop in the design reaches $\mathrm{II} = 1$.

6.3.3 Where the cycles go

A frame takes 325,366 cycles on the board. Ingest and emit are 24,576 cycles each, one per beat. The remaining 276,000 or so are the per-bin pass, which runs two short pipelined loops — correlation and residual — for each of the 512 (channel, bin) pairs. A pipelined loop pays its pipeline depth once on entry before reaching steady state, about 71 cycles for the correlation loop and 56 for the residual loop, so 512 invocations add roughly $512 \times (71 + 56) \approx 65{,}000$ cycles of fill that a trip-count model does not show. Merging the two loops, or processing several bins per invocation, would recover most of it; overlapping the three passes across frames would bring the time per frame down further. Neither was necessary for this thesis, and both are listed as future work.

6.3.4 Fixed-point choices

Coefficients are quantised to `ap_fixed<16,2>` for a_1 (range $[-2, 2)$) and `ap_fixed<16,1>` for a_2 (range $[-1, 1)$), which follows from the stability conditions of Equation (4.16). Over 50 frames this quantisation costs nothing measurable: the residuals are identical to the bit to those of double-precision coefficients.

The autocorrelations are accumulated as exact 64-bit integers rather than in floating point. $R_0^2 - R_1^2$ is a difference of two nearly equal quantities whenever the data is strongly correlated — precisely the regime the predictor exists for — so computing it in floating point would lose most of its significant digits exactly when accuracy matters most. The numerators and the determinant are formed exactly as 128-bit integers before the single division.

6.3.5 An asymmetry that favours the decoder

Pass 3 emits the coefficient table first — the decoder needs all of it before it can reconstruct anything — and then the residuals *in the raster order the samples arrived in*, not in the (channel, bin, ramp) order pass 2 produced them. The decompressor therefore needs no frame buffer at all: it holds the coefficient table and the last $2n_{\mathrm{Tx}}$ reconstructed samples of each (channel, bin), and writes each output sample as its residual arrives. The reordering is paid for once, by the side that already had to buffer. LPC's non-causality is a compressor-side cost only.

6.4 Exactly lossless on any input

Real radar data exercises only the lower part of the entropy coder's range (Section 5.5). Two situations that real data never produces, but synthetic input does (Section 5.5), are handled explicitly so that both designs are lossless on *any* input.

6.4.1 The one value the code cannot represent

Category 15 of the S_4 code covers $|v| \in [16384, 32767]$: counting both signs, exactly the 2^{15} patterns its fifteen payload bits can hold. But 16-bit arithmetic can produce $v = -32768$, for which Equation (4.5) gives $A(-32768) = -1$, which truncated to fifteen bits is `0x7FFF` — the same payload as $v = +32767$. The two would be indistinguishable on the wire.

Three remedies were considered: extend the code with a category 16, which changes the bitstream format; reject the input; or saturate -32768 to -32767. Saturation was chosen. It costs at most one least-significant bit on the affected sample, leaves the format untouched, and never fires on real data.

Saturation alone is not enough, though, and this is where the closed-loop prediction of Section 2.4.4 becomes concrete. If the encoder drove its predictor from the *original* sample while the decoder drives its own from the *reconstructed* one, a single saturated sample would make them diverge, and because DRHE's filters are recursive that divergence would carry through every remaining ramp. The encoder therefore reconstructs exactly what the decoder will see and updates its state from that:

```
int diff_re = wrap_int16(re - pred_re);
if (diff_re == -32768) diff_re = -32767;      // saturate
```

```
int recon_re = wrap_int16(diff_re + pred_re); // what the decoder will see
float curr_mag   = hls::sqrt(recon_re*recon_re + recon_im*recon_im);
float curr_phase = hls::atan2(recon_im, recon_re);   // state from recon
```

On real data the reconstruction always equals the original, so the closed loop changes no output bit. Its cost is in the schedule: the reconstruction sits ahead of the polar conversion on the dependency chain, and accounts for 26 of the pipeline's 58 stages.

6.4.2 Wide-integer conversion in the coefficient solver

The LPC solver forms its numerators and determinant as 128-bit integers. The conversion of such a value directly to `float` is not correct above 2^{64} in this toolchain. All three operands are therefore normalised by one common arithmetic right shift into 62 bits before the division, with the shift chosen from the largest of them:

$$k = \max\big(\mathrm{msb}|D|,\ \mathrm{msb}|N_1|,\ \mathrm{msb}|N_2|\big) - 61, \tag{6.1}$$

clamped at zero. Because numerator and denominator are shifted alike, the quotient is unchanged. On ColoRadar the correlations reach only 44 bits, so the shift is zero; on full-scale synthetic input it is what keeps the coefficients correct.

6.5 Decompressors

A decompressor exists for each algorithm, for two reasons. It is what makes bit-exact verification possible — every frame is compressed, decompressed and compared — and it completes the pipeline a vehicle would need, with the encoder at the sensor and the decoder at the central processor. Each decompressor is built twice from the same C++ source: as an FPGA core, and as a PC program compiled through Vitis C-simulation, which uses the same bit-accurate models of the floating-point functions.

6.5.1 Why decoding is the harder half

Entropy decoding is inherently serial: the length of a codeword is known only once it has been matched, and the next codeword starts where it ends. A DRHE decoder that keeps pace with the compressor — eight values (four channels, real and imaginary) per two clocks — needs

eight chained codeword matches and very wide variable shifters per cycle, plus four copies of the floating-point predictor. HLS estimates such a design at about 182,000 LUTs. HLS overestimates (by about three times for the serial decoder below), but even a third of that would be four times the serial decoder's logic. Keeping pace is also unnecessary: a decoder that takes 2 ms per frame is a hundred times faster than the radar produces frames. The DRHE decoder was therefore designed to be small rather than fast.

6.5.2 DRHE: a two-stage serial decoder

The DRHE decompressor is therefore a dataflow pair of stages connected by a stream:

- **Unpack** decodes one value per clock. It cuts a 29-bit window from two 256-bit input words, matches all sixteen codewords against it in parallel, extracts the payload, and reads the next input word just in time, stopping at TLAST.
- **Rebuild** runs one floating-point predictor that handles one receive channel per step at $\mathrm{II} = 2$ (limited by the ports of the sine/cosine tables), keeps its state in block RAM, and uses exactly the same expressions as the compressor so that its arithmetic matches bit for bit.

The two stages are balanced at about 196,608 clocks per frame, roughly 2 ms at 100 MHz; on the board it takes 196,881 (Section 7.7). HLS estimated 42,846 LUTs; after placement and routing the core uses 14,155.

Bit-for-bit agreement of the floating-point arithmetic is not optional for DRHE. The predictor feeds back on itself, so a decoder whose `atan2` or `sin` differs from the encoder's in the last bit will drift. A double-precision Python decoder written as a cross-check is exact for LPC but drifts by 1 to 6 least-significant bits on 8 of the 50 DRHE frames; the decoder built from the HLS source, which the FPGA core shares, is exact on all of them.

6.5.3 LPC

The LPC decompressor reads the coefficient table into registers, then reconstructs each sample as its residual arrives, holding the last $2n_{\mathrm{Tx}}$ reconstructed values of each (channel, bin) in a ring as the compressor does. It needs no frame buffer and no block RAM for data, and its only arithmetic is two fixed-point multiply–accumulates per value. Its codeword matcher is the simple form: it tries the sixteen codewords in category order and stops at the first match, so its

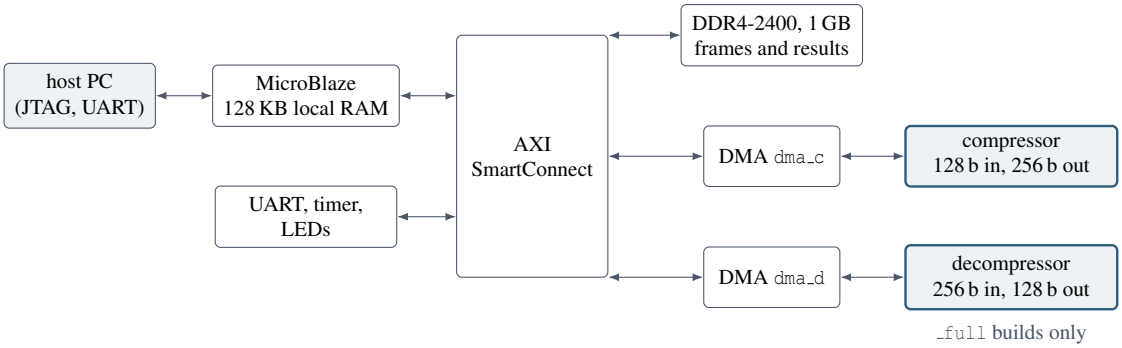


Figure 6.2: The board test system. The host places the frames in DDR4 over JTAG; the MicroBlaze starts the DMAs, times each frame and writes a result record; the host then reads the results and compressed frames back. Everything runs at 100 MHz.

speed depends on the data. Section 7.7 shows what that costs, and how the DRHE decoder's parallel matcher would remove it.

6.6 The board test system

The cores run in a complete system on an AMD KCU116 evaluation board, whose Kintex UltraScale+ `XCKU5P` provides 216,960 LUTs, 433,920 flip-flops, 480 block RAMs (36 Kb) and 1,824 DSP slices (Figure 6.2).

Hardware. A MicroBlaze processor with 128 kB of local memory controls the system. A DDR4-2400 controller gives access to 1 GB of board memory, which holds the input frames, the compressed frames, the reconstructed frames and the results. An AXI DMA (`dma_c`) streams each frame from DDR4 through the compressor and writes the compressed stream back. In the `_full` build a second DMA (`dma_d`) streams the compressed frame through the decompressor and writes the reconstruction back, so the board compresses, decompresses and checks every frame by itself. A UART, a hardware timer and LEDs complete the system, all on one AXI interconnect at 100 MHz. Each algorithm therefore has two bitstreams: a compress-only build and a `_full` build with the decompressor.

Firmware. For each frame the MicroBlaze configures the core's frame dimensions, starts the DMA transfers, and reads a hardware timer at the start, when the DMA's read channel reports the last input word read, and when its write channel reports the last output word written. From these it records the cycles per frame and the end-to-end latency of Section 5.3, together with

Table 6.1: The four board builds, whole system (processor, DDR4 controller, DMAs and cores), post-route on the `XCKU5P` at 100 MHz, with the re-derived dictionaries; the original-dictionary builds also meet timing. BRAM is in 36 Kb blocks.

Build	LUT	BRAM	DSP	WNS (ns)
DRHE, compress only	61,011 (28 %)	104.5	138	+0.243
LPC, compress only	59,203 (27 %)	162.5	164	+0.174
DRHE, `_full`	81,573 (38 %)	144	184	+0.341
LPC, `_full`	85,707 (40 %)	169.5	172	+0.331

the compressed length; the `_full` firmware also records the decompressor's cycles and latency and compares the reconstruction with the original word by word. Each frame's results are a 16-word record in DDR4.

Host. A script on the PC (`run_board.bat`, driving `xsdb`) programs the FPGA, loads the frames into DDR4 over JTAG, runs the firmware, and reads back the result records and compressed frames. For the compress-only build the PC then decompresses every frame with the decompressor built from the HLS source. Finally `compare_results.py` checks every compressed frame byte for byte against the C-simulation output, checks every reconstruction against the original frame, and tabulates compression ratio, cycles and latency.

6.7 Building and timing closure

Each core is exported from Vitis HLS as an IP block and placed in a Vivado block design with the system above; a script builds each of the four bitstreams. HLS estimated both compressors below the 100 MHz target. Its estimate is the tool's own model of operator delays against a budget that reserves about a quarter of the period for clock uncertainty, not a timing analysis of a real circuit, and once placed and routed all four systems meet 100 MHz with positive slack (Table 6.1). Two system details were needed for that: a false path from the DDR4 calibration-complete signal to the processor reset, and the DDR4 interface at 2,400 MT/s.

Chapter 7

Results

This chapter follows the two stages of Section 5.1. It first reports the screening of all eight methods, then breaks down the compression of the two selected algorithms, and then reports their verification and their measured performance on the FPGA board. It closes with the two algorithms side by side.

7.1 Stage 1: screening eight methods

All eight methods were run over the same 50 consecutive ColoRadar frames, with all 16 receive channels and 9,934 reference detections. Table 7.1 and Figure 7.1 give the result.

Table 7.1: Stage 1: all eight methods, 50-frame average, 16 receive channels. "Missed" is the false negative rate and FDR the false discovery rate (Section 5.3); NF is the noise floor. DRHE and LPC predict at lag n_{Tx} and use the original dictionary.

Method	Type	CR	Missed (%)	FDR (%)	NF (dBFS)	SNR (dB)
FP16	reference	1.00	0.00	0.00	−105.64	35.50
FX16	reference	1.00	3.77	1.11	−105.37	35.36
BAQ	lossy	3.76	10.44	9.69	−105.08	35.08
RDVLE	lossy	1.12	24.01	4.62	−102.27	33.17
RDVLE+UDRE	lossy	2.56	28.73	4.44	−101.78	33.00
DRHE fp16	lossless*	1.04	0.05	0.01	−105.64	35.51
DRHE	lossless	**3.75**	3.77	1.11	−105.37	35.36
LPC	lossless	**3.70**	3.77	1.11	−105.37	35.36

*lossless with respect to its FP16 input, which is itself rounded from the reference.

Four observations decide what goes forward.

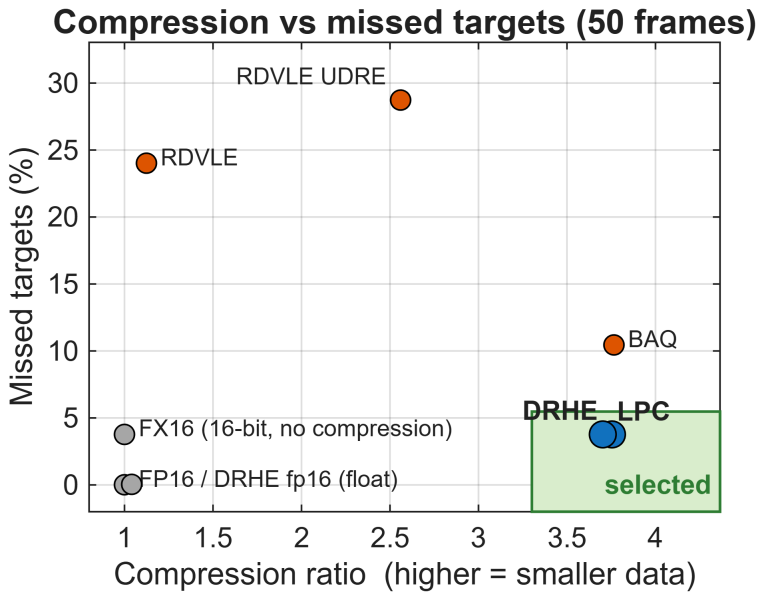


Figure 7.1: Compression ratio against missed targets for the eight methods. A good method sits to the right (small data) and at the bottom (no extra missed targets). Only DRHE and LPC reach the shaded region; the lossy methods pay for their ratio in missed targets, and storing the data as floating point barely compresses.

The lossless coders share FX16's detection metrics exactly. FX16, DRHE and LPC report identical missed-target rates, false discovery rates, noise floors and SNR. This is a consequence of losslessness rather than a coincidence: both coders reconstruct the FX16 codes bit for bit, so the detector sees identical data. What those rows measure is the cost of the *quantisation* step, 3.77 % of targets missed and 1.11 % of detections spurious, none of it the cost of compression.

The lossy methods pay for their ratio in detections. BAQ reaches 3.76, the same ratio as DRHE, but misses 10.44 % of targets instead of 3.77 %, and 9.69 % of what it reports is spurious against 1.11 %: nearly three times the miss rate and nearly nine times the false discovery rate, for no gain in ratio. RDVLE and RDVLE+UDRE are worse on both counts. In a system that brakes a car none of these trades is acceptable, which is the empirical form of the argument of Section 1.3.

Floating-point storage does not compress. DRHE applied to FP16 values almost removes the quantisation loss (0.05 % missed) but compresses by only 1.04. A half-precision mantissa keeps fine noise detail at every magnitude, so the prediction errors never settle near zero. The compressor's *input* must be integer; whether the arithmetic *inside* the predictor is floating point is a separate question, taken up in Section 8.5.

Two methods qualify. DRHE (3.75) and LPC (3.70) are the only methods that compress by more than three without adding a single missed target, and both clear the 2.4–2.7 a Gigabit Ethernet link needs with margin. Both were taken forward to Stage 2 and to hardware.

7.1.1 The cost of the quantisation step

Since FX16 accounts for all the detection loss of the lossless methods, it is natural to ask whether the 16-bit codes could be made more accurate. Table 7.2 compares four ways of producing them on one frame. The standard FX16 scaling, calibrated to a full-scale ADC signal, uses only 1.33 % of the `int16` range: the largest code in the frame is 436. Scaling each frame so that its own peak fills the container cuts the quantisation error by a factor of 67, and the companding curve of Deng and Huang [28] reduces it further.

Table 7.2: Four ways of producing the 16-bit codes, ColoRadar frame 0, all 16 receive channels. "relRMS" is the RMS error of the range-FFT values relative to their RMS; "RD error" is the RMS error of the range–Doppler map in dB; "int16 use" is the largest code as a fraction of 32,767; the last column is the lossless DRHE ratio on the resulting codes.

Quantiser	relRMS (%)	PSNR (dB)	RD error (dB)	int16 use (%)	DRHE CR
FP16 (reference)	0.0205	101.97	0.0049	—	—
FX16, fixed scaling	1.8636	64.44	0.4336	1.33	**3.80**
FX16, scaled to frame peak	0.0278	101.09	0.0054	89.20	1.18
FX16, companded [28]	0.0048	114.38	0.0011	94.28	0.78

The compression ratio moves the other way. With the container filled, lossless DRHE compresses by only 1.18; with companding, which spreads the values over the whole code range, it expands the data. Quantisation error and compression ratio are two sides of the same unused bits: the fixed scaling leaves about six of the sixteen bits as copies of the sign bit and rounds away everything below its coarse step, and stretching the data to fill the container turns those six bits into fine detail at the bottom — detail that is mostly receiver noise, which a lossless coder must keep. The quantisation error can be removed, but only by giving up almost all of the compression. The fixed FX16 scaling was therefore kept, as in Kiem's design, and Section 8.3 returns to what this means for the ratios.

7.2 Stage 2: the prediction axis

Stage 2 uses the four-receiver frames that the hardware processes. The first question is whether the predictors help at all, and the answer depends on the prediction lag (Section 4.3). Table 7.3 compares each predictor against the no-prediction baseline, at lag 1 and at lag n_{Tx}, all with the original dictionary.

Table 7.3: Compression ratio against the no-prediction baseline, 50 frames, four receive channels, original dictionary, 256-bit padding. The predictor rows are HLS C-simulation; all are lossless.

Configuration	CR	vs. no prediction
No prediction (Huffman coding only)	3.479	—
DRHE, lag 1 (adjacent ramp)	3.310	−4.9 %
LPC, lags 1, 2	3.385	−2.7 %
DRHE, lag n_{Tx}	3.751	**+7.8 %**
LPC, lags n_{Tx}, $2n_{\mathrm{Tx}}$	3.697	**+6.3 %**

Predicting from the adjacent ramp is worse than not predicting at all, for both algorithms: on this radar the adjacent ramp is another transmitter, and the difference between the two is set by the target's bearing, not its velocity. Predicting from the same transmitter turns a 4.9 % loss into a 7.8 % gain for DRHE, and a 2.7 % loss into a 6.3 % gain for LPC. The tail of the error distribution shows the same thing more starkly: the 99th percentile of |residual| is 80.6 with no prediction, 102.4 for DRHE at lag 1, and 15.8 for DRHE and 8.2 for LPC at lag n_{Tx}.

LPC loses less than DRHE on the wrong axis, and the reason is instructive. Yule–Walker chooses the coefficients that minimise the residual energy, and when the previous ramp is another transmitter there is little correlation to exploit, so the fitted coefficients shrink towards zero — and $a_1 = a_2 = 0$ *is* no prediction. Over the 51,200 coefficient sets of the 50 frames, the median fitted a_1 at lags $1, 2$ is -0.02 and half of all values lie between -0.13 and 0.09; at lags $n_{\mathrm{Tx}}, 2n_{\mathrm{Tx}}$ the median is 0.40, with a cluster near 1 from bins holding strong, slowly changing returns. DRHE's coefficients are fixed, so on the wrong axis it goes on extrapolating a phase trend that is not there. A fitted model's worst case is "predict nothing"; a fixed model's worst case is "predict wrongly".

Everything that follows uses lag n_{Tx}.

7.3 Where the compression comes from

Both algorithms share one entropy coder, so their compression ratio can be built up in three steps: what the coder achieves on the FX16 codes alone, what prediction adds, and what a dictionary re-derived for the prediction errors adds. Table 7.4 and Figure 7.2 show the result.

Table 7.4: The compression ratio built up step by step, 50 frames, four receive channels (MATLAB). The "held out" column is the total on frames 5–49 only, which the re-derived dictionary was not trained on.

	Huffman coding only	+ prediction	+ re-derived dictionary	**Total**	Total, held out
DRHE	3.479	+0.272	+0.466	4.218	4.207
LPC	3.479	+0.218	+0.707	4.404	4.372

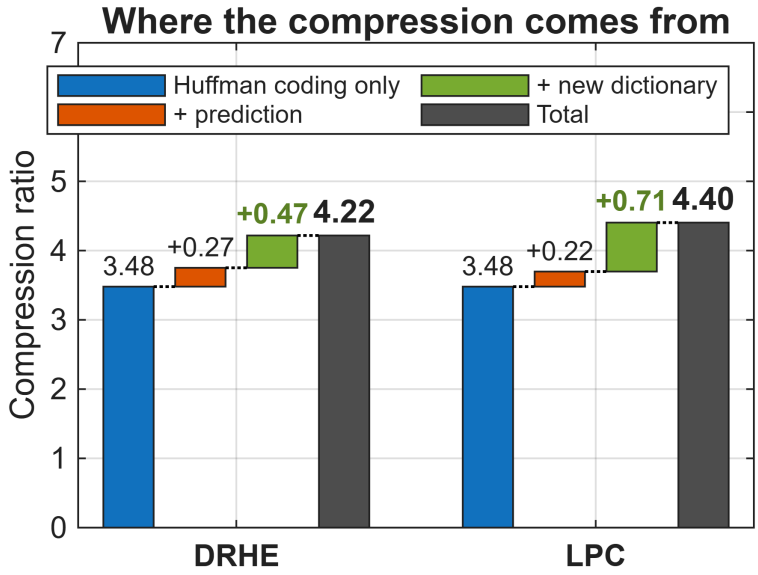


Figure 7.2: The compression ratio built up step by step. Huffman coding alone reaches 3.48; prediction adds 0.22 to 0.27; the re-derived dictionary adds more than prediction does, for both algorithms.

Three things stand out. First, most of the ratio is there before any prediction: the entropy coder alone achieves 3.48. Section 8.3 explains why — largely the unused top bits of the FX16 container, and the sparsity of a range-FFT frame. Second, prediction is worth a modest but real 6 % to 8 %. Third, the dictionary adds more than the predictor does: 0.47 for DRHE and 0.71 for LPC.

Figure 7.3 shows why prediction helps in the currency the coder charges in. The share of values that need zero, one or two bits rises, and the share that need three or more falls; the expensive tail almost disappears.

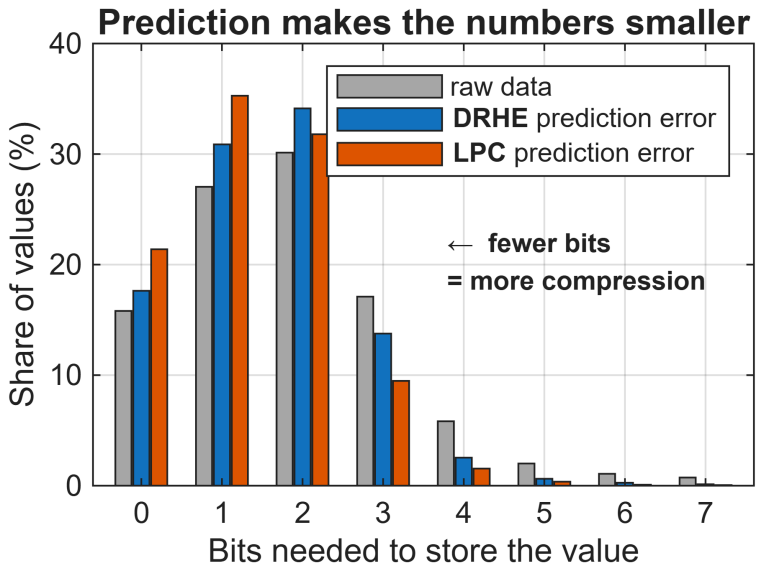


Figure 7.3: How many bits each value needs (S_4), for the raw FX16 codes and for the two algorithms' prediction errors. Prediction moves values towards the cheap end; most prediction errors need two bits or fewer.

7.4 The re-derived dictionary

Table 7.5 sets the share of DRHE's prediction errors in each size category beside the codeword length each dictionary gives it. The original dictionary gives its shortest codes to categories 2 to 4; on this data most errors fall in categories 0 to 2, and categories 3 and 4 hold only 16 %. The re-derived dictionary gives the three commonest categories codes of three, two and one bits, and the average codeword falls from 2.70 to 2.24 bits per value (from 2.80 to 2.12 for LPC).

Table 7.5: Share of DRHE's prediction errors by size S_4, and the codeword length each dictionary assigns (50 frames, four receive channels). Bold marks the three commonest sizes.

Size S_4	0	1	2	3	4	5	Average code
Share of errors (%)	**17.6**	**30.9**	**34.1**	13.8	2.5	0.6	—
Code length, original	4	3	2	2	2	5	2.70 bits
Code length, re-derived	**3**	**2**	**1**	4	5	6	2.24 **bits**

The dictionaries were trained on frames 0–4, so their real test is the 45 frames they did not see. On frames 5–49 DRHE reaches 4.207 and LPC 4.372 (Table 7.4), within 1 % of their all-frame figures: five frames of a scene are enough to fix a dictionary for the rest of the recording. The FPGA cores with the re-derived tables give 4.21704 and 4.40345 per frame, matching MATLAB, both in C-simulation and measured on the board (Section 7.6), and reconstruct all 50 frames exactly.

The dictionary also reverses the ranking of the two algorithms. With the original dictionary

DRHE leads by 1.5 %; with dictionaries matched to each algorithm's own errors LPC leads by 4.4 %. LPC's errors were always the smaller of the two — over five frames, 4.10 against 4.24 bits per value with the original dictionary, before LPC's 0.17 bits per value of coefficients — but a dictionary that cannot code anything in fewer than four bits hides most of that difference.

7.4.1 How close to the limit

The order-0 entropy of Equation (2.9) gives the floor for any code that treats values independently, and an *ideal* S_4 dictionary — a Huffman code built on each design's own category counts — shows how much of the gap to that floor is the dictionary and how much the S_4 construction itself. Table 7.6 gives both, in bits per value, over five frames.

Table 7.6: Bits per value of the residuals (coefficient table excluded), five frames, four receive channels: with the original dictionary, with an ideal S_4 dictionary, and the order-0 entropy of the values.

Residuals	Original dictionary	Ideal S_4	Entropy	Overhead of original
No prediction (raw FX16 codes)	4.586	4.214	4.094	12.0 %
DRHE, lag n_{Tx}	4.240	3.711	3.516	20.6 %
LPC, lags n_{Tx}, $2n_{\mathrm{Tx}}$	4.102	3.242	3.040	34.9 %

Two things follow. The S_4 construction itself is efficient: the ideal dictionary comes within 2 to 7 % of the entropy, so splitting each value into a coded size and raw payload bits loses little. Almost all of the original dictionary's excess is the dictionary, and it grows as prediction improves — from 12 % on the raw codes to 35 % on LPC's errors — because the better the predictor, the further its errors sit below the sizes the original dictionary favours. That is why re-deriving the dictionary gains more for LPC than for DRHE, and why the gain grows with prediction quality rather than shrinking.

7.5 Frame-to-frame variation

Figure 7.4 shows the ratio of every frame. It varies by about a third across the recording, from about 3.5 to 4.8, and the two algorithms move together. The variation tracks how much the scene contains: frames with more strong returns have larger prediction errors and compress less, so the per-frame ratio follows the scene rather than any property of the algorithms. For a system that must carry every frame, it is the worst frame, not the mean, that sizes the link; at

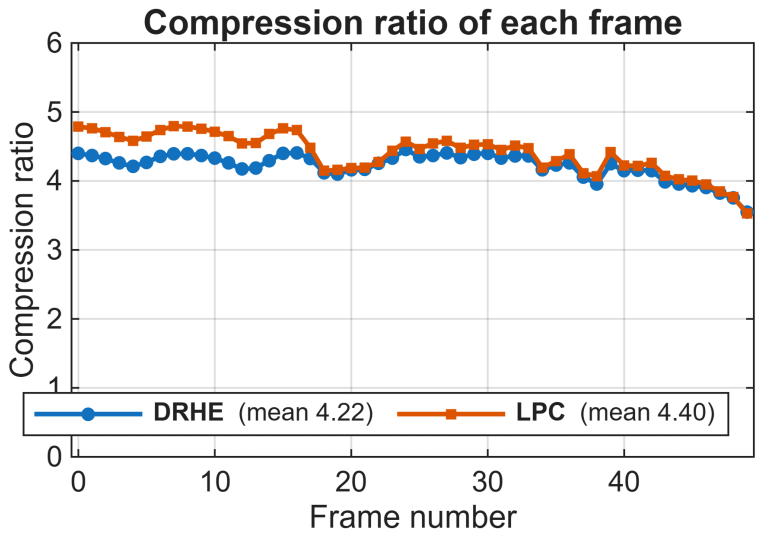


Figure 7.4: Compression ratio of each of the 50 frames, re-derived dictionaries. LPC is ahead on almost every frame; both vary with the scene in the same way.

3.5 it still leaves a wide margin over the 2.4–2.7 required.

7.6 Verification

Every level of the ladder of Section 5.4 that has been run passed with MaxDiff = 0:

- **C-simulation**, 50 frames (4,915,200 complex samples, 157,286,400 input bits per design): both compressors, with both the original and the re-derived dictionaries, reconstruct every sample exactly. The bit counts agree with MATLAB on every frame once the zero padding of the last 256-bit output word is counted: MATLAB's LPC model includes it and agrees exactly, and its DRHE model omits it, so the hardware's DRHE counts exceed MATLAB's by 15 to 252 bits, always less than one word.
- **RTL co-simulation**: both compressors reconstruct exactly.
- **Edge cases**: all eight synthetic cases of Table 5.1 pass for both algorithms. Uniformly random input *expands* by 67 % (CR 0.60), as any lossless coder must on incompressible data: every value lands in a high category, costing a long codeword plus up to 15 payload bits.
- **Board, compressors**: with both dictionaries, both compressors produced compressed output *byte-identical* to the C-simulation on all 50 frames, and every frame, decompressed on the PC, matched the input exactly. With the re-derived dictionaries the mean compression ratio on the board is 4.21704 for DRHE (range 3.548 to 4.454) and 4.40345

for LPC (3.526 to 4.793); with the original dictionary it is 3.7508 (3.517 to 3.815) and 3.6968 (3.461 to 3.761). All four are exactly the C-simulation figures.

- **Board, on-chip decompression**: in the `_full` builds the board compresses each frame, decompresses it with the hardware decoder and compares the reconstruction with the original word by word. Both decoders reconstruct all 50 frames exactly, and the compressed streams are again byte-identical to the C-simulation.

Agreement at the board level is the strongest result here: the bytes produced by the FPGA are the bytes the simulation predicted, and the board's own decoder turns them back into the input exactly. Every compression ratio reported for the hardware is therefore a measured figure, and every link of the chain — sensor data in, compressed stream, reconstructed data out — has been run on the FPGA.

7.7 Speed and latency on the board

Table 7.7 gives the timing measured on the board over 50 frames, and Figure 7.5 draws it.

Table 7.7: Timing measured on the KCU116 at 100 MHz, 50 frames of 393,216 B. Latency is from the last input word read to the last output word written (Section 5.3); for the decompressor, from the last compressed word read to the last reconstructed word written. Compressor figures are from the original-dictionary run; with the re-derived dictionaries they differ by at most 13 cycles.

	DRHE	**LPC**
Clock cycles per frame	61,472	325,366
Time per frame	0.61 ms	3.25 ms
Frames per second	1,627	307
Input throughput	5.12 Gbit/s	0.97 Gbit/s
End-to-end latency	13.1 µs	3,006 µs
as a share of the frame time	2 %	92 %
Decompressor cycles per frame	196,881	1,536,042
Decompression time per frame	1.97 ms	15.4 ms
Decompressor latency	339 µs	2,838 µs

DRHE streams. Its time per frame, 0.61 ms, is two clocks per input beat (II = 2) plus a short pipeline fill on each ramp. Its latency is 13.1 µs: the last compressed word leaves almost as soon as the last raw word arrives. The core's own pipeline accounts for about a microsecond of

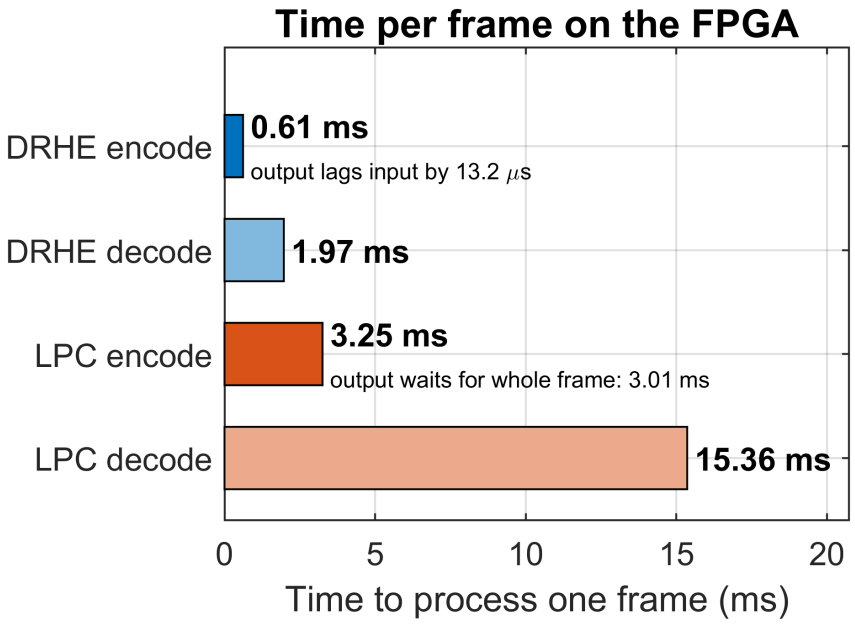


Figure 7.5: Time to encode and decode one frame on the FPGA, measured on the board with the re-derived dictionaries, with each encoder's output delay. DRHE outputs as the data arrives; LPC must wait for the whole frame before it can output anything. The LPC decoder's time comes from its serial codeword search (Section 7.7).

that; the rest is most likely data still in flight in the DMA and interconnect when the last input word is read, though this was not measured separately.

LPC waits. Its time per frame is 3.25 ms, 5.3 times DRHE's, and 92 % of it — 3.0 ms — comes *after* the last input word has arrived, because the coefficients cannot be computed until every ramp of the frame is in. That is 230 times DRHE's latency. It is not an implementation weakness that better scheduling would remove: as long as the coefficients are fitted to the frame being compressed, the first output bit has to wait for the last input sample.

Decoding. The DRHE decoder takes 1.97 ms per frame (196,881 cycles, range 196,858 to 196,919), almost exactly the one value per clock it was designed for (Section 6.5): a frame holds 196,608 values. It is 3.2 times slower than the compressor, by design, since it decodes one value per clock where the compressor encodes four channels at once. Its latency of 339 µs is the backlog this leaves: when the last compressed word has been read, about a sixth of the frame is still to be rebuilt.

The LPC decoder takes 15.4 ms per frame, eight times as long, and its time depends on the data: 1,501,789 to 1,623,838 cycles, or 7.6 to 8.3 cycles per value. The cause is its codeword matcher. Where the DRHE unpack stage compares all sixteen codewords with the input at once,

the LPC decoder tries them one after another in category order and stops at the first match, so the loop runs at no fixed rate; a value in a high category, which is rarer but costs more, takes more clocks to find. Nothing in LPC requires this — its arithmetic is two multiply–accumulates per value — and the parallel matcher of the DRHE decoder would bring it to about one value per clock, near 2 ms per frame. As built, it is the slowest part of either pipeline.

Both are fast in absolute terms. The ColoRadar cascade records at 5 Hz, a frame every 200 ms, so DRHE's compressor uses 0.3 % of the frame period and LPC's under 2 %; the slowest stage of all, the LPC decoder, uses 8 %. Section 8.2 considers what happens on larger frames.

7.8 Chip resources

Table 7.8 gives the post-route resources of each core, taken from the hierarchical utilisation report of the `_full` builds, and Figure 7.6 draws them as a share of the chip.

Table 7.8: Post-route resources of each core on the `XCKU5P` (216,960 LUT, 433,920 FF, 480 BRAM, 1,824 DSP). BRAM is in 36 Kb blocks.

		LUT	FF	BRAM	DSP
DRHE	compressor	35,444	23,528	40	132
	decompressor	14,155	9,245	32.5	46
	both	49,599 (23 %)	32,773	72.5	178
LPC	compressor	33,738	20,165	98	158
	decompressor	20,034	8,371	0	8
	both	53,772 (25 %)	28,536	98	166

Encoder and decoder together use under a quarter of the chip's logic for both algorithms, and the whole test system — processor, memory controller, DMAs and both cores — uses 38 % (DRHE) and 40 % (LPC) (Table 6.1). The two compressors cost almost the same logic. They differ in memory: LPC's compressor needs 98 block RAMs, most of them its frame buffer, against DRHE's 40 for predictor state. The decoders invert this. The DRHE decoder carries its own floating-point predictor and its state, while the LPC decoder needs no data memory at all and only eight DSP slices, because the coefficients arrive ready-made in the bitstream.

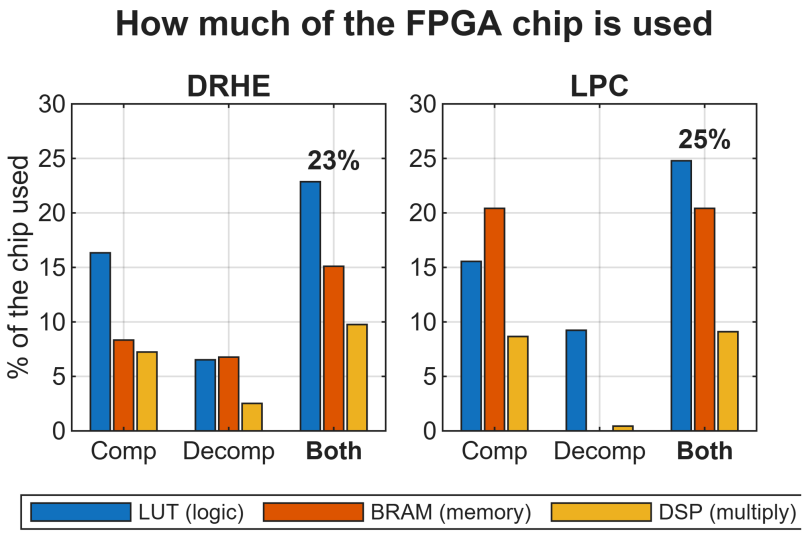


Figure 7.6: Share of the chip used by each core, for the compressor, the decompressor, and both together.

Table 7.9: DRHE and LPC compared on every measured quantity. Bold marks the better of the two on each row.

	DRHE	**LPC**
Lossless on the board (50 frames, both dictionaries)	yes	yes
Compression ratio, original dictionary (board)	3.75	3.70
Compression ratio, re-derived dictionary (board)	4.22	4.40
Time per frame	0.61 ms	3.25 ms
End-to-end latency	13.1 **µs**	3,006 µs
First output available	**immediately**	after the whole frame
Decompression time per frame	1.97 ms	15.4 ms
Compressor frame buffer	**none**	3.0 Mibit
Encoder + decoder LUTs	23 %	25 %
Encoder + decoder BRAM (36 Kb blocks)	**72.5**	98
Side information	**none**	1.04 %

7.9 The two algorithms side by side

Table 7.9 collects the results. On compression ratio the two are close, and which one leads depends on the dictionary: DRHE by 1.5 % with the original one, LPC by 4.4 % with dictionaries matched to each. On logic they are level. On time, latency and memory DRHE is clearly ahead: it compresses a frame five times faster, needs no frame buffer, and its output lags its input by microseconds rather than milliseconds. Its decoder is also eight times faster as built, though that gap comes from the LPC decoder's serial codeword search rather than from the algorithm. Chapter 8 weighs these against each other.

Chapter 8

Discussion

This chapter weighs the two algorithms against each other for the application that motivates the thesis, considers how the results scale and what the compression ratio really measures, compares the hardware and the results with the baseline, and sets out the limits of what the evidence supports.

8.1 Which algorithm for a sensor front end?

On compression ratio, DRHE and LPC are close, and which one leads depends on the dictionary rather than on the predictors (Section 7.4). On logic they are level. The decisive differences are elsewhere.

Streaming. DRHE's output depends only on samples already received, so it emits compressed data as the raw data arrives, needs no frame buffer, and its output lags its input by 13 µs. LPC's coefficients are sums over every ramp of the frame, so its first output bit has to wait for its last input sample: 3.0 ms of latency and a 3.0 Mibit buffer at this frame size. That is a property of the algorithm, not of this implementation; no amount of scheduling removes it while the coefficients are fitted to the frame being compressed.

Memory. A compressor between the range FFT and the memory bus of a radar sensor has little memory to spare, and LPC's buffer grows with the frame (Section 8.2). DRHE's state also grows, with the number of range bins and transmitters, but it is a fraction of a frame.

Where LPC is attractive. LPC's costs fall mainly on the compressor. Its decoder is the cheaper of the two in memory and arithmetic — no data memory, eight DSP slices — because the coefficients arrive ready-made. As built it is also the slowest stage measured, 15.4 ms per frame, but that comes from its serial codeword search, not from LPC, and the DRHE decoder's parallel matcher would bring it to about 2 ms (Section 7.7). Where latency of a few milliseconds is acceptable and the sensor has memory to spare, LPC's 4 % better ratio is available. A backward-adaptive LPC, fitting its coefficients to *previous* frames instead of the current one, would remove both the buffer and the latency at some cost in ratio, and is a natural next step.

Recommendation. For a streaming sensor-edge compressor, DRHE is the better choice: it is lossless, compresses by 4.2 with a matched dictionary, keeps up with the data with a wide margin, needs no frame buffer, and adds microseconds rather than milliseconds of latency. LPC compresses about 4 % better, but only by waiting for and storing every frame.

8.2 Scaling to larger frames

The hardware path compresses four of the cascade's sixteen receive channels, because four complex 16-bit samples fill the 128-bit stream. The following estimates scale the measured figures linearly; they are not measurements.

- **All 16 receive channels** quadruple the data per frame. DRHE would take about 2.5 ms per frame and LPC about 13 ms, both comfortably within the 200 ms frame period of the ColoRadar capture, but LPC would need a 12 Mibit frame buffer and its latency would grow to about 12 ms.
- **Kiem's use case** — 1024 ramps of 1024 samples, four channels, a 50 ms cycle and a 15 ms compression budget — needs 2.4 Gbit/s at the ADC. After the range FFT each frame is 8.4 MB of complex 16-bit data (512 bins of 16-bit I and Q per ramp and channel), delivered in the 21 ms of acquisition: about 3.2 Gbit/s at the compressor's input. DRHE's measured 5.1 Gbit/s keeps up without any buffer. LPC as built would take about 69 ms per frame, longer than the whole cycle, and would need an 8.4 MB frame buffer against roughly 4.4 MB of block and UltraRAM on this device.
- **The decoders** scale with the number of values, at about one per clock for DRHE and about eight for LPC as built. At 16 receive channels they would take about 8 ms and 61 ms per frame. A frame of Kiem's size holds 21 times as many values as the frames

measured here, so the DRHE decoder would take about 42 ms, inside the 50 ms cycle, and the LPC decoder about 330 ms, which would need the parallel matcher or several decoders. A decoder sits at the central processor, where several copies or a wider design are easier to afford than at the sensor.

DRHE scales with the data rate; LPC scales with the frame size, and on large frames it runs out of both time and memory. Its time per frame could be cut by about a third by overlapping its passes and amortising the per-bin pipeline fill (Section 6.3), and further only by processing several bins in parallel, but its buffer and its latency would remain.

8.3 What the compression ratio measures

Figure 8.1 follows one value through the compressor, in bits. The 16-bit container starts with 16. Because the FX16 scale is fixed to a theoretical full-scale signal and the ColoRadar capture sits far below it, the largest code over 50 frames is 437, so only $\log_2 437 + 1 = 9.77$ bits ever carry information; the other 6.2 are copies of the sign bit. Removing them is a factor of 1.64, and any entropy coder does it whether or not anything is predicted. The coder then exploits the sparsity of a range-FFT frame — most bins hold noise near zero and a few hold targets — for a further factor of 2.12, reaching 4.60 bits per value. Only then does prediction act, taking DRHE to 4.27 bits (a factor of 1.08), and the re-derived dictionary takes it to 3.79 (1.12).

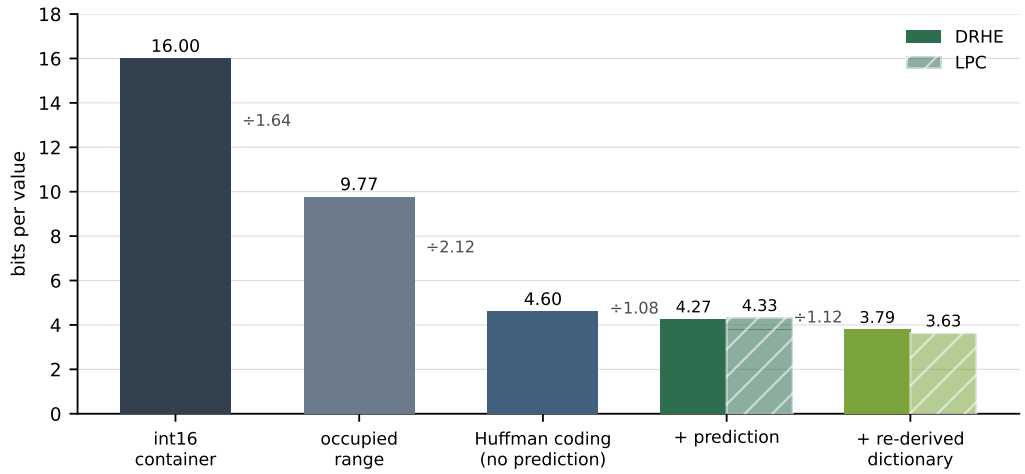


Figure 8.1: Bits per value through the compressor, 50 frames, four receive channels. Most of the reduction comes from the unused top of the 16-bit container and from the sparsity of the data, before any prediction. Factors are for DRHE; LPC is shown hatched.

Two consequences follow.

The ratios are partly a property of the capture. A sensor whose data filled the 16-bit container would have no unused top bits to remove. Its extra bits would instead hold finer detail of the receiver noise, which a lossless coder must keep, and the quantisation study of Section 7.1.1 shows what that does: scaled to fill the container, the same frame compresses by only 1.2. Ratios measured on different sensors are therefore not directly comparable unless their gain staging is. A no-prediction baseline should be reported alongside any radar compression ratio; it costs a few lines of MATLAB and separates the algorithm from the sensor.

The unused bits are a design choice. The same unused bits are why FX16 loses 3.77 % of targets: filling the container removes that loss and, with it, most of the compression (Section 7.1.1). How many bits of the noise floor are worth transmitting is a decision for the radar system designer; a lossless compressor does not change that trade, it only makes whatever choice is made cheaper to carry.

8.4 How far the dictionary and the lag generalise

The re-derived dictionaries were trained on five frames of one recording and tested on 45 later frames of the same recording, where they hold within 1 %. That shows they do not overfit the training frames; it does not show they transfer to other scenes, sensors or gain settings. Because a dictionary is a sixteen-entry table, a production design could hold several and choose per frame, or update one slowly from the statistics of past frames, without changing anything else in the hardware.

The prediction lag is specific in its value and general in its principle. It is $n_{\mathrm{Tx}} = 12$ because this radar rotates twelve transmitters in a fixed order; a radar that changed its MIMO schedule would need the lag reloaded, and one using Doppler- or code-division multiplexing has no interleaved ramp axis. The principle — a slow-time predictor must predict from the same antenna — applies to any slow-time coder carried over from a single-transmitter design to a time-multiplexed one, and the check is cheap: if the predictor does not beat the no-prediction baseline, it is predicting along the wrong axis.

8.5 Hardware cost against the baseline design

Table 8.1 sets the hardware beside Kiem's. The designs differ in board, data and system, so the comparison is of methods rather than of chips.

Table 8.1: This thesis's hardware against Kiem's [11]. Different boards (Zynq `XCZU3CG` against Kintex `XCKU5P`, both UltraScale+), radars and data. Core resources are the compressor alone; BRAM in 36 Kb blocks.

	Kiem	This work: DRHE	This work: LPC
Lossless CR in hardware	3.17	3.75	3.70
CR, re-derived dictionary (board)	—	4.22	4.40
Hardware checked against simulation	FFT model only[c]	byte-identical	byte-identical
Decompressor	none	on chip, 1.97 ms	on chip, 15.4 ms
Throughput	11.92 Gibit/s (theor.)	4.77 Gibit/s (meas.)	0.90 Gibit/s (meas.)
End-to-end latency	79.4 µs[b]	13.1 µs	3,006 µs
Compressor LUT / FF	45,892 / 9,243	35,444 / 23,528	33,738 / 20,165
Compressor BRAM / DSP	10 / 44	40 / 132	98 / 158
Arithmetic	fixed point, II = 1	floating point, II = 2	fixed point, II = 1

[b] includes his FFT core.
[c] a bit-accurate model of his FFT core explained the ratio gap; no decoder.

Appendix A accounts for every row of this table: how Kiem's throughput figure is a theoretical rate in binary units, why the two latencies measure the same kind of quantity over different pipelines, and — through a rebuilt replica of his design and a ladder of single-change variants — which design decision causes each difference in resources. The short version is that the floating-point predictor of this thesis's DRHE is the largest cost, and buys only 0.07 % of compression ratio over fixed point.

8.6 Comparison with the baseline's results

Appendix B sets every one of Kiem's published results beside this thesis's. Its main conclusions are three. Kiem's simulation, rebuilt from his text, reaches his DRHE ratios (3.25 with run-length coding, 3.24 without) at one noise level that his text leaves open, which suggests — it cannot prove it, since his recordings are not public — that his dictionary and his ratio reflect residuals of about 6 to 8 LSB. The DRHE implemented here is his specification to the bit, apart from the prediction lag, which on his single-transmitter radar is 1. And the LPC-Huffman method he excluded compresses poorly on his scene, as he expected, while the slow-time LPC of this thesis is a different design that compresses as well as DRHE. Kiem's DRHE therefore carries over to a twelve-transmitter cascade unchanged except for the lag; with a matched

dictionary it compresses public cascade data by 4.2 (a different sensor and scene from his 3.17 in hardware and 3.25 in simulation, so not a like-for-like comparison), and it comes with a hardware decoder that reconstructs the board's output exactly on the board itself, which his system did not have.

8.7 Observations on verifying a compressor

Three observations from verifying these designs apply beyond them.

Dataset tests and adversarial tests find different problems. Fifty frames of real radar data — 157 million input bits — exercise the typical case exhaustively and the extreme case not at all: ColoRadar's prediction errors never leave category 10 of the 16-category code, so a third of the coder is never executed. An eight-case synthetic suite exercises exactly that third, and it is what makes the designs lossless on *any* input (Section 6.4). Conversely, an edge-case suite says nothing about whether a predictor helps on real data: its cases are built to stress the coder, not to resemble a scene. A synthetic scene can be given transmitter structure, as Section B.6 does, but how much prediction then gains depends on how the scene was made, and only real data settles it. Neither kind of test substitutes for the other.

Lossless and correct are not the same as useful. A compressor can be bit-exact at every level, compress by a ratio in the expected range, and still have a predictor that contributes nothing — as lag-1 prediction on this radar shows (Section 7.2). No losslessness test can detect that, because the entropy coder carries on compressing whatever it is handed. What detects it is one extra measurement: the same coder with no predictor. It costs a few lines of code and belongs in every evaluation of a predictive compressor.

Cycle models hold only for the design shape they were checked on. Trip count times initiation interval, summed over loops, is a good model of a design with a few long loops; it is wrong for a design with many short loop invocations, where the pipeline fill paid on every entry dominates. LPC's per-bin pass is the second kind (Section 6.3): the simple model misses about 65,000 of its cycles. A model validated on one design gives no guarantee on another; cycle-accurate co-simulation and the board are what settle it.

8.8 Threats to validity

- **The ratios are capture-specific.** Much of the ratio comes from an under-filled container (Section 8.3), a property of this recording's gain staging.
- **One recording.** All results use 50 consecutive frames of one ColoRadar sequence. Scenes with more or faster targets would give larger prediction errors and lower ratios, and the re-derived dictionaries were trained and tested on the same recording.
- **The detection metrics measure the quantisation step.** The three lossless schemes share FX16's detection metrics by construction. The detection chain is also simpler than a production one: its Doppler FFT runs across all 192 ramps in recorded order, it has no angle FFT, detections are matched cell by cell with no positional tolerance, and the threshold is not swept. The absolute rates of the lossy schemes should therefore be read as relative.
- **BAQ is a block-floating-point stand-in.** A better-tuned, statistics-driven BAQ could lose fewer detections at the same ratio; it would still be lossy.
- **Channel count differs between stages.** Stage 1 uses all 16 receive channels; Stage 2 and the hardware use four. The two stages' ratios are close (3.75 and 3.751 for DRHE) but not identical by construction.
- **Power was not measured.** All four bitstreams were run on the board and every figure of compression, timing and correctness is measured, but the board's power draw was not, so the energy cost of compression is not reported.
- **Co-simulation covers one frame.** The 50-frame results are from C-simulation and the board, which exercise identical logic; the cycle-accurate RTL simulation was run on one frame.
- **The baseline was rebuilt from its description.** Kiem's source code and recordings are not public. The replica of his hardware and his simulation follow every type, function and parameter his thesis states; the parts it does not state are this thesis's reading of it.

Chapter 9

Conclusions and Future Work

9.1 Summary

This thesis set out to compare data compression methods for SoC-based radar on real, public, multi-transmitter data, to select the best lossless ones, and to implement their encoders and decoders on an FPGA and measure them on a board. The four objectives of Section 1.4 were met.

Eight methods were compared in MATLAB on 50 frames of ColoRadar data from a twelve-transmitter, sixteen-receiver Texas Instruments cascade. Two lossless predictive coders — Doppler Redundancy Hybrid Encoding and a second-order linear predictive coder — were specified, implemented in C++ for Vitis HLS with matching decompressors, and built with Vivado into a test system on an AMD KCU116 board. They were verified bit for bit from MATLAB through C-simulation and RTL co-simulation to the board, where the compressors' output matched the simulation byte for byte and the hardware decoders reconstructed every frame exactly. A third, independent implementation in Python reproduced their bit counts.

9.2 Findings

1. **Only lossless predictive coding meets the requirement.** DRHE (3.75) and LPC (3.70) are the only methods that compress by more than three without adding a missed target. The lossy quantisers reach up to 3.76 but miss 10 % to 29 % of targets, and floating-point storage compresses by only 1.04. All detection loss in the lossless chain comes from the 16-bit quantisation step, not from compression.

2. **On a TDM-MIMO radar, predict from the same transmitter.** Slow-time prediction from the adjacent ramp, which on this radar is another transmitter, compresses worse than no prediction at all. Predicting $n_{\mathrm{Tx}} = 12$ ramps back, from the same antenna, makes prediction worth 6 % to 8 %. For DRHE the change is an index into the state and nothing else.
3. **Most of the ratio comes before prediction.** Huffman coding alone compresses this data by 3.48, mainly because the fixed 16-bit scaling leaves six bits of every value unused and because a range-FFT frame is sparse. Filling those bits removes the quantisation error but also almost all of the compression. A no-prediction baseline should be reported with any radar compression ratio.
4. **The dictionary matters more than the predictor.** A Huffman dictionary re-derived from five frames for each algorithm's prediction errors raises DRHE from 3.75 to 4.22 and LPC from 3.70 to 4.40 on the board, holds on the 45 frames it was not trained on, changes only the contents of a table in hardware, and puts LPC ahead of DRHE on ratio.
5. **The hardware is exact and fast.** On the KCU116 at 100 MHz, both compressors produced output byte-identical to the simulation on all 50 frames, and the board's own decoders reconstructed every frame exactly. DRHE compresses a frame in 0.61 ms (5.1 Gbit/s) with 13 µs of latency; LPC in 3.25 ms with 3.0 ms of latency, because it must see the whole frame first. The decoders take 1.97 ms and 15.4 ms; the LPC decoder's time comes from a serial codeword search, not from the algorithm. Encoder and decoder together use 23 % and 25 % of the chip's logic.
6. **DRHE is the better choice for a streaming sensor.** The two algorithms are close on ratio and level on logic, but DRHE needs no frame buffer, streams its output, and scales with the data rate, while LPC's buffer and latency grow with the frame until, at the frame size of Kiem's use case, it could not keep up.
7. **The baseline carries over, with one change.** Kiem's DRHE is reproduced bit for bit and works on a cascade once the prediction lag is set to the transmitter count; with a matched dictionary it compresses public cascade data by 4.2. Unlike his system it has a hardware decoder, and the board's compressed output reconstructs exactly on the board.

9.3 Future work

Measure power, and speed up the LPC decoder. Measure the board's power draw for each build, to give the energy cost of compression. Replace the LPC decoder's serial codeword search with the parallel matcher of the DRHE decoder, which should cut its 15.4 ms per frame to about 2 ms.

Move DRHE to fixed-point arithmetic. The floating-point datapath is the most expensive design decision and buys 0.07 % of ratio. Kiem's fixed-point types would remove most of its logic, DSPs and pipeline depth while remaining exactly lossless, since the closed loop makes the decoder follow whatever arithmetic the encoder uses. With fixed point, reading the state as zero on the first ramps instead of writing it would also reach $\mathrm{II} = 1$ almost for free.

A streaming LPC. A backward-adaptive LPC that fits its coefficients to previous frames would need no frame buffer and add no frame of latency, combining LPC's better prediction with DRHE's streaming. For the forward LPC, overlapping the three passes and amortising the per-bin pipeline fill would cut its time per frame by about a third, and processing several bins in parallel by more.

Dictionaries that adapt. Train dictionaries on more varied scenes, or let the hardware hold several and choose per frame, or update one slowly from past frames. A seventeenth category for the value -32768 would remove the need for saturation.

All sixteen channels and the range FFT on chip. Widen the stream or time-multiplex the cores to compress all sixteen receive channels, and place a range-FFT core in front of the compressor, as in Kiem's system, so that the board processes raw ADC data end to end.

The quantisation trade-off. Study adaptive or companded scaling as a deliberate design choice: how many bits of the noise floor are worth keeping for the detection margin they buy, and what that costs in link bandwidth.

Other multiplexing schemes and methods. Doppler- and code-division MIMO distribute the slow-time structure differently, and whether slow-time prediction helps on them is open.

Learned and spectral methods such as AdaRadar [24] define the upper end of achievable ratios and could be compared against the lossless coders on the same data.

Bibliography

[1] C. Waldschmidt, J. Hasch and W. Menzel, "Automotive radar — from first efforts to future systems," *IEEE Journal of Microwaves*, vol. 1, no. 1, pp. 135–148, 2021.

[2] S. Yao *et al.*, "Exploring radar data representations in autonomous driving: a comprehensive review," arXiv preprint, 2023.

[3] M. I. Skolnik, *Radar Handbook*, 3rd ed. New York, NY: McGraw-Hill, 2008.

[4] V. Bandur *et al.*, "Making the case for centralized automotive E/E architectures," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.

[5] Z. Liu, W. Zhang and F. Zhao, "Impact, challenges and prospect of software-defined vehicles," *Automotive Innovation*, vol. 5, no. 2, pp. 180–194, 2022.

[6] I. Barabás *et al.*, "Rethinking vehicle architecture through softwarization and servitization," *IEEE Access*, vol. 12, pp. 110782–110795, 2024.

[7] Texas Instruments, "FPD-Link III serializer/deserializer (SerDes) for sensor applications," Application Note SNLA344, 2021.

[8] Analog Devices, "GMSL SerDes technology for automotive applications," White Paper, 2022.

[9] K. Matheus and T. Königseder, *Automotive Ethernet*. Cambridge, UK: Cambridge University Press, 2021.

[10] IEEE, "IEEE P802.3bp 1000BASE-T1 PHY task force," 2016.

[11] L. Kiem, "Design and implementation of data compression methods for SoC-based radar systems," Master's thesis, Institute of Technical Informatics, Graz University of Technology, Graz, Austria, May 2025.

[12] A. Kramer, K. Harlow, C. Williams and C. Heckman, “ColoRadar: the direct 3D millimeter wave radar dataset,” *The International Journal of Robotics Research*, vol. 41, no. 4, pp. 351–360, 2022.

[13] Texas Instruments, *AWR2243 Single-Chip 76- to 81-GHz FMCW Transceiver Datasheet* (Rev. D), February 2024.

[14] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.

[15] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[16] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. Oxford University Press, 2010.

[17] IEEE, “IEEE standard for floating-point arithmetic,” IEEE Std 754-2019, 2019.

[18] B. Widrow and I. Kollár, *Quantization Noise*. Cambridge University Press, 2008.

[19] I. Kuon, R. Tessier and J. Rose, “FPGA architecture: survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.

[20] A. Mani and Z. Yang, “Memory compression and decompression engine for TI mmWave radar,” Technical Report, Texas Instruments, 2019.

[21] NXP Semiconductors, “S32R RADAR signal compression,” Application Note AN5375, 2021.

[22] R. Kwok and W. T. K. Johnson, “Block adaptive quantization of Magellan SAR data,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 27, no. 4, pp. 375–383, July 1989.

[23] F. Cheng *et al.*, “Autoencoder-based feature compression for radar and vision sensor fusion in autonomous driving,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12345–12355, 2021.

[24] J. Park, S. Y. Chun and M. Seok, “AdaRadar: rate adaptive spectral compression for radar-based perception,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[25] Z. Zivkovic and L. Li, "On-the-fly range-velocity radar data compression exploring Doppler spectrum redundancy," in *Proceedings of the 16th International Radar Symposium (IRS)*, Dresden, Germany, 2015, pp. 321–326.

[26] L. Li, "Embedded data compression in automotive FMCW radar system," MSc thesis, University of Twente, Enschede, The Netherlands, 2014.

[27] G. Meucci and F. Mancuso, "FMCW radar data compression," project report, Data Compression course, PhD programme in Information Engineering, University of Pisa, 2022. [Online]. Available: https://github.com/mandugo/FMCW-Radar-Data-Compression

[28] J. Deng and L. Huang, "Synthetic aperture radar image compression based on low-frequency rejection and quality map guidance," *Remote Sensing*, vol. 16, no. 5, art. 891, 2024, doi:10.3390/rs16050891.

Appendix A

Interpreting the Hardware Numbers

Chapter 7 reported the hardware as it runs on the KCU116: the cycles, time and latency of each frame, the throughput, and the resources of each core. Kiem's thesis reports a comparable set of numbers for his DRHE compressor [11]. Set side by side, the two sets differ on almost every line — logic, registers, memory, DSP slices, pipeline depth, initiation interval, throughput and latency — and a difference on its own explains nothing. This appendix interprets every one of these numbers: it says what each one represents, what it should be compared against, how far it can be trusted, and, wherever this thesis's design and Kiem's differ, exactly why.

The interpretation rests on three measurements rather than on argument.

1. **Put the numbers on the same footing.** A hardware figure can be an estimate made by the high-level synthesis (HLS) tool before the circuit exists, or a measurement of the circuit after Vivado has placed and routed it. Kiem's resource and clock figures are of the second kind. Every design in this chapter was therefore implemented through placement and routing, and the two kinds of number are compared directly (Section A.1).
2. **Rebuild the reference in the same tool flow.** Two designs built with different tools, for different devices, from different code differ in many ways at once. To separate them, Kiem's compressor was rebuilt from its description, in this thesis's tool, for this thesis's device (Section A.4).
3. **Change one thing at a time.** Starting from this thesis's DRHE datapath and ending at the rebuilt reference, one design decision is changed per step, and every step is implemented (Section A.5). The change in each number at each step is the cost of that one decision.

The rest of the appendix then reads each metric — latency, throughput, resources, power and

compression ratio — against the yardsticks that give it meaning: the sensor, the system's time budget, the link, the device, and Kiem's design where the comparison is like for like. It closes with a ledger of every difference from Kiem's design, its cause and the evidence for it (Section A.11).

Which designs appear. The designs of this thesis predict at lag $n_{\mathrm{Tx}} = 12$ (DRHE) and at lags n_{Tx} and $2n_{\mathrm{Tx}}$ (LPC); these are the cores on the board and every board figure refers to them. Two further variants appear as comparison points only: the same DRHE datapath predicting at lag 1, called *DRHE-1* for short in this appendix, and the same LPC datapath at lags 1 and 2 (*LPC-1,2*). Their datapaths are identical to those of the final designs except for how the predictor state is indexed (Section 6.2), so their resource and timing figures differ from the final designs' only where that indexing matters, and the appendix says where that is. DRHE-1 has a particular role: Kiem's sensor has a single transmitter, for which lag 1 *is* the per-transmitter lag, so DRHE-1 is the version of this thesis's datapath that implements the same predictor as his, and it is the natural base for a step-by-step comparison with his design.

A.1 Two ways of counting the same circuit

HLS reports what it *expects* a circuit to cost, before the circuit has been built. Vivado reports what the circuit *does* cost, once it has been synthesised into device primitives, optimised, placed and routed. The two numbers are produced for different purposes and can differ widely. Kiem's figures, from his Table 5.3, are the second kind [11]. A comparison with them is meaningful only with numbers of the same kind, so every core in this appendix was implemented, and the two kinds of number are reported side by side.

A.1.1 Implementing a core on its own

`vitis-run --mode hls --impl` takes the RTL that HLS generated and runs the complete Vivado flow on it: logic synthesis, optimisation, placement, routing and post-route reporting. AMD calls this flow "export and implement", and its reports are the same kind of number as Kiem's Table 5.3. Four lines in each design's configuration file select it, and nothing else about the design changes:

```
vivado.flow=impl    vivado.clock=10    vivado.report_level=2
vivado.max_timing_paths=10
```

Each design is implemented as a standalone block ("out of context") constrained to a 10 ns clock on the `xcku5p-ffvb676-2-e`, the device of the KCU116. One run takes about 45 min. It leaves a post-route utilisation report (total and per module), a timing report with the worst paths, and a power report. Out-of-context runs make it cheap to implement many variants of one core, which the ladder of Section A.5 needs; the board cores of Chapter 7 are implemented in context, inside the full test system of Section 6.6.

The two kinds of implementation agree closely in logic and arithmetic. The DRHE compressor uses 34,804 LUTs and 23,514 flip-flops out of context and 35,444 LUTs and 23,528 flip-flops inside the board system, a difference of under 2 %; the LPC compressor uses 34,503 and 33,738 LUTs respectively. DSP counts are identical (132 and 158). Block RAM agrees exactly for DRHE (40 blocks of 36 Kb both ways) but not for LPC, whose memories Vivado maps differently in the two runs: 108 blocks out of context against 98 in the board system. Out-of-context figures can therefore stand in for in-system ones when logic, registers and DSPs of variants are compared with each other, and for block RAM only to within about 10 % for LPC; and the in-system figures of Table 7.8 are the ones quoted for the final cores.

A.1.2 Estimates against routed results

Table A.1 sets the two kinds of number side by side for the two final compressors and their lag-1 variants.

Table A.1: HLS estimates against the same compressors after Vivado placement and routing, out of context, on the `xcku5p-ffvb676-2-e` with a 10 ns constraint. BRAM is in 36 Kb blocks (HLS reports BRAM_18K halves; its counts are halved here). $F_{\max}$ is $1/(10\,\text{ns} - \text{WNS})$ after routing, and HLS's own estimate before it. The lag-1 variants are comparison points.

	LUT		FF		DSP		BRAM (36 Kb)		$F_{\max}$ (MHz)	
Design	HLS	impl.	HLS	impl.	HLS	impl.	HLS	impl.	HLS	impl.
DRHE, lag n_{Tx}	115,736	34,804	57,263	23,514	132	132	40	40	81.9	**131.0**
LPC, lags $n_{\text{Tx}}, 2n_{\text{Tx}}$	86,298	34,503	23,154	20,186	156	158	96	108	85.4	**106.3**
DRHE-1 (lag 1)	115,534	35,343	57,105	23,546	132	132	20	20	81.9	135.5
LPC-1,2 (lags 1, 2)	80,528	29,772	17,393	13,601	168	168	128	122	85.4	118.6

Three features stand out.

- **HLS over-estimates logic by about three times.** HLS estimates 115,736 LUTs for the DRHE compressor; the routed design uses 34,804, a ratio of 3.3. The LPC compressor's ratio is 2.5, and flip-flops are over-estimated by factors of 1.1 to 2.4. The DRHE decompressor of Section 6.5 follows the same pattern: 42,846 LUTs estimated, 14,155 used.
- **Every design meets** 100 MHz**.** HLS estimates 81.9 MHz for DRHE and 85.4 MHz for LPC, both below the target. After routing, the DRHE core on its own (out of context) closes at 131.0 MHz and the LPC core at 106.3 MHz; the complete board systems containing them meet 100 MHz with 0.17 ns to 0.34 ns of slack (Table 6.1).
- **DSPs and block RAM are estimated almost exactly.** These are fixed blocks of known size that HLS allocates itself, so there is little for the placement and routing tools to change. The exceptions are the LPC designs' block RAM, where Vivado re-maps some memories (96 blocks of 36 Kb estimated against 108 used for the final LPC, 128 against 122 for LPC-1,2), and two DSP slices in the final LPC.

A.1.3 Why the logic estimate is so far off

The per-module breakdown of the DRHE compressor in Figure A.1 shows where the difference comes from. It is not spread evenly across the design.

The loop body. Most of the estimate sits in the logic that HLS generates directly from C++: the bit packer, the S_4 and Huffman lookup, the residual arithmetic and the control. Each of the eight inserts into the 512-bit packing register (Section 6.1) is written as `(ap_uint<512>)code << bit_count`, and HLS costs every one of them as a complete 512-bit barrel shifter, about 2,171 LUTs. Because `bit_count` is a signed `int`, it also adds a second shifter in the opposite direction for negative shift amounts, and a 512-bit select between the two. Vivado sees what HLS does not: the value being shifted is at most 29 bits wide and zero everywhere else, so most of every shifter's inputs are constant and the logic behind them disappears. The loop body is estimated at about 60,000 LUTs and implemented in about 9,900.

The arctangent. `hls::atan2(float, float)` is built by HLS from generic C++ — a CORDIC on a 43-bit internal fixed-point format with range reduction around it — rather than taken from a pre-characterised core. HLS estimates each instance at 20,129 LUTs; each

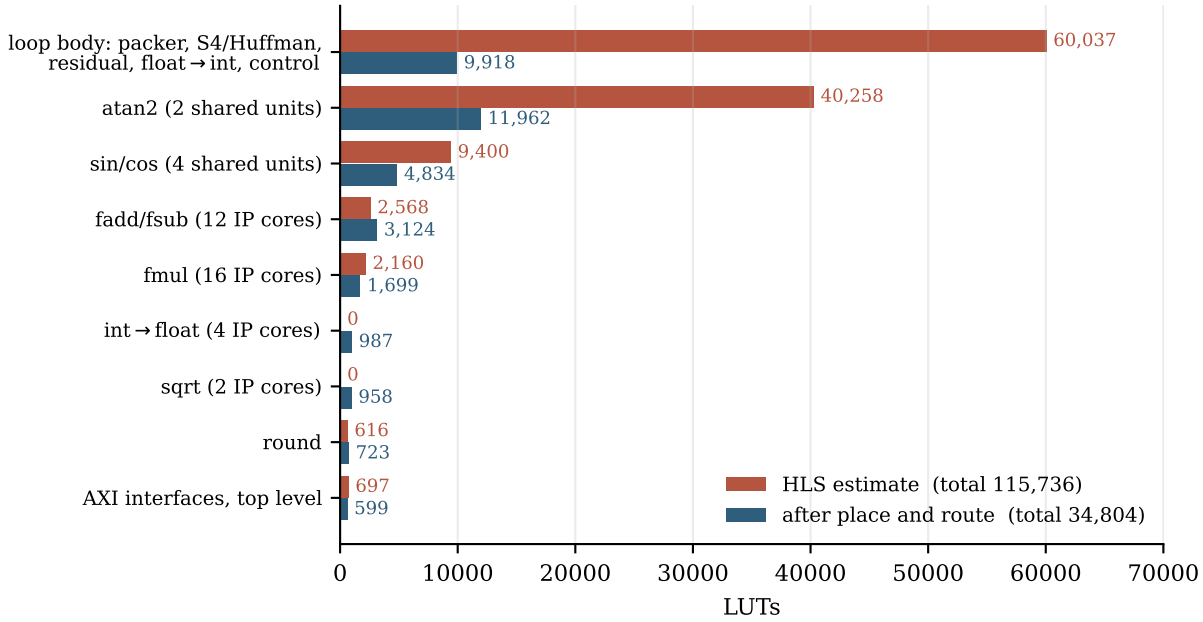


Figure A.1: LUTs of each part of the DRHE compressor (lag n_{Tx}): the HLS estimate (from each module's synthesis report) against the routed design (from the hierarchical utilisation report). The pre-built floating-point adders and multipliers are estimated well. The arctangent and the loop body, which HLS builds itself from C++, are over-estimated by factors of about three and six. The square root and the integer-to-float conversions are not estimated at all.

implemented instance uses about 5,980. The same effect, smaller, applies to `sin` and `cos`.

What HLS does not count. The square root and the integer-to-float conversions are AMD floating-point IP cores that use no DSPs, and HLS reports them as zero LUTs. Implemented, they cost 1,945 LUTs between them. This is the one place where the estimate is too low.

The clock. HLS estimates a critical path of 12.213 ns through an integer-to-float conversion chained into a multiply. That figure is HLS's own model of each operator's delay, scheduled against a budget that already reserves 2.7 ns (27 per cent of the period) for clock uncertainty; it is not a timing analysis of real wires. After routing, the worst path in the DRHE compressor is 7.633 ns long, starts in one of the floating-point sine/cosine units, and passes the 10 ns constraint with 2.367 ns to spare.

What this means for the comparisons. HLS estimates are useful for what they are designed for: comparing one version of a design against another in the same tool, quickly, before committing to a long implementation run. They are not a substitute for implementation when the comparison is with a design that was implemented, and they are not a basis for deciding whether

a design fits a device or meets its clock. Every hardware number in this appendix that is compared with Kiem's is therefore a post-route number, and every timing claim is a routed timing claim.

A.2 Where the numbers come from, and how far to trust them

Table A.2 sorts every hardware number in this thesis by where it is read from and what kind of number it is.

Table A.2: Where each hardware number is read from, and what kind of number it is. *top* is the name of the core's top-level function. All tool reports are produced by Vitis HLS and Vivado 2026.1 and read by a script (`collect_impl.py`); board records are read by `compare_results.py` (Section 6.6).

Metric	Read from	What kind of number
Core LUT, FF, BRAM (36 Kb), DSP (board)	hierarchical utilisation report of the `_full` system builds	*Measured* on the routed system.
Core LUT, FF, BRAM (36 Kb), DSP (variants)	*top*`_export.rpt`, post-implementation section; *top*`_utilization_hierarchical_routed.rpt`	*Measured* on the routed core, out of context.
Clock (F_{max})	*top*`_timing_routed.rpt`, worst negative slack	*Measured*: static timing analysis of routed wires; $F_{max} = 1/(T - \text{WNS})$.
Power	*top*`_power_routed.rpt`	*Estimated* from the routed design with default switching activity.
HLS estimates	`csynth.rpt`, top row	*Estimate* made before the circuit exists.
II, pipeline depth	`csynth.rpt`, loop table; `*.verbose.sched.rpt`	*Exact* for the generated schedule.
Cycles per frame, latency (board)	firmware result records, hardware timer	*Measured* on the KCU116, 50 frames.
Cycles per frame (variants)	*top*`_cosim.rpt`, Verilog row	*Measured*: cycle-accurate simulation of the generated RTL.
Compression ratio	board records; C-simulation logs	*Measured*: bits in divided by bits out, on real data.

Several limits should be kept in mind.

Out-of-context timing is slightly optimistic. A core implemented on its own does not see the clock tree of a real system and cannot include its skew, nor the congestion of other logic

placed around it. Its F_{max} is therefore an upper bound on what the same core achieves in a system. The board builds show the size of the effect: the DRHE core alone has 2.367 ns of slack at 10 ns, while the whole DRHE system closes with 0.243 ns (compress-only) and 0.341 ns (`_full`) (Table 6.1). The system met its target in every case, which is the claim that matters, but a system clock much faster than 100 MHz would have to be closed for the whole system, not only for the core.

Power is an estimate. The power figure uses Vivado's default assumption that every signal toggles on 12.5 % of clock cycles, not activity recorded from real data. It is good for comparing designs with each other and for the order of magnitude, but it is not a prediction of a particular system's power (Section A.9).

Co-simulation and the board agree. The cycle count of a frame is measured twice: in RTL co-simulation, where the testbench feeds the frame back-to-back at one beat per cycle whenever the core will accept it, and on the board, where the DMA feeds it from DDR4. For DRHE the two are 61,253 and 61,472 cycles; for LPC 325,166 and 325,366. The board figure is longer by 219 and 200 cycles respectively — about 0.4 % and 0.06 % — which is the time for the DMA to start the stream and to write the last output word, neither of which the testbench models. Co-simulated cycle counts of the variants in this appendix can therefore be read as board figures to within a fraction of a per cent.

The frame time is a full-speed figure. Both the co-simulated and the board frame times measure one specific situation: the whole frame is waiting in memory and is fed to the core as fast as it will accept it. So 61,472 cycles is the time the DRHE compressor takes if an entire frame is available at once. The real sensor does not deliver data like that, which is why Section A.6 separates this number from the latencies the system actually experiences.

A.3 The yardsticks

Six things can give a hardware number meaning:

1. **The sensor.** This is how fast data arrives and how much time passes between ramps and between frames. It sets the throughput requirement.

2. **The system budget.** This is how long the rest of the processing chain can afford to wait for compressed data. Kiem derived one explicitly for his use case.
3. **The link.** This is how many bits per second the output connection can carry. It sets the compression-ratio requirement.
4. **The device.** This is how much of the chip the design uses, and whether it would fit on a smaller one, in particular on Kiem's.
5. **Kiem's implementation.** This is the only published hardware DRHE, but it is a valid yardstick only where the two numbers measure the same thing.
6. **Internal baselines.** These are the design's own theoretical ideal, the no-prediction coder, and the entropy bound.

Table A.3 gives the sensor and system figures. They are used throughout the rest of the appendix.

Table A.3: Sensor and system figures used as yardsticks. ColoRadar values follow from Table 2.1 and [12]. Kiem's values are from his Section 3.3 [11]. "Post-FFT" means the retained half-spectrum as 16-bit complex values, which is what the compressor receives.

Quantity	ColoRadar cascade	Kiem's sensor
Transmitters (TDM)	12	1
Range bins per ramp (retained)	128	512
Ramps per frame	192	1024
Receive channels	16 (4 per compressor)	4
Ramp interval	78.5 µs	≈20.5 µs
Frame acquisition time	15.07 ms	≈21 ms
Frame period	200 ms (5 Hz)	50 ms (20 Hz)
Compression time budget	—	15 ms
Post-FFT data per frame, 4 RX	3.15 Mbit	67.1 Mbit
Post-FFT rate, 4 RX, during a frame	0.209 Gbit/s	3.20 Gbit/s
Post-FFT rate, all RX, during a frame	0.835 Gbit/s	3.20 Gbit/s
Clock cycles per ramp at 100 MHz	7,850	≈2,050

One cross-check confirms the sensor model. Sixteen receive channels, 192 ramps and 256 complex 16-bit ADC samples per ramp make 25.2 Mbit of raw data per frame, which at 5 Hz is 125.8 Mbit/s. The ColoRadar authors quote 126 Mbps for this sensor [12].

Two device yardsticks are used. The `XCKU5P` of the KCU116 provides 216,960 LUTs, 433,920 flip-flops, 480 block RAMs of 36 Kb and 1,824 DSP slices. Kiem's Zynq UltraScale+ `XCZU3CG` provides 70,560 LUTs, 141,120 flip-flops, 216 block RAMs of 36 Kb and 360 DSP slices [11]. Both are UltraScale+ devices with the same LUT, flip-flop, block-RAM and DSP

primitives, so a design's resource count transfers between them; what changes is the fraction of the chip it occupies.

A.4 Rebuilding the reference design

A.4.1 Why a replica is needed

A comparison of two numbers explains something only if the two numbers differ in one known way. This thesis's DRHE and Kiem's differ in at least seven ways at once: the prediction axis (per-transmitter state for a 12-transmitter TDM sensor against a single transmitter), the arithmetic (floating point against fixed point), the bit packer, how the predictor state is reset at the start of a frame, the feedback structure (closed against open loop), the tool version (Vitis HLS 2026.1 against 2022.2) and the device (an `xcku5p` at speed grade -2 against an `XCZU3CG-SFVC784-1-e` at speed grade -1). Set side by side, the two resource tables cannot say how much of any difference comes from which cause.

The way to separate them is to hold everything constant except one thing at a time, and that requires Kiem's design inside this thesis's tool flow. His source code is not published, so it was rebuilt from the description in his Section 4.3.4 [11], which is detailed enough to do so:

- **Data types.** Magnitude `ap_ufixed<16,16>`, phase `ap_fixed<16,3>`, input `ap_fixed<16,16>`, intermediate phase arithmetic `ap_fixed<32,4>`, all with rounding.
- **Functions.** `hls::sqrt` on `ap_uint<32>`; `hls::atan2` on `ap_fixed<16,16>`; `hls::sin` and `hls::cos` on `ap_fixed<16,3>`.
- **State.** Five arrays of 4×1024 entries, partitioned by channel, driven from the *input* sample (open loop).
- **Packing.** The real and imaginary codes of one channel packed into one `ap_uint<58>`; the four channel words then combined into the 256-bit output buffer.
- **Schedule.** All four channels in parallel, $\mathrm{II} = 1$.

Where his text leaves a choice open, the replica takes the simplest reading. One such choice needs stating. Kiem does not describe how the predictor state is cleared at the start of a frame, but he reports $\mathrm{II} = 1$, and Section A.5.4 shows that $\mathrm{II} = 1$ is only possible if the clearing does not add a third memory access per cycle. The replica therefore clears the state by reading it as zero on the first ramp, which is equivalent.

The prediction axis is the one difference the replica does not need to model. Kiem's sensor has one transmitter, so his lag-1 predictor is the per-transmitter predictor of Section 4.3 for that sensor, and the DRHE-1 variant of this thesis's datapath implements the same predictor. The comparison is therefore made from DRHE-1, and the cost of the per-transmitter state is measured separately, by comparing DRHE-1 with the final lag-n_{Tx} compressor (Section A.5.1): it adds block RAM and nothing else.

A.4.2 One source, four switches

The replica and DRHE-1 are built from one source file with four compile-time switches (`drhe_abl_compress.cpp`).

Switch	0: this thesis's datapath (DRHE-1)	set: Kiem's design
`ABL_PACK`	eight inserts into a 512-bit buffer with a signed offset	1: the same with an unsigned offset; 2: 58-bit word per channel, one 232-bit insert
`ABL_RESET`	state zeroed by a write on ramp 0	state read as zero on ramp 0
`ABL_ARITH`	single-precision floating point	Kiem's fixed-point types and functions
`ABL_LOOP`	closed loop: state from the reconstruction	open loop: state from the input

With every switch at zero the source is, statement for statement, DRHE-1: it synthesises to the same schedule (II = 2, depth 58, 20 blocks of 36 Kb, 132 DSPs) and writes exactly as many bits as DRHE-1 on every one of the 50 frames. With every switch set it is the replica. All the packing modes write the same bits, so one decoder reads them all, and the decoder follows the arithmetic switch. Every variant decodes losslessly on real ColoRadar frames in C-simulation, so each one is a working compressor, not a sketch.

A.4.3 How close is the replica to Kiem's published design?

Table A.4 sets the implemented replica beside Kiem's published figures.

The replica agrees with Kiem's design where the design is fully specified. Block RAM matches exactly: 10 blocks against his 10, because both store twenty arrays of 1,024 16-bit words (Section A.5.1). The DSP count is 48 against 44. Both designs achieve II = 1, and both meet 100 MHz on a speed grade -1 device.

The replica does *not* reproduce Kiem's LUT and flip-flop counts. It needs 17,141 LUTs and 5,050 flip-flops where Kiem reports 45,892 and 9,243, and his design, as he describes it, does

Table A.4: The rebuilt design against Kiem's published compression IP (his Table 5.3 and Section 5.2.3 [11]). Replica figures are post-route, out of context, on the `xcku5p` at speed grades −2 and −1; Kiem's are post-route, in system, on the `XCZU3CG-SFVC784-1-e`. BRAM in 36 Kb blocks.

	Replica (−2)	Replica (−1)	Kiem [11]
LUT	17,141	17,228	45,892
FF	5,050	5,598	9,243
DSP	48	48	44
BRAM (36 Kb)	10	10	10
Initiation interval	1	1	1
Pipeline depth (HLS, cycles)	12	14	23
Achieved clock (MHz)	129.5	125.0	100 (met)
Compression ratio (ColoRadar, 10 frames, lag 1)	3.31904		—

not need the difference. Section A.5.5 tests the explanations his text leaves open — the speed grade, the packer he describes in one sentence, and his deeper pipeline — by building each into the replica. Together they account for about half of the flip-flop difference and almost none of the LUT difference; what remains most likely lies in the tool version (Vitis HLS 2022.2, whose fixed-point math library is four years older than the one used here) or in logic his thesis does not describe.

This matters for how his LUT figure is used. It measures one particular build; it is not a measure of what the DRHE algorithm costs. When the question is "what do this thesis's design decisions cost?", the fair yardstick is the replica, built in the same tool, for the same device, from the same description, and the next section uses it that way. When the question is "how does this thesis's compressor compare with the IP Kiem built?", his published figure is the right one, and Section A.8 uses it that way.

A.5 From this thesis's datapath to Kiem's, one change at a time

Table A.5 is the central result of this appendix. Each row is a complete, lossless compressor, implemented through placement and routing on the same device with the same constraints. The first row is DRHE-1. Each following row changes exactly one design decision, and the last is the replica of Kiem's design. The difference between two adjacent rows is therefore the cost, in every resource at once, of that one decision. The final lag-n_{Tx} compressor is listed beneath

the ladder for reference: it differs from step 0 only in its state memory.

Table A.5: From DRHE-1 (this thesis's DRHE datapath at lag 1) to the replica of Kiem's design, one design decision per step. Every row is a lossless compressor implemented through placement and routing, out of context, on the `xcku5p-ffvb676-2-e` at 10 ns. II and depth are from HLS; everything else is post-route. BRAM in 36 Kb blocks.

Step	Change from the row above	LUT	FF	DSP	BRAM (36 Kb)	II	Depth	MHz
0	DRHE-1 (float, closed loop, lag 1)	35,343	23,546	132	20	2	58	135.5
1	packer shift offset made unsigned	33,502	24,423	132	20	2	58	128.4
2	Kiem's 58-bit per-channel packer	31,124	21,261	127	20	2	62	120.6
3	state cleared by read, not write	52,432	30,373	264	20	1	59	131.4
4	Kiem's fixed-point arithmetic	16,882	5,380	48	10	1	16	133.3
5	open loop = replica of Kiem	17,141	5,050	48	10	1	12	129.5
—	Kiem, published [11]	45,892	9,243	44	10	1	23	100
—	DRHE, lag n_{Tx} (final design)	34,804	23,514	132	40	2	58	131.0

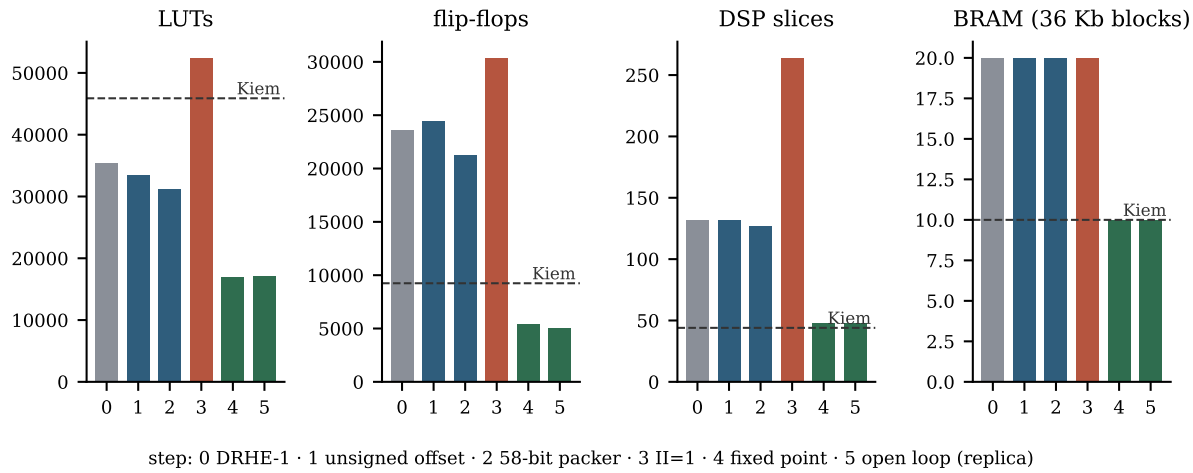


Figure A.2: LUTs, flip-flops, DSPs and block RAM (36 Kb blocks) at each step of Table A.5, all post-route on the `xcku5p`. The dashed line in each panel is Kiem's published figure. Every step but one reduces the design or changes it only slightly; the exception raises it from II = 2 to II = 1, which doubles the arithmetic because the floating-point units can no longer be shared between channels.

The following subsections take each metric in turn and account for its difference from Kiem using the ladder, the per-module reports and, where relevant, the HLS schedule.

A.5.1 Block RAM: the width of a word

The difference. DRHE-1 uses 20 block RAMs of 36 Kb; the final DRHE compressor uses 40; Kiem's design uses 10.

The accounting. Block RAM holds DRHE's predictor state: five values per range bin and channel from previous ramps (the magnitude prediction, the previous magnitude, the phase prediction and the two previous phases). Partitioned by channel, as both designs are, that is $5 \times 4 = 20$ separate arrays. Table A.6 lists how each design stores them.

Table A.6: How the DRHE predictor state is stored, from the HLS memory tables and the routed utilisation reports. BRAM in 36 Kb blocks; a block holds 1,024 words of up to 36 bits, or can be split into two halves of 1,024 words of up to 18 bits.

Design	Arrays	Words per array	Bits	BRAM per array (36 Kb blocks)	BRAM total (36 Kb blocks)
DRHE-1 (float, lag 1)	20	1,024	32	1	20
Replica of Kiem (fixed point)	20	1,024	16	$\frac{1}{2}$	10
DRHE, lag n_{Tx} (float, final)	20	1,536	32	2	40

Every DRHE-1 state word is a 32-bit `float`. A 1,024-word array of 32-bit words is 32 kbit, too much for half a block, so each array takes a full 36 Kb block: twenty arrays, twenty blocks. Kiem's words are 16-bit fixed point (`ap_ufixed<16,16>` and `ap_fixed<16,3>`); a 1024×16 array fits in half a block, so twenty arrays need ten blocks. The replica, built with his types, uses exactly ten. **The factor of two between DRHE-1 and Kiem is the word width, and nothing else.**

The array depths need a closer look, because they are set differently in the two designs. DRHE-1 and the replica size each array at 1,024 words, Kiem's provisioning for up to 1,024 range bins with one word per bin; at ColoRadar's 128 bins only an eighth of each array is used. The final DRHE compressor keeps one word per transmitter and range bin: $n_{\mathrm{Tx}} \times 128 = 12 \times 128 = 1{,}536$ words per array. *Per range bin*, the per-transmitter state is therefore twelve times the words of a lag-1 predictor; it appears here as only one and a half times the depth because the lag-1 arrays are provisioned for eight times more bins than ColoRadar has. HLS rounds 1,536 up to the next power of two and gives each array two 36 Kb blocks, so the design uses 40 blocks, the figure measured in the board system (Table 7.8). That is four times Kiem's: a factor of two for the word width, and a factor of two for the depth. The memory is 67 % full (983,040 bits stored in 1,474,560 bits of block RAM); the rounding of the depth to a power of two and the unused parity bits are the only waste. Nothing else in the design changes with the lag: against DRHE-1, the final compressor uses 539 fewer LUTs (1.5 %), 32 fewer flip-flops and the same 132 DSPs, and it has the same schedule (II = 2, depth 58). **At ColoRadar's 128 range bins, the per-transmitter prediction that makes DRHE work on a TDM-MIMO**

sensor costs 20 blocks of 36 Kb and nothing else — but only because the lag-1 arrays it is compared with are provisioned at 1,024 words. The state grows with bins times transmitters. On a 12-transmitter sensor with Kiem's 512 range bins, each array would hold $12 \times 512 = 6{,}144$ words: in 32-bit floating point that is 8 blocks per array (rounded up to 8,192 words), 160 blocks for the four-channel compressor; in Kiem's 16-bit fixed point, with the same rounding, it is 4 blocks per array, 80 in total. Both fit the `XCKU5P`; on Kiem's `XCZU3CG` (216 blocks) the floating-point version would take three quarters of the device's block RAM. In fixed point, at ColoRadar's frame size, the per-transmitter state fits in 20 blocks.

The LPC compressor stores something different: the whole frame. Its 98 blocks in the board system are dominated by the frame buffer, whose 3.0 Mibit of 32-bit words need at least 96 blocks on their own. The LPC decompressor, which needs no frame buffer (Section 6.3), uses none, and the DRHE decompressor's 32.5 blocks hold mainly its own copy of the predictor state.

A.5.2 DSP slices: counted operator by operator

The difference. DRHE-1 and the final DRHE compressor both use 132 DSP slices; Kiem's design uses 44.

The accounting. Every DSP in DRHE belongs to an identifiable floating-point operator, and the routed hierarchy report counts them exactly (Table A.7). The lag does not change the arithmetic, so the count is the same for both lags.

Table A.7: Where the DRHE compressor's DSP slices go, from the routed hierarchical utilisation report. The counts are identical for DRHE-1 and the final lag-n_{Tx} compressor.

Operator	Instances	DSPs each	DSPs
float multiply (`fmul`, max-DSP)	16	3	48
float add/subtract (`fadd`, full-DSP)	12	2	24
float sine/cosine	4	13	52
float arctangent	2	4	8
Total			**132**

A single-precision multiply needs three DSP slices because its 24-bit significand product does not fit one 27×18 multiplier; an adder uses two, for the alignment shift and the add; the sine/cosine unit uses thirteen for its polynomial and range reduction. In fixed point, a product

of two 16-bit numbers is one slice and an addition needs none. The replica uses 48 DSPs against Kiem's 44.

Four channels at two operations each would need eight sine/cosine units and four arctangents. DRHE has four and two, because at II = 2 each unit is idle every other cycle and HLS shares it between two channels. Step 3 of the ladder removes that sharing: at II = 1 the same float design needs 264 DSPs. **The factor of three is floating-point arithmetic; without the sharing that** II = 2 **allows, it would be six.**

A.5.3 Pipeline depth: which operations wait for which

The difference. DRHE's pipeline is 58 cycles deep, at either lag; Kiem reports 23 (his HLS estimate, the same kind of number).

The accounting. The HLS schedule report lists, for every operation, the pipeline stage it runs in. Grouping the operations of one sample by what they do gives Figure A.3(a). The pipeline is two chains laid end to end. The first is the prediction: load the state, run the two IIR filters, wrap the phase, take the sine and cosine, rescale to Cartesian and round, which ends around stage 31. The second is the polar conversion that updates the state: convert the sample to floating point, take $re^2 + im^2$ and its square root, and take the arctangent, which alone occupies 28 stages. In DRHE the second chain *cannot start until the first has finished*, because it converts the reconstructed sample, and the reconstruction needs the prediction. That is the closed loop of Sections 2.4.4 and 6.4, which guarantees that the encoder's state is always the decoder's state.

Switching the loop open, so the state is driven from the input sample, lets the polar conversion start in stage 3 alongside the prediction. The float design's depth falls from 58 to 32 (Figure A.3(b); the open-loop depth is measured from its own schedule report, not drawn). Fixed-point arithmetic then shortens both chains. The replica's fixed-point `atan2` and `sqrt` take a few cycles each rather than 28 and 11; the closed-loop fixed-point design (step 4) is 16 cycles deep and the open-loop replica (step 5) 12. The replica is shallower than Kiem's reported 23 cycles; his figure comes from an older tool version and from code that split its arithmetic into extra stages to meet timing (his Section 4.4.1), so an exact match is not expected.

Of DRHE's 58 cycles, 26 are the closed loop; the open-loop float design is still 20 cycles deeper than the replica, and that is the latency of floating-point operators. The closed loop

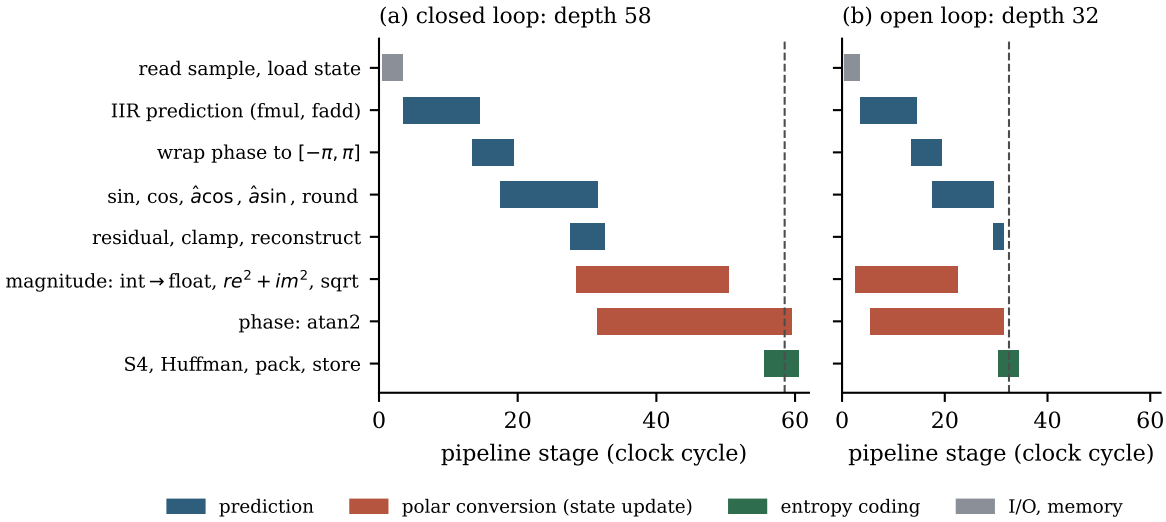


Figure A.3: The pipeline schedule of one sample, read from the HLS schedule reports. (a) The DRHE datapath of this thesis: the state is updated from the reconstructed sample, so the polar conversion (red) waits for the prediction (blue) and the depth is 58. (b) The same float design with the state updated from the input sample (open loop): the two chains run side by side and the depth is 32. Kiem's design is open loop and fixed point, and shorter still.

Table A.8: Pipeline depth (HLS schedule, cycles) of the DRHE datapath and its variants, and what sets it.

Design	Depth	What sets it
DRHE (float, closed loop), either lag	58	prediction, then polar conversion, in series
float, open loop	32	the longer of the two chains (atan2)
fixed point, closed loop	16	fixed-point chains in series
replica (fixed point, open loop)	12	fixed-point chains in parallel
Kiem (HLS estimate)	23	—

is the price of guaranteeing that the encoder and the decoder can never disagree, even on the saturated sample of Section 6.4. At 260 ns it is not a price this application notices (Section A.6), and in fixed point it is four cycles.

A.5.4 Initiation interval: one write too many

The difference. DRHE accepts a new 128-bit beat every two cycles (II = 2); Kiem's design accepts one every cycle.

The cause, as HLS reports it. The synthesis log names it:

```
Unable to schedule 'load' operation 32 bit ('pp2') on array
...s_prev_prev_phase_3 due to limited memory ports (II = 1).
```

Each state array is one block RAM, and a block RAM has two ports. In every cycle each array must be read once (the old value) and written once (the new one). DRHE also clears the state at the start of a frame with an *extra* write (in DRHE-1, `if (r == 0) s_prev_prev_phase[ch][s] = 0;`; in the final design the same write on each transmitter's first ramp, $m < n_{\mathrm{Tx}}$). That write happens on only a few ramps of 192, but HLS must schedule for the worst case, so every cycle is scheduled as if three accesses were needed, and three accesses on two ports take two cycles.

The demonstration. Step 3 of the ladder replaces the write with a read that is ignored on the first ramp (`pp2 = first ? 0 : s_prev_prev_phase[ch][s]`), which clears the state just as well with one read and one write per cycle. The same float design then schedules at II = 1. Steps 1 and 2, which change only the packer, stay at II = 2 with the identical message.

What it costs to change. II = 1 doubles the theoretical throughput. In RTL co-simulation, where the step-3 design also reconstructs its frame losslessly, the measured rate rises from 5.14 to 8.76 Gbit/s (35,909 cycles per frame against 61,253); it does not quite double because the 59-cycle pipeline still drains once per ramp. Step 3 of Table A.5 also shows the bill: the float arithmetic can no longer be shared between channels, so DSPs and logic both rise. This is the one step in the ladder that makes the design substantially larger in every resource, and it is why the final design keeps II = 2 (Section 6.2): with about 24 times more throughput than four ColoRadar channels need (Section A.7), the halved rate buys a design half the size in

arithmetic. In fixed point the balance would be different, because the arithmetic is cheap and the read-based reset costs nothing.

A.5.5 LUTs and flip-flops: two gaps, not one

The difference. DRHE-1 uses 35,343 LUTs and 23,546 flip-flops; the final DRHE compressor, in the board system, 35,444 and 23,528. Kiem reports 45,892 and 9,243. So against his published figures the final in-system compressor uses *fewer* LUTs (0.77×) but *more* flip-flops (2.55×), and DRHE-1 out of context almost exactly the same ratios (0.77× and 2.55×), which looks contradictory: a design with more registers would be expected to have more logic too.

Why one comparison cannot explain it. The two ratios mix two separate gaps. The first is between DRHE-1 and the replica: both built in the same tool for the same device, differing only in the design decisions the ladder changes. The second is between the replica and Kiem's published build: the same design, as far as his description goes, built in a different tool version, in a different system, from code that is not available. Measured against the replica, DRHE-1 uses 2.06× the LUTs and 4.66× the flip-flops, both larger, and there is no contradiction. The apparent contradiction comes from the second gap, which is large for LUTs (Kiem's build uses 2.68× the replica's) and smaller for flip-flops (1.83×). This subsection accounts for the first gap completely and for the second as far as experiments can.

The first gap, module by module. The routed hierarchy report lists every module in DRHE-1 with its own LUTs and flip-flops (Table A.9). The floating-point operators, which HLS instantiates as separate modules, are 69 % of the LUTs and 54 % of the flip-flops. The two arctangent units alone are 11,961 LUTs, a third of the design, because a single-precision `atan2` is a 43-bit CORDIC with range reduction around it. The rest is the loop body — the packer, the S_4 and Huffman encoding, the residual arithmetic, and the registers that carry each value down a 58-stage pipeline until it is used — plus the AXI interfaces. The final lag-n_{Tx} compressor has the same modules with almost the same sizes (floating-point operators 70 % of its LUTs, arctangents 11,962).

The first gap, decision by decision. The ladder of Table A.5 turns the module view into costs. Each step's change is the cost of one decision, and the five changes add up exactly to the difference between DRHE-1 and the replica.

Table A.9: Where DRHE-1's LUTs and flip-flops go, from the routed hierarchical utilisation report.

Module	Instances	LUT	FF
float arctangent (`atan2`)	2	11,961	5,334
float sine/cosine	4	5,022	1,296
float add/subtract	12	3,126	2,778
float multiply	16	1,678	2,054
float square root, int-to-float, rounding	10	2,672	1,354
all floating-point operators		24,459	12,816
loop body (packer, coder, residual, pipeline registers)	1	10,258	8,278
AXI-Stream and AXI-Lite interfaces	3	549	927

Step 1, the shift offset (1,841 **fewer LUTs, 5 %).** DRHE's packer shifts each code into the packing register by `bit_count`, a signed `int`. A shift by a signed amount has to handle negative amounts too, so every one of the eight inserts carries logic for a right shift that never happens, and Vivado keeps it, since nothing in the netlist shows that the counter is never negative. Making the counter an unsigned 10-bit number removes that logic. Flip-flops move by 877 (more), a small change; run-to-run variation of the implementation was not measured, so changes of this size are not interpreted.

Step 2, the packer (2,378 **fewer LUTs,** 3,162 **fewer flip-flops).** Kiem's packer joins each channel's real and imaginary codes into one 58-bit word, joins the four words into one 232-bit word, and inserts that once. Eight variable-offset inserts into a 512-bit register become three small shifts inside a 232-bit word and one insert. The loop body, which holds the packer, falls from 8,723 to 6,482 LUTs and from 9,355 to 6,622 flip-flops. Together, steps 1 and 2 show that DRHE's packer costs about 4,200 LUTs more than it needs to, 12 % of the design.

Step 3, the initiation interval (21,308 **more LUTs,** 9,112 **more flip-flops).** Removing the extra state write lets the loop run at II = 1 (Section A.5.4). At II = 2 each floating-point unit was shared by two channels; at II = 1 each channel needs its own. The floating-point operators grow from 23,487 to 45,407 LUTs, which is the whole of the step's increase, and the DSP count doubles, to 264. It is the only step that makes the design substantially larger — step 1 adds 877 flip-flops and step 5 adds 259 LUTs, both small — and it is the price of doubling the throughput with floating-point arithmetic.

Step 4, fixed-point arithmetic (35,550 **fewer LUTs, 68 %;** 24,993 **fewer flip-flops, 82 %).** Kiem's types replace every floating-point operator with fixed-point logic: 16-bit multiplies in single DSP slices, additions in carry chains, and fixed-point `sqrt`, `atan2`, `sin` and `cos` built by HLS directly into the loop body. The flip-flops fall further than the LUTs because both of their causes shrink at once: the values carried down the pipeline are 16 bits wide instead of 32, and the pipeline is 16 stages deep instead of 59, so there are fewer, narrower values to hold for fewer cycles.

Step 5, the open loop (259 **more LUTs,** 330 **fewer flip-flops).** Driving the state from the input sample instead of the reconstruction shortens the pipeline by four stages and removes 330 flip-flops; the LUT change is small (2 %). In fixed point, the closed loop that makes DRHE immune to encoder–decoder divergence is almost free.

The order of the steps matters for their sizes, though not for their sum. The fixed-point step is measured at $\mathrm{II} = 1$, where the float design has twice as many operators as DRHE-1 does, so it saves more than the same change would save at $\mathrm{II} = 2$. To measure that directly, one more variant changes *only* the arithmetic of DRHE-1, keeping its packer, its state write and so its $\mathrm{II} = 2$, and its closed loop. It uses 20,549 LUTs (42 % fewer than DRHE-1), 9,230 flip-flops (61 % fewer), 44 DSPs — exactly Kiem's count — and 10 blocks of 36 Kb, with a pipeline 12 cycles deep. It reconstructs ColoRadar losslessly, compresses it to within 0.07 % of the floating-point design, and meets 100 MHz, if only just (102.8 MHz): with the loop closed and only 12 stages, the fixed-point arithmetic feeding the state write is the critical path (Section A.5.6). This is the saving available to this thesis's datapath with every other decision kept. It still carries the more expensive packer, which in the float design costs about 4,200 LUTs more than Kiem's (steps 1 and 2); its cost in fixed point was not measured separately. Applied to the final lag-n_{Tx} compressor, whose operators are the same, the same change would be expected to save a similar share; its 16-bit state would also halve the block RAM to 20 blocks.

So why does DRHE-1 have more flip-flops than Kiem, but fewer LUTs? Because the two gaps go in opposite directions for LUTs and the same direction for flip-flops. DRHE-1 needs 4.66× the replica's flip-flops, for the reasons of steps 2 to 5: a 58-stage pipeline of 32-bit values, the registers inside each floating-point operator, and the packer. Kiem's build needs only 1.83× the replica's, so DRHE-1 ends up with more than he has. DRHE-1 needs 2.06× the replica's LUTs, but Kiem's build needs 2.68×, so DRHE-1 ends up with fewer. Both ratios

against Kiem are therefore the product of a gap that can be fully accounted for and one that can only partly be explained, which is the subject of the rest of this subsection.

The second gap: the replica against Kiem's build. Section A.4.3 left the replica 28,751 LUTs and 4,193 flip-flops short of Kiem's published figures. His thesis does not give enough detail to reconstruct the difference, but three explanations can be tested, and each was: the replica was changed in one respect at a time and implemented again (Table A.10).

Table A.10: Testing explanations for the gap between the replica and Kiem's published LUT and flip-flop counts. Each row is the replica with one thing changed, implemented and routed on the `xcku5p`. "Share" is the change from the replica as a fraction of the gap to Kiem (28,751 LUTs, 4,193 flip-flops).

Variant	Tests	LUT	FF	LUT share	FF share
Replica, speed grade −2	the reference point	17,141	5,050	—	—
Replica, speed grade −1	Kiem's speed grade	17,228	5,598	0 %	13 %
Replica, literal packer	four inserts, `int` offset	18,576	6,214	5 %	28 %
Replica, 23-cycle pipeline	Kiem's reported depth	17,060	5,217	0 %	4 %
Kiem, published [11]	in system, 2022.2	45,892	9,243	100 %	100 %

Speed grade. Kiem's device is speed grade −1, the KCU116's −2. HLS schedules for the part it is given: for the slower part it inserts two more pipeline stages (14 against 12), which adds 548 flip-flops, 13 % of the flip-flop gap, and almost no LUTs. This is, measured, the effect Kiem describes in his Section 4.4.1, where he split arithmetic into extra registered steps to meet timing.

The packer. Kiem describes his packer in one sentence: each channel's 58-bit word is "combined" into the 256-bit buffer. The replica joins the four words first and inserts once. The most literal alternative reading inserts each channel's word in turn, with a signed `int` bit counter, as DRHE does. That costs 1,435 more LUTs and 1,164 more flip-flops: 5 % of the LUT gap and 28 % of the flip-flop gap. So a less careful packer would account for about a quarter of his extra flip-flops, but no plausible packer accounts for his LUTs. DRHE's own packer, the most expensive one measured in this work, costs about 4,200 LUTs more than Kiem's (steps 1 and 2), still a small fraction of 28,751.

Pipeline depth. Kiem reports 23 cycles where the replica schedules 12. Forcing the replica's loop to 23 stages with an HLS latency constraint adds only 167 flip-flops, because HLS places the extra stages where few values are live. Depth by itself therefore does not explain his flip-flop count; where his extra stages sit, inside the 32-bit intermediate arithmetic his Section 4.3.4 describes, is what would matter, and that cannot be reproduced without his code.

What is left. If their effects add, the three tested explanations account for about 45 % of the flip-flop gap but only about 5 % of the LUT gap. Two explanations could not be tested:

- **The tool version.** Kiem used Vitis HLS 2022.2. The replica's logic is dominated by the fixed-point `sqrt`, `atan2`, `sin` and `cos` that HLS builds from its math library into the loop body (the loop body, which also holds the packer, coder and residual logic, is 12,691 of the replica's 17,141 LUTs), and that library's fixed-point implementations are the part of the flow most likely to have changed in four years. Only 2026.1 was available, so this is a hypothesis rather than a finding.
- **Code or debug logic his thesis does not describe.** His Section 4.2.1 and Figure 5.4 show an integrated logic analyser probing the compressor's internal signals in the implemented system. Signals marked for debug are kept through optimisation, so a build with probes can be larger than one without; his thesis does not say whether the build he reports had them, nor describe his control logic in detail.

In-system implementation, the other difference, is unlikely to matter: Vivado synthesises each IP in a block design out of context by default, exactly as here, and placement and routing change LUT counts only slightly — by under 2 % for the DRHE compressor between its out-of-context run and the board system (Section A.1).

What this means for the comparison. The difference between this thesis's DRHE and Kiem's published LUT count is *not* a property of either algorithm or of any design decision described in either thesis: the same design, rebuilt, needs 17,141 LUTs. The accurate statement is therefore the one of Section A.4.3: Kiem's published figure measures his build; the replica measures his design; and against his design, DRHE's extra cost is fully accounted for by its distinctive decisions: floating-point arithmetic, its packer, the extra state write, the closed loop and, for the final design, the per-transmitter state.

A.5.6 Clock frequency and speed grade

What it is. $F_{\max}$ is the fastest clock at which every signal still settles within one period. Vivado's timing analysis after routing gives the worst negative slack (WNS): how much time the slowest path has to spare at the 10 ns constraint. The slowest path can then be run at a period of $10 - \text{WNS}$ nanoseconds.

Why measure it. Throughput and latency in seconds both depend on the clock, and the compressor has to share a clock with the FFT, DMA and processor around it. Kiem ran all his IP on one 100 MHz clock; the KCU116 system does the same.

The numbers. Every design meets 100 MHz (Table A.11).

Table A.11: Worst negative slack at 10 ns and the resulting $F_{\max}$ after routing. Core rows are out of context; system rows are the board builds of Table 6.1.

Design	WNS (ns)	$F_{\max}$ (MHz)	Worst path
DRHE, lag n_{Tx} (core)	2.367	131.0	from a float sine/cosine unit into the loop
LPC, lags $n_{\text{Tx}}, 2n_{\text{Tx}}$ (core)	0.595	106.3	frame-buffer read into the solver's 64-bit multiply
DRHE system, compress only	0.243	102.5	—
LPC system, compress only	0.174	101.8	—
DRHE-1	2.621	135.5	inside a float sine/cosine unit
DRHE-1, speed grade -1	2.769	138.3	inside a float sine/cosine unit
Fixed-point DRHE-1 (II = 2)	0.272	102.8	26-stage carry chain into the state write
Replica, speed grade -2	2.276	129.5	20-stage carry chain, fixed-point arithmetic
Replica, speed grade -1	1.999	125.0	18-stage carry chain, fixed-point arithmetic

Against Kiem: valid, with one adjustment. Kiem's design meets 100 MHz on a speed grade -1 part, which is slower silicon than the -2 part used here. The replica closes at 125.0 MHz on a -1 part and 129.5 MHz on a -2, so the speed grade is worth about 3.5 % of clock for the same source (HLS also schedules the slower part two stages deeper, so it is not quite the same netlist). Implemented on a -1 part, DRHE-1 closes at 138.3 MHz — slightly faster than on the -2 part, because HLS schedules it 63 stages deep instead of 58 for the slower silicon — so

this thesis's floating-point datapath meets Kiem's clock on Kiem's grade of silicon with room to spare.

One design with little margin. The fixed-point version of DRHE-1 (Section A.5.5) closes at only 102.8 MHz. With the loop closed and the pipeline only 12 stages deep, the path from the fixed-point arithmetic to the state write has 38 levels of logic, 26 of them carry chains. That is the cost of a short closed-loop pipeline. Adopting fixed point in the final design would mean either accepting that margin or asking HLS for one or two more stages, which the replica's schedule shows are cheap.

HLS's timing estimate points the wrong way. HLS estimates 81.9 MHz for DRHE and 85.4 MHz for LPC and places the critical path somewhere other than where the routed design has it. Every design that the estimate places below 100 MHz passes after routing; the DRHE core on its own, out of context, has a quarter of the period to spare. Pipelining the float datapath further, or replacing it with a hand-written CORDIC "to meet timing", would address a problem that the routed design does not have.

When does it become a problem? If a design cannot share the system clock, it needs its own clock and a clock-domain-crossing FIFO on each side, which is extra design work and extra memory. None of these designs needs one. Of the two final cores, LPC has the least margin, 0.595 ns out of context; and the whole board systems, at 0.17 ns to 0.34 ns, have much less than either core alone. A slower speed grade or a more congested system could consume that margin, and it is the system, not the compressor, that would then have to be re-timed.

A.6 Latency: three different things with the same name

"Latency" means *how long something waits*. The question is always *what* waits, and for *what*. Three quantities in this thesis are called latency, and they answer different questions: how long one sample takes to pass through the pipeline, how long a whole frame takes to process, and by how much compression delays the moment a frame is available downstream. Confusing them is the easiest way to draw a wrong conclusion, including when comparing with Kiem. Figure A.4 puts all of them on one time axis, together with the time-scales of the two sensors.

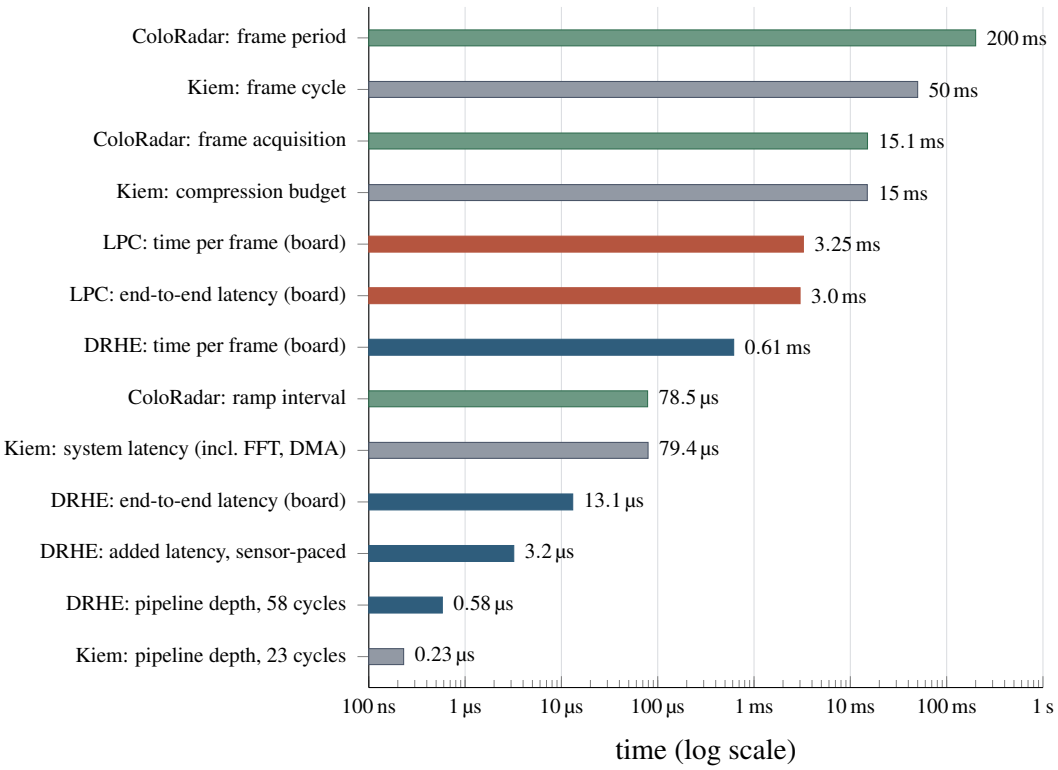


Figure A.4: Every latency in this appendix on one logarithmic time axis: DRHE (blue), LPC (red), Kiem's design and use case (grey) and the ColoRadar sensor (green). Board figures are measured on the KCU116 at 100 MHz; DRHE's sensor-paced added latency is the work of one ramp. The compressor's own delays are two to five orders of magnitude shorter than the sensor's time-scales; the only ones that approach them are LPC's, because LPC must see the whole frame before it can emit anything.

A.6.1 Pipeline latency: how long one sample takes to pass through

What it is. Pipeline latency is the number of clock cycles between a sample entering the compressor and its compressed bits leaving the datapath. For DRHE it is the pipeline depth, 58 cycles, or 580 ns at 100 MHz. Many samples are in flight at once, so this is a delay, not a rate.

Why measure it. It fixes two things: how much data is "inside" the compressor at any moment, and so how deep any buffering around it must be; and the smallest possible delay the compressor can add to anything.

Against Kiem: valid. Kiem reports 230 ns, which is 23 cycles. This is his HLS estimate, the same kind of number as DRHE's 58. He also measured 31 cycles with an integrated logic analyser, the extra 8 cycles being the time to fill the first 256-bit output word. Estimate against estimate, DRHE is 35 cycles (350 ns) deeper, and Section A.5.3 accounted for them: 26 cycles are the closed loop, and the rest is the latency of floating-point operators.

When does it become a problem? In this application it does not. 580 ns is 0.7 % of one ColoRadar ramp interval and under 3 % of one of Kiem's. It would matter only in a control loop that must react within about a microsecond, and no radar perception chain works on that time-scale. The difference from Kiem is real, but it has no practical consequence here.

A.6.2 Frame time: how long a whole frame takes, if it all arrives at once

What it is. Frame time is the time from the start of a frame's transfer to the last compressed word being written, with the frame fed as fast as the core accepts it (Section 5.3). On the board it is 61,472 cycles (614.7 µs) for DRHE and 325,366 cycles (3,253.7 µs) for LPC. It is really a *throughput* measurement expressed in time: the time to process one frame's worth of work at full speed, and its reciprocal is the frame rate the design can sustain, 1,627 and 307 frames per second.

Where the cycles go. DRHE's frame time is set by its initiation interval. A frame is 24,576 beats of 128 bits; at $\mathrm{II} = 2$ that is 49,152 cycles, or 256 per ramp. The board measures 320 cycles per ramp: the extra 64 or so are the pipeline draining once at the end of each ramp's sample loop, and the DMA start and final write add about 200 cycles per frame (Section A.2).

LPC's frame time is its three passes (Section 6.3): 24,576 cycles to store the frame, about 276,000 to correlate, solve and form residuals for its 512 (channel, bin) pairs, and 24,576 to emit.

What it is not. It is not a delay the system ever experiences. The real sensor delivers a frame over 15.07 ms. DRHE needs 0.61 ms of work to process it, so the compressor is busy about 4 % of the time during a frame and idle the rest, waiting for the next ramp.

Against Kiem: no equivalent number. Kiem does not report a per-frame processing time. His 0.08 ms is a different quantity, discussed next.

A.6.3 End-to-end and added latency: how long the answer is delayed

This is the latency that matters to the system, and the right answer to the question "what does the latency represent?"

What it is. Without compression, the downstream processor could start work on a frame the moment its last ramp had been through the range FFT. With compression, it must wait until the compressor has emitted the last bit of that frame. The difference is the *added latency*. The board measures it with Kiem's definition (Section 5.3): the time from the DMA reporting the last input word read from memory to the DMA reporting the last compressed word written back.

DRHE: 13.1 µs on the board. DRHE is a streaming design: it processes each ramp as that ramp arrives, so when the last input word has been read, only the words still on their way through the system remain. The board measures 1,311 cycles, 13.1 µs, which is 2 % of the frame time. The core's own share is about a microsecond: the 58-cycle pipeline and the flush of the last partial output word. The remainder is consistent with input words still queued in the DMA and interconnect buffers when the DMA reports its last read, each of which the core consumes at two cycles per beat; the buffer occupancy was not measured separately.

DRHE in a sensor-paced system: one ramp. On the board the frame is fed at full speed, so a backlog can form in front of the core. In a real sensor chain the core is idle between ramps (it needs 320 cycles of the 7,850 each ramp provides), so no backlog forms: when the last

ramp arrives, only that ramp's work remains, about 320 cycles or 3.2 µs at 100 MHz. The board figure is therefore an upper bound on what DRHE adds in the sensor chain.

LPC: 3,006 µs on the board. LPC cannot emit anything until it has seen the whole frame, because its coefficients are computed from every ramp. When the last input word arrives, it still has all of its solve, residual and emit passes to run. The board measures 300,636 cycles, 3,006 µs, which is 92 % of the frame time and equals the solve and emit passes almost exactly (about 276,000 plus 24,576 cycles). Unlike DRHE's, this figure does not depend on how the frame is fed: in a sensor-paced system the ingest pass overlaps the acquisition, and the same 3.0 ms of work remains after the last ramp.
This is where the two algorithms differ most. DRHE adds microseconds and LPC adds milliseconds: a factor of 230 on the board, and of about a thousand in a sensor-paced system.

Kiem's 0.08 ms: what it measures. Kiem's system figure, 79,410 ns, is measured with the same definition: from the DMA interrupt that marks the last raw ramp being *read* from memory to the DMA interrupt that marks the last compressed word being *written* back. In his system that span includes the range FFT of the last ramp (1,787 cycles, 17.9 µs), DMA transfers and interrupt handling as well as the compressor. It is therefore an *added-latency* measurement for his whole processing chain, not a compression time and not a per-frame time. The board system of this thesis has no FFT core, so its 13.1 µs is the same kind of quantity for a shorter chain: the definitions match, but the pipelines measured do not, because his includes an FFT core in front of the compressor and this one does not. The two can be set side by side — both are tens of microseconds and both are negligible against the frame — but the difference between them says nothing about the compressors alone.

Decompression. The decoder side has its own frame time and latency, measured on the board in the `_full` builds with the same timer. The DRHE decompressor's two serial stages were designed for one value per clock, 196,608 cycles per frame (Section 6.5); on the board it takes 196,881 (1.97 ms), within 0.14 % of that. Its latency, from the last compressed word read to the last reconstructed word written, is 339 µs: because it decodes one value per clock while the compressed stream arrives faster, about a sixth of the frame is still to be rebuilt when the input ends. The LPC decompressor takes 15.4 ms per frame and 2.84 ms of latency, both set by its serial codeword search (Section 7.7). Both latencies are a decoder's own backlog, not a

dependence on the whole frame: the decoders emit reconstructed data from the first codeword on.

A.6.4 When does latency become a problem?

There are three thresholds, from hardest to softest.

1. The throughput threshold: failure. If the compressor needs longer to process one ramp than the sensor takes to produce the next one, a backlog builds up, and no finite buffer can hold it for ever. Data is lost. This is the only threshold at which the design stops working, so it is the first to check. For the ColoRadar sensor the limit is 7,850 cycles per ramp at 100 MHz. DRHE uses 320, so it would still keep up at a clock about 25 times slower. LPC's ingest pass runs at one beat per cycle, so it also keeps up with the ramps; what it must keep up with in addition is the frame rate, since its post-ingest passes must finish before the next frame's are due. Its 3.25 ms per frame against a 200 ms frame period meets that by a wide margin.

2. The budget threshold: the system's schedule. The downstream processor must receive each frame in time to finish its own work before the next frame arrives. Kiem made this explicit for his sensor. It has a 50 ms cycle and 21 ms of acquisition, which leaves 29 ms, and he allocated 15 ms of that to compression. For ColoRadar at 5 Hz the gap after acquisition is about 185 ms, and both designs use a tiny fraction of it.

The budget becomes informative when the designs are *scaled to Kiem's sensor*, whose frame is 21 times larger (512 bins by 1024 ramps against 128 by 192). Scaling the board's cycle counts linearly gives the following estimates (Chapter 8 returns to them when weighing the two algorithms):

- **DRHE** still processes one ramp at a time. At 512 bins and $\mathrm{II} = 2$ a ramp takes about 1,090 cycles, or 10.9 µs, against a ramp interval of about 20.5 µs. It keeps up, and its added latency stays in microseconds.
- **LPC**'s post-ingest passes scale with the frame. Scaling the measured 300,636 cycles gives about 64 ms of added latency at 100 MHz (69 ms per frame), which is more than four times Kiem's 15 ms budget and longer than his whole 50 ms cycle. Its frame buffer would need 67 Mbit (8.4 MB). That exceeds the entire block RAM of Kiem's `XCZU3CG`

(8.0 Mbit) and of the `XCKU5P` (17.7 Mbit, or about 36 Mbit with its UltraRAM), so it would have to move to external DRAM.

So on ColoRadar, LPC's latency is acceptable; on a longer-frame sensor like Kiem's it is the limiting factor, and it is the strongest hardware argument for DRHE in this thesis. The scaled figures are extrapolations from measured cycle counts, not measurements. They are accurate enough to show the budget is exceeded, because the margin is large: even the one-third reduction available from overlapping LPC's passes (Section 6.3) would leave it near 45 ms.

A related, quieter problem is *double buffering*. If one frame follows another with no idle gap, LPC is still emitting frame k while frame $k+1$ is arriving. It then needs a second frame buffer, doubling its largest cost. DRHE has no frame buffer to double.

3. The application threshold: reaction time. Finally, any added delay lengthens the time between something happening in front of the vehicle and the vehicle knowing about it. A physical scale helps. At 100 km h^{-1} a car travels 2.78 cm per millisecond. LPC's 3.0 ms is 8.4 cm of travel. DRHE's 13.1 µs on the board is 0.36 mm, and its sensor-paced 3.2 µs under a tenth of a millimetre. Both are small next to the frame period itself: a 5 Hz radar is already up to 200 ms, or 5.6 m, behind reality. Added latency becomes an application problem when it is a noticeable fraction of the frame period. LPC on ColoRadar is 1.5 %, so it is not a problem. LPC scaled to Kiem's sensor would exceed his entire 50 ms cycle, so it would be.

A.7 Throughput and initiation interval

What they are. Throughput is how many bits per second the compressor can accept. Initiation interval (II) is how many cycles it needs between accepting one input beat and the next. With a 128-bit input beat at clock f, the *theoretical* throughput is $128f/\mathrm{II}$. The *measured* throughput is the bits in a frame divided by the measured frame time, and is lower because of pipeline fill and drain and, on the board, the DMA start-up.

Why measure it. Throughput says whether the compressor keeps up with the sensor, which is the throughput threshold of Section A.6.4. II is the main thing that sets throughput, and it is the thing a designer can change.

Table A.12: Throughput in decimal Gbit/s. "Measured" is on the KCU116 for the final designs and in RTL co-simulation for the ladder variants, both including fill and drain; "theoretical" is $128f/\mathrm{II}$. Kiem's figure is theoretical: his 11.92 Gibit/s is 12.8 Gbit/s.

	Theoretical	Measured	bits/cycle (meas.)
DRHE, 100 MHz, II = 2 (board)	6.40	**5.12**	51.2
DRHE core at its OOC $F_{\max}$ (131.0 MHz)	8.38	6.7[a]	51.2
LPC, lags $n_{\mathrm{Tx}}, 2n_{\mathrm{Tx}}$, frame average (board)	—	**0.97**	9.7
DRHE-1, II = 2 (ladder step 0, co-sim)	6.40	5.14	51.4
DRHE-1 with II = 1 (ladder step 3, co-sim)	12.80	8.76	87.6
Replica of Kiem, II = 1 (co-sim)	12.80	11.54	115.4
LPC-1,2, frame average (co-sim)	—	1.59	15.9
Kiem [11]	12.80	not reported	—
Required: ColoRadar, 4 RX	0.209		
Required: ColoRadar, all 16 RX	0.835		
Required: Kiem's sensor	3.20		

[a]Not measured: the board's 51.2 bits per cycle scaled to the standalone core's $F_{\max}$.

Against Kiem: theoretical against theoretical. Kiem's 11.92 Gibit/s is calculated as stream width times clock, 128×100 MHz, expressed in binary units (his Equation 5.1). It is the rate his design would reach if it accepted a beat every cycle without pause; it is *not* a measured figure. Setting it against the board's *measured* 5.12 Gbit/s would suggest a gap of 2.5 times, but that mixes a theoretical number with a measured one. Theoretical against theoretical, the gap is exactly 2, which is II = 2 against II = 1. The remaining 20 % is per-ramp pipeline drain. It appears in the measured number, and it would not appear in Kiem's theoretical one even if his design had it too: the replica, his design rebuilt, measures 11.54 rather than 12.8 in co-simulation for the same reason.

Against the sensor: the comparison that matters. Figure A.5 shows capacity against requirement. On the board, DRHE accepts data 24.5 times faster than four ColoRadar channels produce it. Throughput beyond the requirement does not improve compression. It is spare capacity, and it can be spent in three ways:

- **Time-multiplexing.** One DRHE instance could serve all 16 receive channels with about six times headroom at 100 MHz, instead of four instances, at the cost of four times the state memory (Section A.8).
- **A slower clock.** Dynamic power scales roughly with clock frequency. A design with 24 times headroom does not need to run at 100 MHz.

- **Larger sensors.** On Kiem's sensor, which needs 3.2 Gbit/s during a frame, DRHE's measured 5.12 Gbit/s still has 1.6 times headroom, with no buffer.

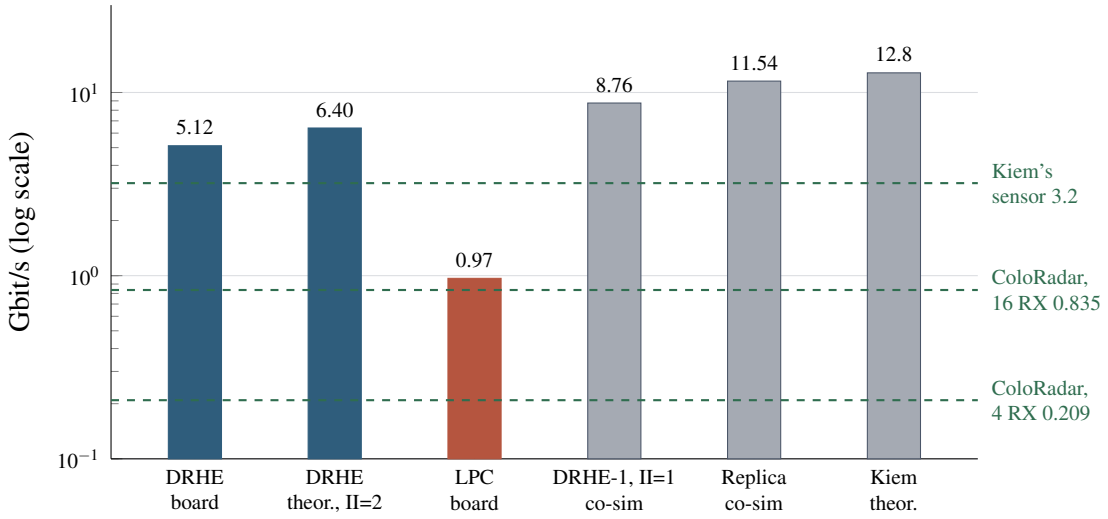


Figure A.5: Throughput capacity (bars) against three requirements (dashed). Capacity above a dashed line is headroom. DRHE (blue) clears every requirement on the board, including Kiem's much faster sensor. LPC (red), measured over a whole frame, clears ColoRadar's full 16-channel rate only narrowly, which matters only if frames arrive back-to-back. Grey bars are the II = 1 comparison points of the ladder and Kiem's theoretical figure.

Against the architecture's own ideal. The 128-bit input could in principle carry 128 bits per cycle. DRHE achieves 51.2 on the board, which is 40 % of the input width and 80 % of what II = 2 allows. This is the fairest measure of implementation efficiency, because it needs nothing external. It also points at the two improvements: the II, which Section A.5.4 traced to one write and showed how to remove, and the per-ramp drain, which a loop that runs across ramp boundaries without draining would remove.

LPC's throughput is a frame average. LPC's 0.97 Gbit/s is the frame's bits divided by the whole frame time. Its ingest pass alone accepts 12.8 Gbit/s, one beat per cycle; it is the post-ingest passes that bring the average down. The average is the relevant figure when frames arrive one after another, as on the board. Against ColoRadar's 16-channel rate it leaves only 1.16 times headroom, which would matter only for a sensor that delivers frames back to back.

When does it become a problem? When headroom falls below 1. For a single four-channel ColoRadar stream that would take a design about 25 times slower than this one. The practical risk is elsewhere. At II = 2, the compressor accepts data at half the rate a streaming FFT core

can deliver it within a ramp, so the FFT must be allowed to pause (AXI back-pressure), or there must be a small FIFO between them. One ramp for four channels is 16 kbit, half a 36 Kb block. At $\mathrm{II} = 1$ the question would not arise.

A.8 Resources against the device

Why measure resources at all? Every resource number answers three questions. *Does it fit* on the chip that would actually be used, next to everything else that chip must do? *What does it cost* compared with another design that does the same job? And *how does it scale* when the sensor gets bigger? Section A.5 answered the second. This section answers the first and third, starting from the board cores of Table 7.8.

Table A.13: Post-route resources of each core as a percentage of the `XCKU5P` used here (216,960 LUT, 433,920 FF, 480 BRAM, 1,824 DSP) and of Kiem's `XCZU3CG` (70,560 LUT, 141,120 FF, 216 BRAM, 360 DSP); BRAM counted in 36 Kb blocks. The final cores are the in-system figures of Table 7.8; DRHE-1 and the replica are out of context. Kiem's compression IP is shown for reference.

	% of `XCKU5P`				% of `XCZU3CG`			
Core	LUT	FF	BRAM[a]	DSP	LUT	FF	BRAM[a]	DSP
DRHE compressor	16	5	8	7	50	17	19	37
DRHE decompressor	7	2	7	3	20	7	15	13
LPC compressor	16	5	20	9	48	14	45	44
LPC decompressor	9	2	0	0.4	28	6	0	2
DRHE-1 (lag 1)	16	5	4	7	50	17	9	37
Replica of Kiem	8	1	2	3	24	4	5	13
Kiem's IP [11]	21	2	2	2	65	7	5	12

[a] 36 Kb blocks.

It fits the board with room to spare. On the `XCKU5P` each compressor uses about a sixth of the logic, and encoder and decoder together under a quarter (23 % for DRHE, 25 % for LPC). The whole test system with both cores — processor, DDR4 controller, two DMAs — uses 38 % and 40 % of the LUTs (Table 6.1). The scarcest resource is LPC's block RAM, at a fifth of the device for one four-channel compressor.

It fits Kiem's device. Every compressor fits the `XCZU3CG`, and the DRHE compressor uses fewer of its LUTs than Kiem's own IP does (50 against 65 %), though more of its flip-flops,

DSPs and block RAM. Kiem's complete system (FFT, DMA, processor interface and compressor) used 59,353 LUTs, so everything except his compressor used 13,461. The DRHE compressor in place of his would bring the system to about 48,900 LUTs, or 69 % of the device, inside the usual 70–80 % guideline above which placement and routing become congested. The decompressor is not part of this budget: it sits at the receiving end of the link, on the central processor, not beside the sensor.

Scaling to the whole sensor. The cores compress four receive channels; the ColoRadar cascade has sixteen. Two ways of covering all sixteen follow from the numbers:

- **Four DRHE compressors**, about 141,800 LUTs and 160 blocks of 36 Kb. That fits the `XCKU5P` (65 % of its LUTs, 33 % of its block RAM) but not the `XCZU3CG`.
- **One DRHE compressor, time-multiplexed.** Its throughput headroom of about six over the 16-channel rate makes this possible. Logic stays at one instance; the state memory grows four-fold to 160 blocks, which fits the `XCKU5P` comfortably and the `XCZU3CG` only tightly (74 % of its block RAM, before the rest of the system is placed). In Kiem's 16-bit fixed point the same state would be 80 blocks.

LPC's memory is its limit: its compressor needs 98 blocks per four channels, so all sixteen channels would need about 390 blocks, 82 % of the `XCKU5P` and nearly twice the `XCZU3CG`. At Kiem's frame size the frame could not be held on chip at all (Section A.6.4).

What the resources are spent on. The resource accounting of Section A.5 carries over to the final cores, because their datapaths are DRHE-1's with deeper state. About two thirds of the DRHE compressor's logic and all of its DSP slices are floating-point operators, a third of its logic is the two arctangent units, and its block RAM is twice what 16-bit state would need. Replacing the arithmetic with Kiem's fixed-point types would roughly halve the compressor's logic, cut its DSPs to a third and its block RAM to half, at a cost in compression ratio of 0.07 % (Section A.5.5). It is the largest single saving available to the design.

A.9 Power

What it is. The routed design's power report splits consumption into *device static* power, which the chip draws whenever it is switched on, whatever it is doing, and *dynamic* power,

which the design draws by switching its logic, registers, DSPs, memories and wires every clock cycle. Only the dynamic part belongs to the design.

Why measure it. A compressor exists to save link bandwidth, and in a vehicle bandwidth and power are both paid for. A compressor that costs more power than the bits it saves is not worth fitting.

Table A.14: Vectorless power estimates from the routed out-of-context designs at 100 MHz, with Vivado's default switching activity. Static power is the `XCKU5P`'s own and is the same for every design. Energy per frame is dynamic power multiplied by the co-simulated time to compress one frame; energy per bit divides that by the frame's 3,145,728 input bits.

Design	Total (W)	Static (W)	Dynamic (W)	µJ per frame	pJ per bit
DRHE, lag n_{Tx}	1.113	0.456	0.657	402	128
LPC, lags $n_{\mathrm{Tx}}, 2n_{\mathrm{Tx}}$	1.011	0.455	0.555	1805	574
DRHE-1 (lag 1)	1.017	0.455	0.562	344	109
LPC-1,2 (lags 1, 2)	1.165	0.457	0.709	1399	445
Replica of Kiem	0.738	0.453	0.286	78	25

Against Kiem: nothing to compare. Kiem does not report power. The replica is therefore the only reference. DRHE-1 draws about twice the replica's dynamic power, which is what its resource counts predict, since dynamic power scales with the logic, registers and DSPs that switch; and because the replica finishes a frame in 27,270 cycles against DRHE-1's 61,253, DRHE-1 spends about 4.4 times as much energy per frame. The final DRHE compressor draws a little more than DRHE-1 (0.657 against 0.562 W), because its state memory is twice as large. LPC is the most expensive per frame by a wide margin, because its dynamic power is similar to DRHE's but it takes five times as long.

Against the job. At the sensor's actual rate the compressor is busy only a few per cent of the time. Energy per frame is therefore the useful figure: at 5 frames/s, the DRHE compressor's 402 µJ per four-channel frame is about 2 mW of average switching power, and LPC's 1805 µJ about 9 mW. The clock network keeps toggling between ramps unless it is gated, and is about a ninth of the dynamic figure. Both are negligible next to the 0.46 W the device draws simply by being switched on. That static power is a property of the evaluation board's large FPGA, not of the compressor; in a production ASIC or a small device it would be far lower.

How far to trust it. These are vectorless estimates from routed netlists with an assumed 12.5 % toggle rate on every signal, not measurements on the board and not driven by recorded switching activity. Vivado rates their confidence as "medium". They are good for ranking designs against each other and for the order of magnitude; a measurement of the KCU116's supply current, or a power analysis driven by the switching activity of a co-simulated frame, would be needed for an absolute figure.

A.10 Compression ratio: what counts as good

What it is. Compression ratio (CR) is bits in divided by bits out, including the padding of the last output word, measured on real data (Section 5.3). It is the reason the compressor exists. On the board, with the original dictionary, it is 3.7508 for DRHE and 3.6968 for LPC; with the re-derived dictionaries, which change only the contents of the codeword table, it is 4.217 and 4.403. All four are identical to C-simulation (Section 7.6).

Against the link. The link sets the requirement. Kiem's sensor produces 2.4 Gbit/s of raw ADC data. To fit an automotive Gigabit Ethernet link it needs a CR of 2.4, or 2.7 once protocol headers are included. Both designs clear that with either dictionary, and so does the worst single frame of the recording (3.517 for DRHE and 3.461 for LPC with the original dictionary). For ColoRadar the picture is different, and it should be stated plainly: its post-FFT rate for all 16 channels is 0.835 Gbit/s while a frame is being acquired. That already fits under 1 Gbit/s, if only just, before compression. After compression it is 0.223 Gbit/s (original dictionary) or 0.198 Gbit/s (DRHE, re-derived). On this sensor, compression buys *margin* and room for more channels, not the ability to use Gigabit Ethernet at all.

Against Kiem: not valid. Kiem's hardware CR was 3.17, on his own data. The number looks comparable, but it is not. The decomposition of Section 7.3 shows that a large part of this thesis's ratio comes from unused headroom in the 16-bit container, before any prediction. That is a property of how ColoRadar's data is scaled, not of the algorithm. A CR measured on a different sensor with different scaling measures a different thing, so a higher or lower number on another dataset says nothing about which implementation is better. Appendix B makes the same point from Kiem's side, with a rebuilt simulation of his chain, and goes through each of his results in turn.

The valid yardsticks. These are the internal ones.

- **No prediction.** The same entropy coder applied to the FX16 codes directly gives 3.479 over the same 50 frames. This is the test of whether prediction helps at all. Both designs beat it, by 7.8 % (DRHE) and 6.3 % (LPC) with the original dictionary (Section 7.3). Predicting at lag 1 instead would fail it: the lag-1 variants compress to 3.310 and 3.385, below the coder alone, because on a TDM-MIMO sensor the adjacent ramp belongs to a different transmitter (Section 7.2).
- **The entropy bound.** The order-0 entropy of the values being coded is the least any coder of this type could spend per value. Table 7.6 in Section 7.4.1 sets it against what the coder spends: the original dictionary's excess over the bound grows as prediction improves, while an ideal S_4 dictionary comes within a few per cent of it. The re-derived dictionaries close most of that excess and recover 12 % (DRHE) and 19 % (LPC) of ratio (Section 7.4).
- **Each other.** DRHE and LPC run on the same data through the same coder and the same packer, so their ratios are directly comparable: 3.751 against 3.697 with the original dictionary, 4.218 against 4.404 with re-derived ones.

When does it become a problem? When the compressed rate, plus protocol overhead, no longer fits the link. The container headroom of Section 7.3 matters here. A sensor whose ADC output filled the 16-bit container would not have it, and the quantisation study of Section 7.1.1, which fills the container with the same frame's own data, measures a lossless DRHE ratio of only 1.18 — far below the 2.4 Kiem's link requires. A CR of 3.75 on this dataset is therefore not a guaranteed pass on every sensor; what transfers between sensors is the prediction gain over the no-prediction baseline, not the ratio itself.

A.11 Every difference from Kiem, accounted for

Table A.15 closes the appendix's account. It lists every number in which this thesis's DRHE and Kiem's differ, says whether the comparison is valid as published, names the cause of the difference and points to the measurement that shows it. Where the cause is one design decision, the evidence is the step of the ladder that changes only that decision. Where it is not, the table says so.

Table A.15: Every difference between this thesis's DRHE and Kiem's published design, with its cause and the evidence for it. "Final" is the lag-n_{Tx} compressor on the board (in-system resources, board timing; its $F_{\max}$ is given both for the board system and for the standalone core); DRHE-1 and the replica are post-route, out of context, on the `XCKU5P`. "Valid" says whether the published numbers measure the same thing. The replica is the fair yardstick for what the design decisions cost.

Metric	Final	DRHE-1	Replica	Kiem	Valid	Cause of the difference	Evidence
LUT	35,444	35,343	17,141	45,892	yes	Against the replica: floating-point operators (69 % of DRHE-1's LUTs), then the packer. The replica against Kiem: about 5 % explained.	ladder steps 1, 2, 4; Table A.10
FF	23,528	23,546	5,050	9,243	yes	Registers inside float operators (54 %) and along a 58-stage pipeline. Kiem's extra over the replica: about 45 % explained (speed grade, packer, depth).	ladder steps 4, 5; Table A.10
DSP	132	132	48	44	yes	Every DSP belongs to a float operator; each float multiply needs 3, each sine/cosine 13. Sharing at II = 2 halves the count.	Table A.7; step 3 (264 at II = 1)
BRAM (36 Kb)	40	20	10	10	yes	32-bit state words need a full block per array; 16-bit words need half. The final design doubles it again with per-transmitter state.	Table A.6; replica matches exactly
II	2	2	1	1	yes	One extra write per cycle to clear the state; three accesses on two ports.	HLS message; step 3 reaches II = 1
Depth (cycles)	58	58	12	23	yes, both HLS	Closed loop: polar conversion waits for the prediction (26 cycles); the rest is float operator latency.	Figure A.3; steps 4, 5
$F_{\max}$ (MHz)	100 met, WNS +0.24 to +0.34 ns (system); 131.0 (core, OOC)	135.5 (OOC)	129.5 (OOC)	≥100	partly	All meet 100 MHz; Kiem reports only that he met it, on speed grade −1.	Table A.11; Table 6.1
Throughput (Gbit/s)	6.40 th., 5.12 board	6.40 th., 5.14 co-sim	12.80 th., 11.54 co-sim	12.80 th.	theor. only	II = 2 against II = 1. Kiem's 11.92 is in binary units and theoretical.	Table A.12
Latency	13.1 µs board; 580 ns pipeline	580 ns pipeline	—	79.4 µs system; 230 ns pipeline	yes, same definitions	Pipeline: the depth above. System: same definition, but his pipeline includes an FFT core and this one does not.	Section A.6
Power (dyn.)	0.657 W	0.562 W	0.286 W	not reported	—	Scales with switching resources; float design ≈ 2× replica.	Table A.14
Compr. ratio	3.75; 4.22 (board)	3.310	3.319 (10 fr.)	3.17	no	Different sensor and data scaling; float and fixed point differ by 0.07 % on the same data.	Section A.10
`XCZU3CG` LUTs	50 %	50 %	24 %	65 %	yes	Follows from the LUT row.	Table A.13

The account in one paragraph. Measured post-route, as Kiem's figures are, this thesis's DRHE compressor uses fewer LUTs than his IP, meets the same 100 MHz clock (with a quarter of the period to spare as a standalone core, and with 0.24 ns to 0.34 ns of slack in the complete board systems), and fits his device at half its logic. Against a replica of his design built in the same tool, it is larger, and by exactly the amounts its distinctive decisions cost: floating-point arithmetic (most of the logic and registers, every extra DSP, twice the block RAM), a less economical packer (about 4,200 LUTs), a closed loop (26 cycles of depth in floating point, almost nothing in fixed point), one extra state write (II = 2, which halves throughput but also halves the arithmetic), and per-transmitter state (twice the block RAM again, and nothing else). The closed loop buys a guarantee that the encoder and decoder can never diverge; II = 2 buys a smaller design the sensor does not notice; the per-transmitter state is what makes prediction pay at all on a TDM-MIMO sensor; the floating-point datapath buys only 0.07 % of compression ratio. What remains unexplained is the replica's distance from Kiem's own LUT count, and Table A.10 shows how much of it the testable explanations cover. Chapter 8 draws the design conclusions; the comparison of compression results with Kiem's is in Appendix B.

Appendix B

Comparison with the Baseline

Chapter 3 set out what Kiem measured; Chapter 7 set out what this thesis measures. This appendix puts the two side by side. That is less simple than it sounds, because every comparison has to cross two gaps at once: different data and a different radar. Kiem's compression ratios come from private recordings of a single-transmitter sensor with real-valued samples; this thesis's come from a public twelve-transmitter cascade with complex samples. Four questions cannot be answered across both gaps together:

1. Kiem reports a ratio of 3.25 for DRHE on his recordings; on ColoRadar this thesis's DRHE reaches 3.75. What actually sets the ratio, and are the two numbers comparable? As a test of that question, this appendix also runs this thesis's own chain on synthetic scenes of the kind Kiem describes, and they compress to only about 1.1 (Section B.6). A comparison has to explain all three figures.
2. Kiem calls LPC "the same approach as DRHE" but less promising. The LPC of this thesis is level with DRHE. Which is right?
3. Is the DRHE of this thesis the algorithm Kiem specified?
4. The hardware differs from his in every resource figure. Which differences come from design decisions, and which from tools, devices and data?

The appendix answers them with two reconstructions. The first rebuilds Kiem's MATLAB simulation *from his text alone* — his radar, his scene, his processing chain, one transmitter — and runs on it every coder discussed in this thesis (Sections B.2–B.9). It closes the data gap for the first three questions. The second rebuilds his compression IP in this thesis's tool flow; it is the subject of Appendix A (Section A.4 onwards), which closes the tool and device gap for the

fourth question, and Section B.12 summarises what it means for this comparison. With both in place, every result and claim of his is compared with this thesis's in turn (Sections B.13–B.19).

B.1 Three kinds of comparison

Not every pair of numbers can be compared directly, and the most common error in comparing compression results is to compare numbers that measure different things. Each comparison in this appendix is placed in one of three classes.

Like for like

The two numbers measure the same quantity of the same thing, and a difference is a real difference. Examples: the arithmetic specification of DRHE; post-route LUTs of two circuits built for the same device.

Through a bridge

The numbers come from different data or setups, but something built for the purpose connects them: the rebuilt simulation of his chain, or the rebuilt copy of his hardware.

Not comparable

The numbers look alike but measure different things, and no bridge closes the gap. Example: his CR on his recordings against this thesis's on ColoRadar. For these the appendix explains what each number does measure.

B.2 Rebuilding Kiem's simulation

The replication is a separate MATLAB code base that uses base MATLAB only and shares no code with the evaluation chain of Chapter 5. Its stages follow Kiem's Figure 3.1 and are listed in Table B.1 with the section of his thesis each one implements.

Every lossless path is decoded and compared with its input on every run; none is assumed lossless. One detail of the DRHE decoder is specific to a floating-point model. MATLAB's `round` returns -0 for small negative values, and $\operatorname{atan2}(-0, x < 0) = -\pi$ while $\operatorname{atan2}(+0, x < 0) = +\pi$. An encoder that saw -0 and a decoder that rebuilds the value as an integer, and so sees $+0$, would form phase predictions 2π apart. The FX16 quantiser of the replication therefore normalises -0 to $+0$. A hardware implementation never meets this, because integers have no negative zero, but a floating-point model of one can.

Table B.1: The rebuilt simulation, stage by stage, with the part of Kiem's thesis each stage is built from.

File	Role	Kiem
`kr_config.m`	every parameter, with its source or "ASSUMPTION"	Tables 3.2, 3.3, 4.4; App. A
`kr_generate_adc.m`	beat signals of the targets, AWGN, 14-bit ADC; optional TDM interleaving	§3.2.2
`kr_calibrate_noise.m`	ADC noise level that gives a chosen noise floor after NCI	Table 3.2
`kr_preprocess_fft.m`	shift to 32-bit full scale, Hanning, FFT/N, first half	§3.2.1, §4.3.3
`kr_fp16.m`, `kr_fx16.m`	FP16 reference; FX16 in two scalings	§3.4.1
`kr_second_stage.m`, `kr_detect.m`	Doppler FFT, NCI, noise threshold and local maxima	§3.2.1
`kr_metrics.m`	CR, FN%, FP%, NF, SNR against FP16	§3.4.2
`kr_drhe_*.m`	DRHE model prediction and its inverse	§3.5.3, §3.5.4
`kr_bits_s4.m`, `kr_bits_r4s4.m`	S_4 coder (fixed or ideal dictionary); R4S4 + run-length coder	Table 3.7, §2.7.3, §3.5.1
`kr_lpc_raw.m`	LPC-Huffman of [27]: fast-time prediction of raw ADC	§2.7.4
`kr_lpc_fft.m`	this thesis's slow-time LPC, for comparison	—
`kr_lossy.m`	approximations of CMPRA/B and EGE10/8/6/4	§2.7.1, §2.7.2
`run_kiem_replication.m`	the whole workflow at two noise levels	Fig. 3.1

B.2.1 The radar and the scene

The radar is Kiem's Table 3.2: one MMIC, **one transmitter**, four receivers, 1024 samples per ramp, 512 ramps, a 14-bit ADC. The scene has his five targets at range bins [100, 150, 150, 200, 400] and Doppler bins [50, 50, 100, 200, 250] with amplitudes [0.8, 0.3, 0.4, 0.5, 0.7] of full scale, and white Gaussian noise set so that the noise floor after NCI is his -75 dBFS. Figure B.1 shows it; it is visibly the scene of his Figures 3.5 and 3.7.

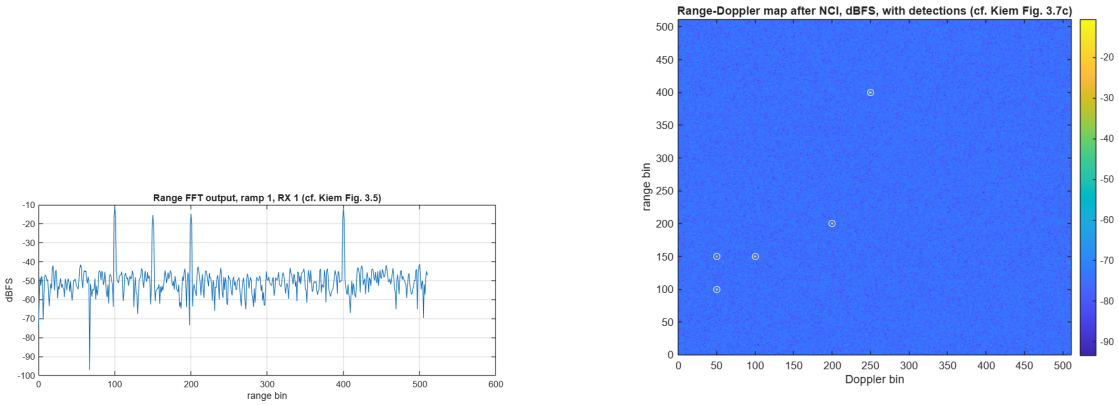


Figure B.1: Kiem's Table 3.2 scene as rebuilt. Left: range FFT of ramp 1, receiver 1 (compare his Figure 3.5). Right: range–Doppler map after NCI with the five reference detections circled (compare his Figure 3.7c).

B.2.2 Where Kiem's text leaves a choice

Seven parameters are not given in his thesis. Each is set in one place (`kr_config.m`), marked there as an assumption, and can be changed.

Target angles

Not given. Fixed, distinct angles ($-30°$, $-10°$, $0°$, $15°$, $40°$) on a half-wavelength array, so the four receive channels differ.

Amplitudes

His five amplitudes sum to 2.7 of full scale, which a 14-bit ADC cannot hold without clipping. They are scaled together so the worst-case sum just fits.

FX16 scaling

Two modes, both fixed and independent of the data. In *hw*, the default, the FX16 code is the top 16 bits of the 32-bit FFT word, which is what his FFT core delivers: the full scale of the *word* maps to 32,767. In *full scale*, the scaling of this thesis's chain

(Equation (2.11)), the spectral peak of a full-scale *sinusoid* maps to 32,767. They differ by exactly a factor of four, because a full-scale real sinusoid reaches only a quarter of the word's full scale after the range FFT: the Hanning window's coherent gain halves it, and a real sinusoid's amplitude is split equally between its positive- and negative-frequency bins, which halves it again. Equation (2.11) divides both factors out.

0 dBFS

Not defined in his thesis. The NCI power of a full-scale sinusoid through the whole chain is the default; the power of a unit value in the 32-bit word is also reported, and reads 12.0 dB lower.

Detector

Noise estimate the mean dB of the lowest 75 % of cells; threshold 15 dB above it; 8-neighbour local maxima — the detector of Section 5.2.

Noise floor and SNR

The noise floor is the mean dB outside a 5×5 patch around each detection; SNR is the RMS peak power in dB minus it, as in his Section 3.4.2.

LPC-Huffman of Meucci and Mancuso

Order not given, so $p = 1 \ldots 10$ is swept; one coefficient set per chirp and receiver, sent as 32-bit floats; the Huffman dictionary counted at 21 bits per symbol used.

His lossy schemes cannot be reproduced exactly, because their bit layouts are proprietary. They are approximated — CMPRA and CMPRB as block floating point with one 4-bit exponent per 256-bit word, EGE-k as blocks of eight complex bins with an 8-bit header and least significant bits dropped to meet CR $16/k$ — and are included for completeness rather than as evidence.

B.3 Results at Kiem's noise level

Table B.2 runs the complete workflow at the -75 dBFS of his Table 3.2 and sets his published Table 3.5 beside it. The published numbers are from his real recordings, not from this scene; they are there to be read against, not matched.

Three things stand out.

DRHE compresses to 2.28 on Kiem's own scene, not 3.25. The scene is his, the chain is his and the algorithm is his (Section B.10 shows it is exactly his). The ratio is not, which already

Table B.2: Kiem's workflow on his Table 3.2 scene at his −75 dBFS noise floor, beside his Table 3.5. ADC noise $\sigma = 375.9$ codes; FX16 noise 20.35 LSB; five reference detections. "(approx.)" marks schemes approximated because their exact layout is proprietary.

	replication					Kiem, real data		
Algorithm	CR	FN%	FP%	NF	SNR	CR	FN%	FP%
FP16 (reference)	1.00	0	0	−75.00	61.50	1.00	0	0
FX16	1.00	0	0	−75.00	61.50	1.00	0.59	1.18
CMPRA (approx.)	2.00	0	0	−74.86	61.36	2.00	0.59	0.59
CMPRB (approx.)	4.00	0	13980	−75.56	40.55	4.00	7.69	4.73
EGE10 (approx.)	1.60	0	0	−75.00	61.50	1.60	1.18	1.18
EGE8 (approx.)	2.00	0	0	−74.98	61.48	2.00	2.37	2.37
EGE6 (approx.)	2.67	0	0	−74.79	61.29	2.67	7.69	7.10
EGE4 (approx.)	4.00	0	680	−74.71	52.31	4.00	40.24	26.63
DRHE, R4S4 + RLE	**2.28**	0	0	−75.00	61.50	**3.25**	0.59	1.18
DRHE, S_4 only, ideal dictionary	2.28	0	0	−75.00	61.50	3.24	0.59	1.18
DRHE, S_4, Kiem's fixed dictionary	1.85	0	0	−75.00	61.50	(3.17 HW)		
LPC-Huffman, raw ADC [27], $p = 10$	1.32	0	0	−75.00	61.50	(2.056 in [27])		
LPC, slow time (this thesis)	2.06	0	0	−75.00	61.50			
No prediction, fixed dictionary	2.03	0	0	−75.00	61.50			

shows that his 3.25 is a property of his recordings rather than of his illustrative scene.

The detection columns do not resemble his. This scene has five targets 60 to 74 dB above the noise; his recordings had about 170 detections at 22 dB. With five detections, one miss would be 20 %, so FN and FP are not comparable at all. The same high SNR is why the approximated lossy schemes produce large false-discovery rates here: at 60 dB SNR a 3-bit mantissa leaves quantisation spurs in the range–Doppler map that are 20 to 40 dB above the noise and are detected as targets. Those FP percentages describe this scene, not the schemes.

Prediction is doing almost nothing. No prediction at all gives 2.03 through the fixed dictionary; DRHE gives 1.85 and the slow-time LPC 2.06. In a scene that is 99.99 % noise cells there is nothing to predict: noise at ramp $m-1$ says nothing about noise at ramp m. Section B.5 shows that this holds at every noise level.

B.4 The noise level at which DRHE gives 3.25

On a white-noise scene the compression ratio depends on one thing above all: the standard deviation of the noise in FX16 least significant bits. A bisection on the noise floor (`kr_find_nf_for_cr.m`) finds the level at which DRHE with run-length coding gives Kiem's

3.25: −87.9 dBFS, where the ADC noise is 85.4 codes and the FX16 noise 4.63 LSB. Table B.3 gives the full workflow there.

Table B.3: The same workflow at the noise floor where DRHE reproduces Kiem's 3.25. ADC noise $\sigma = 85.4$ codes; FX16 noise 4.63 LSB. Detection metrics are omitted: every lossless row equals FX16's (no misses, no false detections).

Coder	replication	Kiem
DRHE, R4S4 + RLE, ideal Huffman	**3.25**	**3.25** (his Table 3.5)
DRHE, S_4 only, ideal Huffman	**3.24**	**3.24** (his Table 3.6)
DRHE, S_4, Kiem's fixed dictionary	3.23	3.17 (hardware, different FFT)
LPC-Huffman on raw ADC [27], $p = 10$	1.54	not run (2.056 in [27])
LPC, slow time, fixed dictionary	3.38	—
LPC, slow time, ideal Huffman	3.43	—
No prediction, fixed dictionary	3.26	—
No prediction, R4S4 + RLE	3.38	—

At this one noise level the replication reproduces Kiem's DRHE pattern exactly: 3.25 with run-length coding, 3.24 without (his Table 3.6), and almost no loss from fixing the dictionary (3.23). It reproduces his observation that run-length coding adds nearly nothing. But the operating point was *chosen* to give 3.25; it is not a prediction. And it sits 17.2 dB below the −70.7 dBFS noise floor he reports with that 3.25, a discrepancy that Section B.7 examines.

B.5 Compression ratio across noise levels

Figure B.2 sweeps the noise floor of the same scene from −120 to −60 dBFS, for both FX16 scalings (`kr_noise_sweep.m`). Kiem's published result is placed on it, and so are the results of the synthetic-scene test of Section B.6, which runs this thesis's own chain.

The curves have three regimes.

- **Very low noise** (left). The FX16 noise rounds to zero, the residuals are nearly all zero, and the fixed dictionary's floor takes over: every value costs at least four bits (Section 4.2.6), so the ratio saturates just below 4.0 (3.92). With run-length coding and an ideal dictionary it would grow without limit (82 at −140 dBFS), because long runs of zeros cost almost nothing; that regime is irrelevant to a real radar.
- **The transition** (centre). Each 6 dB of extra noise doubles the noise in LSB, adds about one bit to every residual and lowers the ratio.
- **High noise** (right). Every value carries several bits of noise and the ratio falls towards 1.

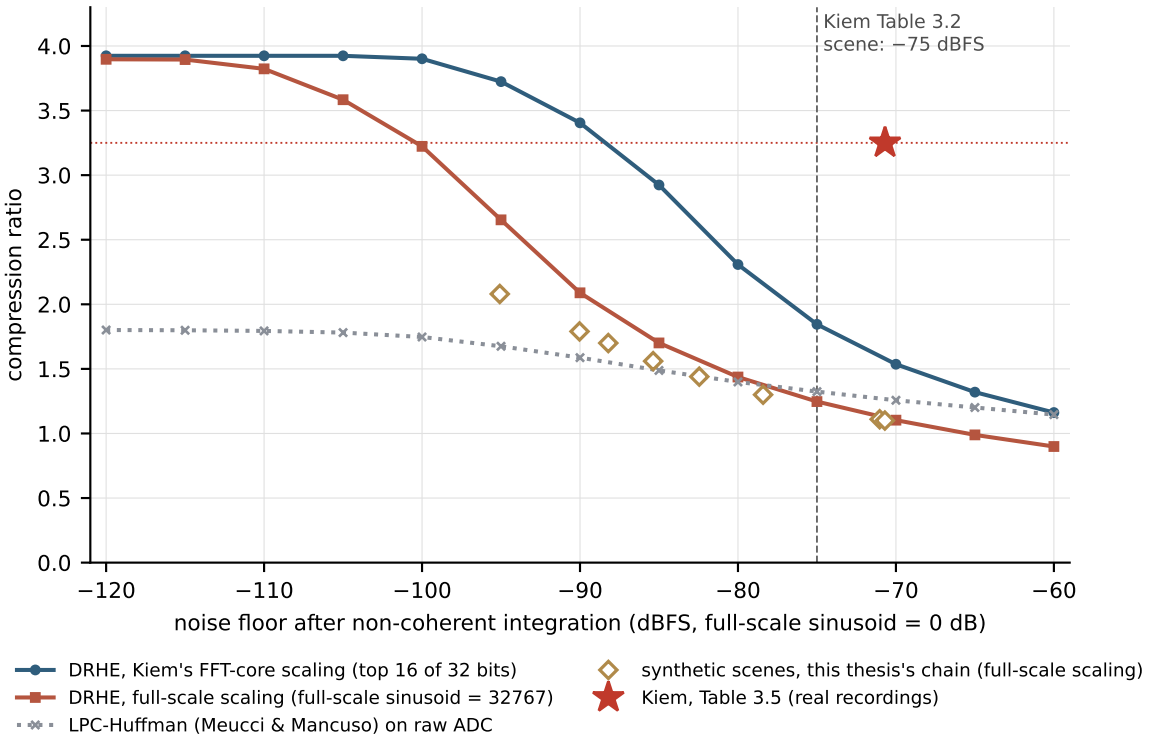


Figure B.2: Compression ratio against noise floor on Kiem's Table 3.2 scene with one transmitter. Blue: DRHE with Kiem's fixed dictionary and his FFT core's scaling. Red: the same with the full-scale scaling of this thesis's chain, four times larger. Grey: LPC-Huffman of [27] on raw ADC. Diamonds: the synthetic-scene test of Section B.6, run through this thesis's chain. Star: Kiem's published 3.25 at −70.7 dBFS, which lies 17 dB to the right of where his own scaling's curve passes 3.25.

The two scalings produce the same curve shifted by 12 dB, as a factor of four in amplitude must. Along both curves, no prediction and the slow-time LPC stay within about ten per cent of DRHE either way, and with the fixed dictionary no prediction is often the better of the two (Tables B.2 and B.3): on white noise the predictor is not what sets the ratio. The diamonds, from this thesis's chain, fall on the red curve; the next section explains why.

B.6 This thesis's chain on synthetic scenes

The replication makes possible a direct test of how much of a compression ratio belongs to the chain rather than to the algorithm. Synthetic scenes of the kind Kiem's thesis describes — a handful of point targets in white Gaussian noise — were generated at eight noise floors between −95 and −70.7 dBFS and put through this thesis's own MATLAB chain (Section 5.2), with its full-scale FX16 scaling, DRHE and Kiem's fixed dictionary. They are the diamonds of Figure B.2. Near the −70.7 dBFS that Kiem reports, they compress to only 1.10–1.12; at −95 dBFS they reach 2.08. Running this thesis's chain on scenes of Kiem's kind therefore gives about 1.1, a third of his 3.25, with the same algorithm and the same dictionary. The replication explains why. Table B.4 runs Kiem's scene at three noise floors with both scalings.

Table B.4: DRHE and the other coders on Kiem's scene for both FX16 scalings. "Noise" is the standard deviation of FX16 codes in noise-only range bins. "DRHE, fixed" uses Kiem's fixed dictionary; "R4S4" is the run-length coder with an ideal dictionary.

FX16 scaling	NF (dBFS)	noise (LSB)	DRHE, fixed	DRHE, R4S4	LPC	no pred.
hw (Kiem's FFT core)	−70.7	33.4	1.57	2.07	1.73	1.71
hw	−75.0	20.4	1.85	2.28	2.06	2.03
hw	−87.9	4.6	**3.24**	3.25	3.38	3.26
full scale (this thesis)	−70.7	133.4	**1.12**	1.64	1.20	1.19
full scale	−75.0	81.3	1.25	1.78	1.34	1.32
full scale	−87.9	18.4	1.91	2.32	2.13	2.08

The row "full scale, −70.7 dBFS" gives 1.12, the same ratio as this thesis's chain on its synthetic scenes (1.10–1.12), reproduced in an independent code base from nothing but Kiem's text and the full-scale scaling. Two factors multiply to produce it:

- **The scaling, a factor of four.** Both chains use a fixed full-scale scaling, but they calibrate it to different references. This thesis's maps the peak of a full-scale sinusoid to 32,767; Kiem's core keeps the top 16 bits of the 32-bit FFT word, in which the same sinusoid

peaks at about 8,191, a quarter of the word's full scale (Section B.2.2). The same noise therefore occupies four times as many LSB here, about two more bits per value.

- **The noise floor, a factor of seven.** Matching Kiem's stated −70.7 dBFS with white noise gives 33 LSB even with his scaling; his CR of 3.25 needs about 4.6 LSB.

Together: 133 LSB of noise where his result implies about 5, some thirty times more. It is tempting to explain the low ratio by saying that white noise lacks the slow-time correlation DRHE needs. White noise is indeed unpredictable, but that is not what sets the ratio: the no-prediction column shows the same ratios without any predictor. The ratio is set by how many bits of noise each value carries, and that is fixed by the scaling and the noise level before any coder runs. The same mechanism, seen from the other side, is why the ColoRadar capture compresses as well as it does: its FX16 codes occupy fewer than ten of the sixteen bits (Section 7.3).

B.7 Kiem's noise floor and his compression ratio do not agree

The replication cannot make Kiem's reported noise floor and his reported compression ratio hold together on a white-noise scene, under any reading of his text. With his FFT scaling, CR 3.25 needs −87.9 dBFS by the full-scale-sinusoid reference or −99.9 dBFS by the unit reference; he reports −70.7 dBFS. These are the only two references his text suggests, and the second only widens the gap.

B.7.1 His dictionary says what his residuals looked like

Kiem built his fixed Huffman dictionary from the S_4 statistics of his own residuals (Section 3.6). A Huffman code is optimal only for the distribution it was built from, so its lengths can be read backwards. Table B.5 (`kr_dictionary_fingerprint.m`) codes integer-rounded Gaussian residuals of increasing spread with his dictionary and with the ideal code for each spread.

The dictionary is optimal for residuals with a standard deviation of 6 to 8 LSB and costs 26 % for the 33 LSB that his reported noise floor would give on white noise. Its fingerprint agrees with his CR of 3.25 and with his residual histograms (his Figures 3.9–3.10, "most values" within ±40); it does not agree with a white-noise floor at −70.7 dBFS. The same table explains,

Table B.5: Kiem's fixed dictionary against an ideal Huffman code on Gaussian residuals. The overhead is smallest where the dictionary matches the data it was made from.

Residual std (LSB)	CR, Kiem's dictionary	CR, ideal code	Overhead	Commonest S_4
1	4.00	6.80	70.1 %	1
2	3.92	4.84	23.2 %	2
4	3.60	3.75	4.1 %	2
5	3.46	3.48	0.6 %	3
6	**3.33**	**3.33**	0.0 %	3
7	**3.20**	**3.20**	0.0 %	3
8	**3.07**	**3.07**	0.1 %	3
10	2.82	2.88	2.2 %	4
15	2.38	2.63	10.2 %	4
20	2.12	2.45	15.8 %	5
33	1.76	2.21	25.6 %	5

from the other end, why the dictionary is a poor fit for the lag-n_{Tx} residuals of ColoRadar: more than four fifths of those fall in categories 0 to 2 (Table 7.5), the region of the first two rows, where the fixed dictionary's overhead is largest.

B.7.2 The most likely reconciliation

His Table 3.5 was measured on real recordings. In real data the "mean of the noisy part of the signal in dB" can be raised by clutter, leakage, sidelobes and slowly varying returns in the cells the estimator averages, while most cells — which set the compression ratio — sit much lower. In a white-noise simulation the noise floor and the per-cell noise are the same number, so matching his noise floor forces far too much noise into every cell. This reconciliation fits every figure available: his ratio, his dictionary and his histograms. It cannot be confirmed without his data or code, and it is reported as an inference, not a finding.

B.8 LPC-Huffman, the method Kiem excluded

`kr_lpc_raw.m` implements LPC-Huffman as Kiem describes it: a p-th order predictor of each raw ADC sample from the previous p samples of the same chirp, Levinson–Durbin coefficients per chirp and receiver, sent as 32-bit floats, and a Huffman code built from the residuals. The decoder runs sample by sample and the round trip is checked.

On Kiem's own scene, at either noise level, the method he excluded loses to DRHE by a wide

Table B.6: LPC-Huffman on the raw ADC samples of Kiem's scene, by predictor order (CR including the dictionary). Five real sinusoids need up to ten poles.

	−75 dBFS		−87.9 dBFS	
p	CR	residual std (codes)	CR	residual std (codes)
1	1.174	2686	1.176	2660
2	1.172	2682	1.174	2655
4	1.204	2014	1.208	1966
6	1.312	921	1.388	602
8	1.310	899	1.426	506
10	**1.324**	799	**1.543**	318

margin (1.32 against 1.85–2.28; 1.54 against 3.23–3.25), and it stays below 1.8 across the whole sweep (Figure B.2). **For the LPC he meant, Kiem's ranking holds.** It holds for a reason that has little to do with the predictor.

Raw-ADC coding must keep bits the FX16 step has already discarded. Kiem's chain shifts the 14-bit samples left by 18 bits, windows them, takes the FFT divided by N and keeps the top 16 bits. White ADC noise of standard deviation σ codes therefore reaches the FX16 word with standard deviation

$$\sigma_{\mathrm{FX16}} = \sigma \cdot \frac{2^{18}}{2^{16}} \cdot \frac{\sqrt{\sum_k w_k^2/2}}{N} = \sigma \cdot 4 \cdot \sqrt{\frac{0.375}{2 \times 1024}} = 0.0541\,\sigma, \tag{B.1}$$

using $\sum_k w_k^2 \approx 0.375N$ for a Hanning window. The simulation measures exactly this ratio: 375.9 codes become 20.35 LSB and 85.4 codes become 4.63 LSB. The FFT scaling and the truncation to 16 bits thus discard $\log_2(1/0.0541) \approx 4.2$ bits of noise per value before DRHE ever sees the data. A lossless coder of the raw samples must keep them. At −87.9 dBFS even a perfect predictor would leave a residual of at least the ADC noise, 85 codes, whose entropy is about $\log_2(85\sqrt{2\pi e}) \approx 8.5$ bits, capping the ratio near $16/8.5 \approx 1.9$; the prediction filter then amplifies white noise, and the measured ratio is 1.54.

So the comparison Kiem made on paper is between a lossless coder of the ADC stream and a lossless coder of already-truncated data. The truncation is the lossy step, and his Table 3.5 shows its cost in the FX16 row (0.59 % missed, 1.18 % spurious). DRHE inherits that cost; LPC-Huffman on raw ADC does not.

The coefficient method is not the problem. The Levinson–Durbin (autocorrelation) method is biased for pure sinusoids over a finite record; the covariance (true least-squares) method is

not. Replacing one with the other (`kr_lpc_raw_method_check.m`) raises the ratio only when there is almost no noise — from 1.83 to 6.51 at −140 dBFS, $p = 10$ — and changes nothing at the noise levels that matter: 1.56 against 1.59 at −87.9 dBFS and 1.34 against 1.34 at −75 dBFS.

The slow-time LPC is a different matter. On the same scene the LPC of this thesis — same domain and coder as DRHE — gives 2.06 against DRHE's 1.85 at −75 dBFS and 3.38 against 3.23 at −87.9 dBFS, and it stays level with or slightly above DRHE across the sweep from −95 dBFS upwards. On Kiem's scene, the controlled comparison comes out the other way from his remark.

B.9 Adding transmitters

The replication can also do what Kiem's radar cannot: interleave the ramps over twelve transmitters, each at its own position in a virtual array, as in the ColoRadar cascade (`kr_tdm_experiment.m`). Two scenes are used: Kiem's five-target scene, and a denser one that is *not* from Kiem — 300 slowly moving scatterers between −70 and −30 dB of full scale at random angles, a stand-in for a real indoor scene.

Table B.7: DRHE (fixed dictionary) and the other coders with one transmitter and with twelve interleaved, predicting from the previous ramp (lag 1) or from the previous ramp of the same transmitter (lag 12).

Scene, noise floor	radar, lag	DRHE	DRHE+RLE	LPC	no pred.
Kiem, −75 dBFS	1 TX, lag 1	1.850	2.285	2.070	2.036
	12 TX, lag 1	1.836	2.262	2.051	2.036
	12 TX, lag 12	1.859	2.283	2.056	2.036
Kiem, −95 dBFS	1 TX, lag 1	3.726	4.237	3.760	3.603
	12 TX, lag 1	3.641	4.120	3.666	3.603
	12 TX, lag 12	3.694	4.208	3.665	3.603
Dense, −75 dBFS	1 TX, lag 1	1.772	2.203	1.960	1.656
	12 TX, lag 1	1.647	2.066	1.798	1.657
	12 TX, lag 12	1.689	2.110	1.840	1.657
Dense, −95 dBFS	1 TX, lag 1	**3.398**	3.516	3.596	1.952
	12 TX, lag 1	**2.371**	2.514	2.462	1.955
	12 TX, lag 12	**2.560**	2.673	2.712	1.955

On Kiem's scene the transmitters make almost no difference, because there is almost nothing

but noise to predict. On the dense scene at low noise, prediction earns its keep — DRHE 3.40 against 1.95 without prediction — and TDM interleaving costs it a third of its ratio (2.37). Predicting from the same transmitter recovers part of the loss (2.56). The recovery is only partial in this synthetic scene, for two likely reasons: each transmitter's predictor starts from zero, so there are twelve cold starts per frame instead of one; and the synthetic scatterers move, so each lag-12 prediction spans a Doppler step twelve times larger than a lag-1 prediction on a single-transmitter radar. On ColoRadar, whose scene changes slowly, the same change of lag turns a 4.9 % loss against no prediction into a 7.8 % gain (Section 7.2). The mechanism is the same in both: on a TDM-MIMO radar, a slow-time predictor carried over from a single-transmitter design must be told which earlier ramp belongs to the same antenna.

B.10 The DRHE algorithm, line by line

Kiem's Section 3.5.3 specifies the final scheme and his Section 4.3.4 the hardware form of it. Table B.8 takes every element in turn and compares it with this thesis's MATLAB model and its HLS design.

B.10.1 Verification that the algorithm is the same

Agreement on paper is checked by agreement in output, at every link from Kiem's text to the board.

- **Kiem's text to this thesis's MATLAB model.** The replication's DRHE, `kr_drhe_encode.m`, written from Kiem's Section 3.5.3 without reference to this thesis's code, and this thesis's MATLAB DRHE model at lag 1, as a single-transmitter scene requires, were run on the same FX16 frames of the replicated scene. The comparison script, `kr_crosscheck_matlabsim.m`, is the one file of the replication that touches the main code base, and only to read it. They produce *identical* residuals and identical bit counts: 18,186,374 bits at −75 dBFS and 10,370,342 at −87.9 dBFS, a difference of zero in each.
- **MATLAB model to HLS.** The HLS designs reproduce the MATLAB models' bit counts on all 50 ColoRadar frames, apart from the zero padding of the last 256-bit output word (Section 7.6); this holds for the lag-n_{Tx} design and for its lag-1 variant, whose datapath is identical apart from the indexing of the state.

Table B.8: Every element of Kiem's final DRHE scheme (his §3.5.3, §3.5.4 and §4.3.4) against this thesis's MATLAB model and HLS design. The letters in the last column refer to the explanations in Section B.10.2.

Element	Kiem	MATLAB model	HLS design	Same?
Input	FX16 range FFT, first $N/2$ bins, per RX, ramp by ramp	same	same (128-bit AXIS, 4 RX)	yes
Cartesian to polar	$a = \sqrt{re^2 + im^2}$, $\theta = \text{atan2}(im, re)$	same, double	same, single precision	yes; (b)
Magnitude, his Eq. 3.5	$\hat{a}[m] = \alpha\hat{a}[m-1] + (1-\alpha)a[m-1]$	same, at lag L	same, at lag L	yes; (e)
Phase, his Eq. 3.6	$\hat{\theta}[m] = \beta\hat{\theta}[m-1] + (2-\beta)\theta[m-1] - \theta[m-2]$	same, at lag L	same, at lag L	yes; (e)
Phase wrap	once into $[-\pi, \pi]$ (his listing, §4.3.4)	same	same	yes
α, β	0.6, 0.4 (his Table 4.4)	same	same	yes
Polar to Cartesian	rounded to integer (`AP_RND`)	`round`	`roundf`	yes
Difference, his Eq. 3.7	$X - \hat{X}$ in 16-bit two's complement	`wrap_int16`	`wrap_int16`	yes
Initial state	zero (§3.5.4)	zero per frame	zero; each transmitter's state reset on its first ramp of every frame	yes; (c)
State update	from the input sample	from the input	from the reconstructed sample	**changed**; (a)
Run-length code	removed (§3.5.1)	none	none	yes
Size category S_4	his Table 3.7, 0–15	$\lfloor \log_2 \lvert v \rvert \rfloor + 1$, capped at 15	same regions, by comparison	yes
APPEND	v if $v > 0$; $v + 2^{S_4} - 1$ if $v < 0$	counted	same formula	yes
Residual -32768	no code ($S_4 = 15$ spans $\pm[16384, 32767]$); not addressed	counted as $S_4 = 15$	saturated to -32767	**changed**; (a)
Dictionary	fixed, his Appendix A	same 16 lengths	same codes, bit-reversed	yes; (d)
Arithmetic	fixed point: `ap_ufixed<16,16>`, `ap_fixed<16,3>`, `ap_fixed<32,4>`	double	single-precision float	**changed**; (b)
Output packing	58-bit word per channel into 256-bit AXIS	bit count only	512-bit accumulator, 256-bit AXIS	layout; (d)
Prediction lag	ramp $m-1$ (one TX)	$L = n_{\text{Tx}}$; $L = 1$ for one TX	$L = n_{\text{Tx}}$ (lag-1 variant: $L = 1$)	extension; (e)

- **HLS to an independent implementation.** A third implementation, in Python, written from the specification without reusing any code, reproduces the HLS bit count of every frame exactly (Section 5.6).
- **HLS to the board.** The FPGA's compressed output is byte-identical to the C-simulation on all 50 frames (Section 7.6).

The chain from Kiem's specification to the hardware is therefore closed at every link. The DRHE of this thesis is Kiem's DRHE, extended in one documented way (the lag) and changed in implementation in four others, all set out below.

B.10.2 The five places the hardware differs, and why

(a) Saturation of -32768 and the closed loop: exactly lossless on any input. Kiem's Table 3.7 gives category 15 the range $\pm[16384, 32767]$, exactly 2^{15} values filling the 15 APPEND bits, so a residual of -32768 has no code; his text does not address it. It can occur (for example $re = -32768$ with a prediction of zero), and when it does, the APPEND formula aliases it onto $+32767$ (Section 6.4). The HLS design saturates it to -32767. Once a sample has been saturated, an open-loop encoder — one updating its state from the original sample, as Kiem's does — would disagree with the decoder, which can only update from the reconstruction, and because the filters are recursive the disagreement would persist for the rest of the frame. Driving the encoder's state from the reconstruction confines the error to one sample and one LSB. Whenever no saturation occurs the reconstruction equals the input, so the closed loop is *bit-identical* to Kiem's open loop; on ColoRadar the FX16 codes never exceed 437 in magnitude, no residual comes near -32768, and the two produce the same output. The cost is in hardware only: 26 cycles of pipeline depth in floating point, about four in fixed point (Section A.5.3).

(b) Floating-point arithmetic: a choice, and the most expensive one. Floating point follows Kiem's own suggestion for future work (Section 3.9) and keeps the HLS arithmetic close to the MATLAB model. Close is not identical, however: the HLS design computes in single precision and the MATLAB model in double. Their bit counts agree on all 50 ColoRadar frames (Section 7.6), but because the predictor feeds back on itself, any last-bit difference in `atan2` or `sin` accumulates in a decoder: a double-precision decoder drifts by 1 to 6 LSB on 8 of the 50 DRHE frames, and only the decoder built from the HLS source is exact (Section 6.5). Floating point therefore does not remove the requirement Kiem met with an HLS decoder — an exact

decoder must share the encoder's arithmetic bit for bit — and it buys little else. On ColoRadar the floating-point and fixed-point predictors compress to within 0.07 % of each other, while the floating-point datapath costs most of the design's logic, every extra DSP slice and twice the block RAM (Section A.5). The fixed-point variant is the better engineering choice and is the first improvement to make (Section 9.3).

(c) Resetting the state every frame: a clarification. Kiem's model is "initialized with zero"; his MATLAB decodes one frame at a time, and his hardware text does not say what happens between frames. The HLS design resets each transmitter's state on that transmitter's first ramp of every frame, so every frame decodes on its own and a lost frame cannot corrupt the next. The extra write per cycle that this reset costs is what holds the design at $\mathrm{II} = 2$ (Section 6.2); clearing the state by reading it as zero instead reaches $\mathrm{II} = 1$, at a cost measured in Section A.5.4.

(d) Bit packing: the same bits in a different order. The codewords and APPEND fields have exactly Kiem's lengths, so the compressed size is his apart from padding. The codewords are stored bit-reversed because the packer is least-significant-bit first (Section 6.1), and the stream is therefore not bit-compatible with a decoder written for his 58-bit packets; it does not need to be, since this thesis supplies its own decoder. The packer costs about 4,200 LUTs more than his (Section A.5.5).

(e) The prediction lag: an extension. On a single-transmitter radar the previous ramp is the previous observation of the same channel. On the ColoRadar cascade it is a different transmitter eleven times in twelve. The DRHE of this thesis keeps one state per transmitter and predicts from ramp $m - n_{\mathrm{Tx}}$ (Section 4.3); with $n_{\mathrm{Tx}} = 1$ it is Kiem's algorithm unchanged. The lag-1 variant, which predicts from ramp $m - 1$ on the cascade, is what Kiem's algorithm does when applied to this radar without the extension.

B.11 What the replication can and cannot show

It shows that Kiem's workflow, rebuilt from his text, reproduces his DRHE pattern (3.25 with run-length coding, 3.24 without, almost nothing lost to the fixed dictionary) at one noise level; that the ratio of about 1.1 that this thesis's chain gives on synthetic scenes of Kiem's kind is explained completely by the FX16 scaling and the noise level; that the LPC-Huffman he excluded does lose to DRHE, mainly because it codes raw ADC noise that the FX16 step discards; that the slow-time LPC of this thesis does not lose; and that the DRHE of this thesis is his algorithm, to the bit.
It cannot reproduce his absolute numbers, which come from recordings that are not available, and the operating point that matches his CR is chosen, not predicted. The reconciliation of his noise floor with his ratio is an inference. His lossy schemes are approximations, and the detection metrics of a five-target scene say nothing about a 170-detection one.

B.12 Kiem's hardware, rebuilt

The hardware side of the comparison is made in Appendix A, which rebuilds Kiem's compression IP from his description in this thesis's tool flow (Section A.4) and connects it to this thesis's DRHE datapath one design decision at a time (Table A.5). Against that replica every resource difference has one identified cause — above all the floating-point datapath, which a fixed-point variant removes for a change in compression ratio of under 0.07 % — while part of the gap between the replica and Kiem's published LUT and flip-flop counts remains unexplained (Table A.10).

B.13 Every result, compared

Table B.9 lists every result and claim of Kiem's that this thesis can speak to, with this thesis's final figures. The sections after it discuss the ones that need more than a line.

B.14 The compression ratio: 3.25 and 3.75 measure different things

This thesis's DRHE compresses ColoRadar by 3.751 with Kiem's own dictionary (3.75 in Stage 1 over 16 receive channels), and Kiem's compresses his recordings by 3.25. The numbers are of similar size, and it is tempting to read the similarity as agreement or the difference of

Table B.9: Kiem's results and claims against this thesis's. "Class" is the kind of comparison of Section B.1: L like for like, B through a bridge, N not comparable. This thesis's hardware figures are for the final lag-n_{Tx} DRHE compressor (in-system, original dictionary); BRAM in 36 Kb blocks.

Result or claim	Kiem	This thesis	Class	Same?	Why
DRHE algorithm	his §3.5.3	identical residuals and bits	L	yes	Two independent implementations agree to the bit; the hardware differs in five documented places (Section B.10.2).
DRHE CR, simulation	3.25 (real data)	3.75 (MATLAB, 16 RX); 3.751 (C-sim, 4 RX)	N	no	Different data, sensor, scaling and mechanism (Section B.14).
DRHE CR on his scene	—	2.28 at his −75 dBFS; 3.25 at −87.9 dBFS	B	pattern	Reproduces 3.25/3.24 only at a noise level 17 dB below his stated floor (Section B.4).
Run-length coding adds little	3.25 vs 3.24	3.25 vs 3.24 on his scene; ≤1.6 % on ColoRadar	B, L	yes	Holds on both datasets (Section B.15).
Fixed dictionary costs little	3.24 → 3.17 (incl. HW FFT)	0.3 % on his scene at −87.9 dBFS; on ColoRadar a re-derived dictionary gains 12.4 % (DRHE) and 19.1 % (LPC)	B, L	his data only	The dictionary encodes his residual statistics (Section B.16).
DRHE CR, hardware	3.17	3.7508 on the board (lag n_{Tx}); 3.310 for the lag-1 variant (C-sim)	N	no	As above; his hardware FFT is fixed point, while this thesis's cores take MATLAB's FX16 codes.
Hardware output checked against simulation	3.24 (bit-accurate sim.) vs 3.17 (HW)	byte-identical on all 50 frames	—	—	His hardware FFT differs from his simulated one; here the compressor input is identical in both (Section 7.6).
LPC is "the same approach" but less promising	not measured; 2.056 quoted from [27]	fast-time LPC-Huffman: 1.32–1.54 on his scene; slow-time LPC: 3.697 vs DRHE 3.751, 4.404 vs 4.218 with re-derived dictionaries	B	for his LPC only	Two different algorithms share the name (Section B.17).
DRHE is the best of his design space	yes (LPC not in it)	yes against every lossy scheme (BAQ, RDVLE); against LPC it leads slightly with the original dictionary (3.751 vs 3.697) and trails with re-derived ones (4.218 vs 4.404)	L, B	partly	The lossy schemes lose detections (Table 7.1); the two lossless predictors are within a few per cent (Section B.17).
FX16 detection cost	FN 0.59 %, FP 1.18 %	FN 3.77 %, FDR 1.11 %	N	no	Different sensor, scene and processing chain (Section B.18).
Noise floor	−70.7 dBFS	−105.4 dBFS	N	no	Different sensor, reference and integration (Section B.18).
Hardware resources, clock and throughput	45,892 LUT, 9,243 FF, 10 BRAM, 44 DSP; II = 1; 100 MHz; 11.92 Gbit/s (theoretical)	35,444 LUT, 23,528 FF, 40 BRAM, 132 DSP; II = 2; 100 MHz; 5.12 Gbit/s measured	L	no	Every difference is accounted for against a replica of his design, metric by metric, in Table A.15; the main cause is the floating-point datapath.
System latency	79.4 µs	13.1 µs measured on the board	L	same kind	Same definition (last DMA read to last DMA write), but his pipeline includes an FFT core in front of the compressor (Section A.6).
Decompressor	none in hardware	built for every design; on the board, every frame reconstructed exactly (1.97 and 15.4 ms per frame)	—	—	He listed a bit-accurate decompression model as future work (Section 3.9).

15 % as an improvement. Neither reading is valid: the implementation is the same algorithm (Section B.10), but the two ratios are produced by different mechanisms.

ColoRadar's 3.751. Section 7.3 decomposes it. The capture uses 9.77 of the 16 FX16 bits, which is worth a factor of 1.64 before any coding; the sparsity of a range-FFT frame is worth a further 2.12 through the entropy coder, which together give the no-prediction ratio of 3.479; and DRHE's predictor, applied at lag n_{Tx}, adds 7.8 %. Applied at lag 1, as it would be if carried over unchanged from a single-transmitter radar, the same predictor *costs* 4.9 % and the ratio falls to 3.310, below no prediction.

Kiem's 3.25. On his single-transmitter radar the predictor runs on the right axis by construction, and his dictionary — built from his own residuals — is optimal for residuals of 6 to 8 LSB (Section B.7.1). His ratio is what a matched coder gives on residuals of that size; on his own synthetic scene the replication reaches it only when the FX16 noise is about 4.6 LSB.

Why they cannot be compared directly. Beyond the different scenes, four properties of the chain differ between the two datasets, each of which moves the ratio on its own:

- **FX16 scaling.** Both scalings are fixed and independent of the data — this thesis keeps a fixed scaling, as Kiem's design does, for the reason given in Section 7.1.1 — but they are calibrated to different references: this thesis maps the peak of a full-scale sinusoid to 32,767, Kiem's FFT core the full scale of its 32-bit output word, in which that sinusoid peaks at about 8,191. The same signal occupies two more bits here (Section B.6).
- **ADC width and gain staging.** Kiem's CTRX is 14-bit (12-bit in his use case) and his data is shifted by $16 - \mathit{bitWidth} + 16$ bits before the FFT; the ColoRadar capture sits far below the full scale of its FX16 scaling, so its codes peak at 437 and leave six of the sixteen bits unused.
- **Real against complex samples.** His FFT input is real and half the spectrum is redundant; ColoRadar's is complex and the negative half is discarded by choice.
- **One against twelve transmitters.** His ramp axis is slow time; ColoRadar's is interleaved, and the prediction lag has to account for it.

What *can* be said is this. Applied to ColoRadar exactly as specified, at lag 1, Kiem's algorithm gives the lag-1 variant's 3.310, because it is that variant; with the lag set to n_{Tx} it gives 3.751.

Applied to Kiem's scene, this thesis's DRHE gives exactly the ratio of his algorithm as re-implemented from his text, with identical residuals and bit counts at both noise levels at which the cross-check was run (-75 and -87.9 dBFS, Section B.10.1), because with $n_{\mathrm{Tx}} = 1$ it is his algorithm. The comparison of *algorithms* is therefore settled; the comparison of *datasets* cannot be made, and the 4.2 reached with a dictionary re-derived for ColoRadar (Section 7.4) is likewise a property of this capture, not a margin over his 3.25.

B.15 Run-length coding

Kiem removed the run-length half of the original coder after finding that it changed the ratio only from 3.25 to 3.24. The same comparison, with ideal dictionaries in both cases, holds on both datasets available here: 3.25 against 3.24 on the replicated scene at -87.9 dBFS (Table B.3), and on ColoRadar (every fifth frame, four receive channels) 4.340 against 4.273 for DRHE at lag n_{Tx} and 3.466 against 3.457 for the lag-1 variant, a gain of at most 1.6 %. Run-length coding pays only where long runs of exact zeros occur, and neither radar's residuals contain many: even at lag n_{Tx}, fewer than a fifth of DRHE's prediction errors on ColoRadar are zero (Table 7.5). His simplification is sound for this sensor too, and it is what allows every value to be coded independently in one clock.

B.16 The fixed dictionary

Kiem fixed his dictionary on one recording and found the cost small. On his simulated scene at the noise level that reproduces his ratio, the fixed dictionary costs 0.3 % (3.23 against 3.24), as it should on residuals of the size it was built for. On ColoRadar the answer depends on the predictor (Table B.10). For the raw codes and the lag-1 predictors, whose errors are large and spread over many categories, a dictionary re-derived from frames 0–4 and tested on frames 5–49 gains only 3–8 %. For the lag-n_{Tx} predictors, whose errors are much smaller than the ones the dictionary was built for, it gains 12 % for DRHE and 18 % for LPC on the held-out frames (12.4 % and 19.1 % over all 50 frames, Table 7.4).

A dictionary is a property of the data it was trained on, and this one was trained on a different radar. Kiem's conclusion that a fixed dictionary costs little is correct for data like his; the right generalisation of it is that a *fixed* dictionary costs little when it is fixed on the data it will code.

Table B.10: Kiem's fixed dictionary against a dictionary re-derived from frames 0–4, measured on the held-out frames 5–49 (four receive channels, 256-bit padding, LPC coefficients charged).

Design	Kiem's dictionary	Re-derived	Gain
No prediction	3.478	3.720	+7.0 %
DRHE, lag-1 variant	3.309	3.419	+3.3 %
LPC, lag-1,2 variant	3.384	3.640	+7.6 %
DRHE, lag n_{Tx}	3.748	4.207	+12.2 %
LPC, lags n_{Tx}, $2n_{\mathrm{Tx}}$	3.691	4.372	+18.5 %

Because the dictionary is a sixteen-entry table, changing it changes nothing else in the hardware (Section 6.1).

B.17 Linear prediction

Table B.11: Kiem's statement about LPC and what was measured. CRs with Kiem's fixed dictionary unless stated.

Comparison	DRHE	LPC	Source
LPC-Huffman of [27] (fast time, raw ADC) — the method Kiem meant			
Kiem's scene, −75 dBFS, $p = 10$	1.85 / 2.28 (R4S4)	1.32	Table B.2
Kiem's scene, −87.9 dBFS, $p = 10$	3.23 / 3.25 (R4S4)	1.54	Table B.3
Slow-time LPC on FX16 (this thesis)			
Kiem's scene, −75 dBFS	1.85	2.06	Table B.2
Kiem's scene, −87.9 dBFS	3.23	3.38	Table B.3
ColoRadar, lag-1 variant (C-sim, 50 frames)	3.310	3.385	Table 7.3
ColoRadar, Stage 1 (MATLAB, 16 RX)	3.75	3.70	Table 7.1
ColoRadar, lag n_{Tx} (C-sim and board, 4 RX)	**3.751**	3.697	Table 7.3
ColoRadar, lag n_{Tx}, re-derived dictionaries (also board)	4.218	**4.404**	Table 7.4

Table B.11 answers the question with measurements. Kiem's remark is right about the method he meant, and for a reason he did not state: coding raw ADC samples losslessly means keeping about four bits of noise per value that his FX16 step discards (Section B.8). His remark does not carry over to the LPC of this thesis, which predicts in the same domain and along the same axis as DRHE (Section 4.5.6). That LPC is level with DRHE everywhere it was measured: a little ahead on his scene and on ColoRadar at lag 1, a little behind at lag n_{Tx} with his dictionary, and ahead once each algorithm has a dictionary fitted to its own errors. LPC's errors are in fact the smaller of the two throughout — 4.10 against 4.24 bits per value with the original dictionary, before LPC's 0.17 bits per value of coefficients — but a dictionary that cannot code any value

in fewer than four bits hides most of that difference (Section 7.4). The ranking on compression ratio is within a few per cent and depends on the coder; what separates the two algorithms is structure — DRHE streams, LPC needs the whole frame — and that is the basis of the choice made in Section 7.9.

B.18 Detection metrics and the noise floor

Kiem's FX16 row loses 0.59 % of detections and adds 1.18 % spurious ones; this thesis's loses 3.77 % and adds 1.11 % (Table 7.1). Both measure the same thing — the cost of quantising to 16-bit fixed point, since every lossless coder shares FX16's metrics — but on different sensors, scenes and chains, and they are not comparable. Two differences probably dominate. The ColoRadar codes use 9.77 of 16 bits, so the quantisation step is coarse relative to the weakest returns; the quantisation study shows that scaling the codes to fill the container removes almost all of the quantisation error, at the price of almost all of the compression (Table 7.2). And the detection chain here integrates sixteen receive channels over a Doppler FFT taken across transmitter-interleaved ramps (Section 5.2), which spreads each target over several Doppler cells and makes its local maxima more fragile, while matching detections cell by cell with no positional tolerance (Section 5.3.3).

The noise floors (-70.7 against -105.4 dBFS) are not comparable at all: 0 dBFS is not defined in Kiem's thesis, the sensors and gains differ, and NCI over 16 channels and a 192-point Doppler FFT change the level. Within one dataset, the noise floor is useful only for comparing schemes with each other.

B.19 What the comparison says

Same: the DRHE algorithm, to the bit; the finding that run-length coding adds almost nothing; that DRHE beats every lossy scheme in the design space on the combination of ratio and detection; that the datapath meets 100 MHz on his speed grade and fits his device (Appendix A).

Different, and explained: every hardware resource figure and the initiation interval (float datapath, packer, closed loop, state reset, per-transmitter state — each measured in Appendix A); the throughput (a factor of two, once his theoretical binary figure is compared with this thesis's); the value of his fixed dictionary on other data; the LPC ranking, which holds for his LPC and not for this thesis's.

Extended: the prediction lag, which is n_{Tx} on a TDM-MIMO cascade and reduces to his lag of one ramp when $n_{\mathrm{Tx}} = 1$; a decompressor for every design; and verification of the hardware against simulation byte for byte.

Not comparable: the compression ratios and detection metrics on the two datasets. The algorithms are the same; the datasets and their scaling are not, and neither the similarity of 3.25 and 3.75 nor the difference between them says anything about the algorithms.

Unexplained: his published LUT and flip-flop counts exceed what his described design needs by about 2.7 and 1.8 times (Section A.5.5); and his reported noise floor is 17 dB higher than a white-noise scene with his compression ratio would have (Section B.7). The second most likely reflects the difference between a real noise floor and a simulated one.

Appendix C

The Huffman Dictionaries

Table C.1 gives the complete original dictionary [11, Table A.1], used in Stage 1, in the no-prediction baseline and in the original-dictionary board builds. The published codes are most-significant-bit first; the implementation packs least-significant-bit first, so the stored codewords are bit-reversed. Both forms are listed, and all sixteen entries were verified to agree with the published table.

The reversed code set remains prefix-free, which is what allows the decoder to match candidates from the low end of its bit buffer with no ambiguity.

Table C.1: The original Huffman dictionary. "Cost" is the total bits to code one residual in that category, $\ell + S_4$, from Equation (4.7).

S_4	$\lvert v \rvert$ range	ℓ	code (MSB-first)	stored (LSB-first)	cost
0	$v = 0$	4	`1010`	`0x0005`	4
1	1	3	`100`	`0x0001`	4
2	2–3	2	`01`	`0x0002`	4
3	4–7	2	`00`	`0x0000`	5
4	8–15	2	`11`	`0x0003`	6
5	16–31	5	`10110`	`0x000D`	10
6	32–63	6	`101110`	`0x001D`	12
7	64–127	7	`1011110`	`0x003D`	14
8	128–255	8	`10111110`	`0x007D`	16
9	256–511	9	`101111110`	`0x00FD`	18
10	512–1023	10	`1011111110`	`0x01FD`	20
11	1024–2047	11	`10111111110`	`0x03FD`	22
12	2048–4095	14	`10111111111111`	`0x3FFD`	26
13	4096–8191	14	`10111111111110`	`0x1FFD`	27
14	8192–16383	13	`1011111111110`	`0x0FFD`	27
15	16384–32767	12	`101111111110`	`0x07FD`	27

Note the non-monotonicity at categories 12–15: the longest codewords are assigned to categories 12 and 13 rather than to 15. This is a property of the published dictionary and is reproduced faithfully. It has no effect on this dataset, where no residual exceeds category 10.

The re-derived dictionaries

Table C.2 gives the codeword lengths of the dictionaries re-derived for each algorithm (Section 4.6), built from the S_4 counts of frames 0–4 with one added to every category, and the total cost of each category with each. The codewords themselves are the canonical Huffman codes for these lengths, bit-reversed for the packer; they are the tables `DRHE_TDM_HUFFMAN_TABLE` and `LPC_TDM_HUFFMAN_TABLE` in the HLS source.

Table C.2: Codeword length ℓ and total cost $\ell + S_4$ per category for the original and the two re-derived dictionaries.

	original		DRHE		LPC	
S_4	ℓ	cost	ℓ	cost	ℓ	cost
0	4	4	3	3	3	3
1	3	4	2	3	1	2
2	2	4	1	3	2	4
3	2	5	4	7	4	7
4	2	6	5	9	5	9
5	5	10	6	11	6	11
6	6	12	7	13	7	13
7	7	14	8	15	8	15
8	8	16	9	17	9	17
9	9	18	10	19	10	19
10	10	20	11	21	13	23
11	11	22	14	25	13	24
12	14	26	14	26	13	25
13	14	27	13	26	13	26
14	13	27	13	27	12	26
15	12	27	13	28	12	27

The re-derived dictionaries code their commonest category in three bits (DRHE, category 2) or two (LPC, category 1), where the original cannot code anything in fewer than four. They are longer above category 3, which costs little because those categories are rare after prediction.

Appendix D

Bitstream Layouts

D.1 DRHE

DRHE's bitstream is a single continuous sequence of coded residuals, in the raster order the samples arrive:

$$\underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})}_{\text{ramp 0, bin 0, ch 0}}\ \underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})}_{\text{ramp 0, bin 0, ch 1}}\ \cdots$$

where $\mathrm{enc}(v)$ is the Huffman codeword for $S_4(v)$ followed by $S_4(v)$ APPEND bits. There is no header: the frame dimensions arrive over the AXI4-Lite control interface. The final AXI word is zero-padded.

D.2 LPC

LPC's bitstream has two sections:

$$\underbrace{\left[a_1^{\Re}\ a_2^{\Re}\ a_1^{\Im}\ a_2^{\Im}\right]_{\text{ch,bin}}}_{\text{coefficient table, } 4\times16 \text{ bits each}}\qquad\underbrace{\mathrm{enc}(d_{\Re})\ \mathrm{enc}(d_{\Im})\ \cdots}_{\text{residuals, raster order}}$$

The coefficient table comes first because the decoder needs all of it before it can reconstruct anything. The residuals follow in the original raster order — not in the order the compressor computed them — which is what allows the decompressor to operate without a frame buffer (Section 6.3).

Appendix E

Reproducing the Results

All sources are in the thesis repository under `ThesisA/`. The build tree must sit on a path without spaces, which Vitis rejects.

E.1 Source files

File	Purpose
MATLAB (`Matlab_Sim/`)	
`loadColoRadarFrame.m`	reads a frame and applies the TDM flattening
`preprocessing.m`, `rangeFFT.m`	DC removal, window, range FFT
`compress_fx16.m`	FX16 quantisation, Equation (2.11)
`compress_drhe.m`, `decompress_drhe.m`	DRHE; lag from `config.predLag`, default n_{Tx}
`compress_lpc_huffman.m`, `decompress_lpc_huffman.m`	LPC at lags L, $2L$
`huffman_tables.m`, `huffman_code_lengths.m`	the original and re-derived dictionaries; retraining
`run_coloradar_benchmark.m`	Stage 1: all eight methods with detection metrics
`make_poster_figs.m`	Stage 2 breakdown, dictionary tables, and the figures of Chapter 7; parameters at the top
`export_frames_for_vitis.m`	writes the `int16` stimulus for the HLS testbenches
HLS (`hls_component/`)	
`drhe_tdm_common.h`, `drhe_tdm_compress.cpp`, `drhe_tdm_decompress.cpp`	DRHE cores and the DRHE dictionary
`lpc_tdm_common.h`, `lpc_tdm_compress.cpp`, `lpc_tdm_decompress.cpp`	LPC cores, Yule–Walker solver and the LPC dictionary

`*_tb.cpp`, `*_tb_edge.cpp`	50-frame and edge-case testbenches
Board (`fpga_kcu116/`)	
`hls/export_ips.bat`	C-simulation (writes the golden streams) and IP packaging
`vivado/build.bat`	block design, synthesis, implementation, bitstream; `decomp 1` adds the decompressor
`sw/src/main.c`, `board_map.h`	MicroBlaze firmware
`host/run_board.bat`, `run_board.tcl`	program the board, load frames, run, read results
`host/pc_decompress.bat`	PC decompression with the HLS decoder source
`host/compare_results.py`	byte comparison with C-simulation, reconstruction check, tables
Independent check (`verify_indep/`, Python)	
`indep.py`	the whole chain from raw frame to bit count
`retrain_complete.py`	dictionaries trained on frames 0–4, tested on 5–49
`golden_check.py`, `independent_decode.py`	residual-level check and decoding of the hardware bitstreams

E.2 Commands

C-simulation, synthesis and co-simulation of a core (DRHE shown):

```
vitis-run --mode hls --csim  --config test_tdm.cfg --work_dir hls_tdm
v++ -c    --mode hls         --config test_tdm.cfg --work_dir hls_tdm
vitis-run --mode hls --cosim --config test_tdm.cfg --work_dir hls_tdm
```

Board run, from `fpga_kcu116/host` (`<tag>` is `drhe`, `lpc`, `drhe_full` or `lpc_full`):

```
run_board.bat <tag> 50
pc_decompress.bat <tag>          (compress-only builds)
python compare_results.py <tag>
```

Figures and tables of Chapter 7, in MATLAB from `Matlab_Sim`: `make_poster_figs`. Simulation results are cached; set `P.recompute = true` after changing the algorithm code.

Appendix F

Verification Record

Table F.1 lists every check on which the results rest, where it was made, and its outcome.

Table F.1: The verification record.

Check	Where	Outcome
Stage 1, eight methods, 50 frames	MATLAB	Table 7.1; re-run reproduces every cell
DRHE and LPC lossless, original dictionary	HLS C-sim, 50 frames	MaxDiff 0 on every frame
DRHE and LPC lossless, re-derived dictionaries	HLS C-sim, 50 frames	MaxDiff 0; CR 4.21704, 4.40345
MATLAB against HLS bit counts	both	equal up to the last 256-bit padding word
Independent bit counts, original dictionary	Python, 50 frames	equal to HLS on every frame
Independent decode, re-derived dictionaries	Python	no residual mismatch
Edge cases, eight synthetic inputs	HLS C-sim	all pass within their allowed MaxDiff
RTL behaviour	co-simulation, 1 frame	MaxDiff 0, both algorithms
Timing closure, four builds	Vivado	all met at 100 MHz (Table 6.1)
Compressed bytes against C-sim, original dictionary	KCU116, 50 frames	byte-identical, both algorithms
Decompression of board output	PC, 50 frames	MaxDiff 0, both algorithms
Compressed bytes against C-sim, re-derived dictionaries	KCU116, 50 frames	byte-identical; CR 4.21704, 4.40345
On-chip decompression (`_full`)	KCU116, 50 frames	MaxDiff 0, both algorithms; 1.97 and 15.4 ms per frame

Check	Where	Outcome
DRHE matches Kiem's specification	rebuilt simulation	bit-identical output